AutoExam.ai

Final Year Project – Mid Report

Session 2023-2027

A project submitted in partial fulfillment of the
requirements for the Degree

of

BS in Software Engineering



Department of Computer Science

COMSATS University Islamabad (CUI), Lahore Campus

Project Details

Project ID (for office use)	
Type of project	[✓] Traditional [ ] Industrial [ ] Continuing
Nature of project	[✓] Development [ ] Research & Development
Sustainable Development Goals(SDGs)	[ ] Good Health and Well-Being [✓] Quality Education [ ] Industry, Innovation, and Infrastructure [ ] Gender Equality [ ] Decent Work and Economic Growth [ ] Climate Action
Area of specialization	[ ] Artificial Intelligence (AI) [ ] Blockchain [ ] Cybersecurity [ ] Data Science and Analytics [ ] Game Development [ ] Internet of Things (IoT) [ ] Natural Language Processing (NLP) [ ] Mobile App Development [ ] Web Development

Project Group Members

Sr.#	Reg. #	Student Name	Email ID	Signature
(i)	FA22-BSE-204	Hammad Zubair	Fa22-bse-204@cuilahore.edu.pk	
(ii)	SP23-BSE-109	Muhammad Ahsan Junaid Amin	Sp23-bse-109@cuilahore.edu.pk	
(iii)				

Declaration: The candidates confirm that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.

Plagiarism Free Certificate

This is to certify that, I am _____ S/D/o _____, group leader of FYP under registration no CUI /_____/LHR at the Computer Science Department, COMSATS University Islamabad, Lahore Campus. I declare that my FYP proposal is checked by my supervisor and the similarity index is _____% that is less than 20%, an acceptable limit by HEC. The report is attached herewith as Appendix A.

Date: _____ Name of Group Leader: _____ Signature: _____

Name of Supervisor: _____ Co-Supervisor (if any):_____

Designation: _____ Designation: _____

Signature: _____ Signature: _____

HoD: _____ Signature: _____

ABSTRACT

Education is a pillar that the country is supposed to be made on, but in Pakistan, there are some very deep cracks on the pillar. Literacy rates are estimated at 60 percent and the real situation in some regions is even less than the figures. Students are dropping out at early ages, the gap in skills continues to increase, and teachers who are already overworked are struggling to get by with pre-modern tools and techniques. Assessment is one of the largest, yet least discussed issues of this entire picture. The exams are meant to achieve three functions, to test what students know, to assist them in realizing their position and to provide the teachers with an indicator on the quality of their lessons. In reality none of these three things occur in a reliable manner. The teachers waste hours setting papers manually and most of the time they end up repeating same questions in subsequent sessions without being aware. Judgment personal, fatigue, workload all creep in the process in a way that is detrimental to consistency and fairness. An educator with a five-subject load in three classes just cannot find the time or bandwidth to develop well-calibrated, outcome-reviewed exams papers on a semester basis. All the other aspects have transformed with the digital age: industries, communication, trade, but evaluation in most Pakistani schools remains identical to how it was two decades ago. Such technologies as Natural Language Processing and Large Language Models are currently sophisticated enough to read course material, comprehend it, and produce valuable questions based on it. That capability exists. The segregating area is that no one has ever constructed something useful, affordable and in fact, adapted to local educational setting. That is the very gap that AutoExam.ai is attempting to cover.

Acknowledgement

We would also like to take this opportunity to thank our supervisor to their continued guidance, constructive feedback and since support through the process of developing this project. We also owe the Department of Computer Science COMSATS University Islamabad, Lahore Campus, the resources and the academic environment so that we can do this work.We also recognize the open-source communities that make this project technically possible, i.e., React.js, FastAPI, Supabase, OpenAI, and Hugging Face.

Table of Contents

Chapter 1. Introduction 9
1.1 Introduction ..... **Error! Bookmark not defined.**
1.2 Problem Statement 9
1.3 Proposed Solution ..... **Error! Bookmark not defined.**
1.4 Main Objectives 13
1.5 Assumptions & Constraints 14
1.6 Project scope 14
1.7 Software Development Lifecycle Model 15
Chapter 2. Requirement Analysis 17
2.1 Literature Review 17
2.2 Stakeholders List 20
2.3 Requirements Elicitation 21
2.3.1 Functional Requirements ..... **Error! Bookmark not defined.**
2.3.2 Non-functional Requirements ..... **Error! Bookmark not defined.**
2.3.3 Requirements Traceability Matrix ..... **Error! Bookmark not defined.**
2.4 Use Case Description 25
Chapter 3. System Design 32
3.1 Use Case Design 32
3.2 Activity Diagram ..... **Error! Bookmark not defined.**
3.3 Sequence Diagram 43
3.4 Software Architecture Diagram ..... **Error! Bookmark not defined.**
3.5 Class Diagram 52
3.6 Database Diagram 53
3.7 Collaboration Diagram 54
Chapter 4. System Testing 55
4.1 Test Cases Design 55
4.2 Unit / Integration / Acceptance Testing 81
Chapter 5. Implementation 86
5.1 Work Breakdown Structure (WBS) 87
5.2 Team Roles and Responsibilities 87
5.3 Tools and Technologies 88
5.4 Implementation Details 89
5.5 Screenshots of Prototype / System 89
5.6 Challenges During Implementation ..... **Error! Bookmark not defined.**
Chapter 6. References ..... **Error! Bookmark not defined.**

List of Tables

Table 1 Functional Requirments 22
Table 2 Non-Functional Requirments 23
Table 3 Requirements Tracebility Matrix 24
Table 4 TC-01-01 55
Table 5 TC-01-02 55
Table 6 TC-01-03 56
Table 7 TC-01-04 57
Table 8 TC-01-05 57
Table 9 TC-01-06 58
Table 10 TC-01-07 58
Table 11 — TC-02-01 59
Table 12 — TC-02-02 60
Table 13 — TC-02-03 60
Table 14 — TC-02-05 61
Table 15 — TC-02-06 61
Table 16 --TC-03-01 62
Table 17— TC-03-02 63
Table 18— TC-03-03 63
Table 19— TC-03-04 64
Table 20 — TC-03-05 64
Table 21 TC-04-01 65
Table 22: TC-04-02 66
Table 23 : TC-04-03 66
Table 24 : TC-04-04 67
Table 25 : TC-04-05 67
Table 26 : TC-05-01 68
Table 27 : TC-05-02 69
Table 28 : TC-05-03 69
Table 29 : TC-05-04 70
Table 30 : TC-05-05 70
Table 31 : TC-05-06 71
Table 32 : TC-06-01 72
Table 33 : TC-06-02 72
Table 34 : TC-06-03 73
Table 35 : TC-06-04 74
Table 36 : TC-07-01 74

Table 37 : TC-07-02 75
Table 38 : TC-07-03 76
Table 39 : TC-07-04 76
Table 40 : TC-08-01 77
Table 41 : TC-08-02 78
Table 42 : TC-08-03 78
Table 43 : TC-09-01 79
Table 44 : TC-09-02 79
Table 45 : TC-09-03 80
Table 46 : TC-09-04 81
Table 5.1. Individual Tasks 88

List of Figures

Figure 1 Use Case Diagram for Authentication & Access Control Module 33
Figure 2 Use Case Diagram for Content Upload & Processing Module 33
Figure 3 Use Case Diagram for Exam Generation Module 34
Figure 4 Use Case Diagram for Exam Management Module 34
Figure 5 Use Case Diagram for Exam Attempt Module 34
Figure 6 Use Case Diagram for Evaluation System Module 35
Figure 7 Use Case Diagram for AI Detection Module 35
Figure 8 Use Case Diagram for Feedback System Module 35
Figure 9 Use Case Diagram for Analytics Dashboard Module 36
Figure 10 Activity Diagram for Authentication Workflow 36
Figure 11 Activity Diagram for Content Upload & Processing Workflow 37
Figure 12 Activity Diagram for Exam Generation Workflow 38
Figure 13 Activity Diagram for Exam Management Workflow 39
Figure 14 Activity Diagram for Exam Attempt Workflow 40
Figure 15 Activity Diagram for Evaluation System Workflow 41
Figure 16 Activity Diagram for AI Detection Workflow 42
Figure 17 Activity Diagram for Feedback Generation Workflow 43
Figure 18 Sequence Diagram for User Login & Authentication Flow 44
Figure 19 Sequence Diagram for User Registration & Verification Flow 45
Figure 20 Sequence Diagram for Content Upload & Processing Flow 46
Figure 21 Sequence Diagram for Exam Generation Flow 47
Figure 22 Sequence Diagram for Exam Management & Publishing Flow 48
Figure 23 Sequence Diagram for Exam Attempt Flow 49
Figure 24 Sequence Diagram for Evaluation & AI Detection Flow 50
Figure 25 Sequence Diagram for Feedback & Analytics Flow 51
Figure 26 Software Architecture Design Diagram 52
Figure 27 Class Diagram 52
Figure 28 DataBase Diagram 53
Figure 29 DataBase ERD Diagram 53
Figure 30 Collaboration Diagram 54
Figure 31 --WorkBreakDown Structure 87
Figure 32--GANTT Chart 88

Chapter 1. Introduction

1.1 Introduction

Intro

- Domain Background:

The education forms the core of any operational society and the quality of education is directly associated with the quality of the way students are tested during their learning process. Assessment has never been given the due it deserves in Pakistan however. The official national literacy rate is about 60 percent, although the real situation in the fields shows an even more dire picture because in rural and semi-urban regions, after years of school education, many learners do not know how to read simple texts or solve simple problems. It is not much of a resource issue - at least it is a feedback issue. Students continue with semester after semester without even having a clue of the gaps in their knowledge as the system was not even created to do so.

The process of examination in the majority of Pakistani institutions is the one which has not been altered over decades. A teacher will sit down, normally very near to the test, and handwrite a paper, depending on what they can recall, on habit, and what time they have remaining following their teaching task. No one follows a systematic arrangement to ensure that questions are not biased in covering all subjects, difficulty is not skewed inappropriately, or that the paper is testing what was taught in the classroom. The questions asked are repeated in different sessions and sometimes the same questions appear word to word and those students who have access to past papers have an unfair advantage over others. When the exams are marked back, the students receive a number - perhaps some red mark on their answer paper, but seldom a formal explanation of what they got wrong and why.

The issue becomes exponential. One teacher at a Pakistani university may have three or four courses at a time, each course has its own exam, and its own list of student submissions to assess. Extensive thoroughness is a luxury in such an atmosphere. The grading process becomes quicker and less attentive when there is fatigue and the feedback that is given to the students becomes less valuable the larger the number of students. This is not an indictment against the teachers, it is a structural issue that there is no personal endeavor that can counter it completely without the proper equipment.

- Motivation & Significance:

We decided to work on AutoExam.ai because this issue is not only extremely familiar but also not actually solved at the place where it is most important the classroom.

Students always respond well to consistent, direct, and prompt feedback, which has been proven to be one of the most effective forms of promoting learning among students. In their most cited publications on feedback, Hattie and Timperley discovered that it was among the most impactful effects on academic performance with proper delivery [1]. However, in the Pakistani context, such kind of feedback is virtually never provided, not because teachers do not wish to do so, but because it is not possible to provide such feedback manually to dozens or even hundreds of students. The difference between what the research tells us to be working and what actually occurs in the classrooms is immense and that is what inspired this project.

In addition to the feedback, there has also arisen a new and urging challenge which no one was ready to face due to the emergence of generative AI tools. ChatGPT and similar systems were not well accessible two years ago and most institutions have simply had no time to build any form of policy or detection infrastructure around them. Now students are able to write any essay or short-answer question in a few seconds, and present it as their own work, written in a correct grammar, having

the topical theme, and being well written. Teachers that peruse these answers are in many cases unable to distinguish them, as the writing is good and the content is believable. This essentially nullifies the intent of assessing in writing, which is to determine whether the student indeed comprehends what it is. No such exam platform handles the issue, particularly in a manner that is systematically incorporated into the grading process, which is what made the issue one of the main reasons why we built AutoExam.ai the way we did.

Another thing that we aimed to know, through this project, is how we can apply NLP and Large Language Models to a practical educational problem, instead of a benchmark dataset. It is one thing to read about prompt engineering in a research paper and another to go out there and construct a pipeline around it, practice it on actual course material, and ensure that it behaves in a manner that is repeatable. It is that practical difficulty of passing the chasm between the force of AI and software that should be used, that it is that we hoped would be cultivated in the work.

- Objectives & Scope:

AutoExam.ai will be best achieved by establishing a functioning, final product prototype of an intelligent exam system that will automate the aspects of assessment that are not supposed to be automated and leave the teachers with control of the aspects that should be automated.

The objectives of the project are specific and these are:

- To create an Artificial Intelligence-based question generation pipeline that processes raw course content and generates structured and curriculum-aligned question formats on multiple types and cognitive levels.
- To create a semantic evaluation engine that is able to score short-answer and essay responses with similarity-based and LLM-assisted methods of grading.
- To add an AI Answer Authenticity Detection feature that compares written answers to the GPTZero API and marks the probable AI-generated answers as such to be reviewed by the teacher.
- To develop and provide a customised feedback mechanism providing the students with topic-based performance disaggregation, strength and weaknesses identification and advise them on what is needed to learn further.
- To develop a comprehensive, web-based test platform with teacher and student role-based access, a developed on a current and sustainable technology stack.

The scope of this project will involve supporting three types of questions, namely MCQs, short-answer questions, and essays difficulty balancing with the help of the taxonomy of Bloom, delivery of timed exams via a web interface, automated scoring of each type of questions, AI-assisted detection of answers to written questions, and personalised feedback after an exam.

Out of scope is multi-language support, identity based proctoring e.g. facial recognition and browser lockdown, institution wide analytics beyond individual exam performance, mobile app development, and non-functional optimisations e.g. load balancing or enterprise level security hardening. These omissions are intentional - having the scope narrowed means that the team will have something truly functional and properly tested within the FYP deadlines instead of being thinly spread across too many features.

- Existing Solutions and weaknesses.

The idea of the NLP in terms of question-making in exams is not novel. A number of research groups including rule-based syntactic transformations to the most recent generation of transformers, including T5 and BART, study the automated generation of text-based questions [2][3]. Ghuge et al. developed a question generation system that was built around the Bloom Taxonomy and pre-processed using the NLP and rule based templates that performed well on the lower level of cognitive

difficulties but were incapable of producing analytical and evaluative questions [2]. This is also applied to other work, based on NLP pipelines, and filtering with KNN, to generate short-answer questions by reading passages, but is designed to be applied during an exercise of comprehension, rather than during formal examination [3].

In the business sector, quizzes and low-stakes assessment platforms like Google Forms, Quizlet and Kahoot are a common but also do not create questions on their own based on course materials and do not provide any meaningful analysis of written response. Turnitin is a document-level plagiarism detector, also, that is not based on the exam workflow in any way and is not applicable in detecting AI-generated answers in the framework of specific exam responses on a real-time basis. These tools, which openAI offers, can ask questions with a command, although these tools have not yet been packaged into a convenient examination platform that a teacher cannot simply upload a PDF and get a ready-to-publish exam paper.

The gap between all of these available approaches is the same: none of them can deal with the whole lifecycle. They do generate queries but cannot make a judgment, or make judgment but cannot generate or recognize cheating and not AI action in test scores in specific instances. AutoExam.ai strives to eliminate this distance with a single place that consists of all these, but which is built upon workflow unique to a teacher and student.

- Prposed Solution:

AutoExam.ai solves the mentioned issues by deploying a twelve-step technical pipeline encircled within a role-based web application that is clean.

The first step a teacher does is to upload his or her course content - it could be a PDF textbook, a plain text set of notes or a web address to online content. The system finds and filters the text, divides the text into segments based on topics using sentence embeddings and topic modelling and then runs each segment through a question generation pipeline that runs on GPT models of OpenAI. Questions are created in three formats: MCQs including distractors, short-answer conceptual-based questions, and essay prompts and filtered based on quality, topicality, and cognitive challenge and finally assembled into a structured exam paper. Any question can be reviewed, edited or deleted by the teacher before it is published which keeps the human judgment in the loop until the final exam.

After students take the exam, they do it by using a timed interface to a browser which automatically saves their responses. MCQs are also automatically scored by matching, after submission. Answers in written form are evaluated by a semantic evaluation engine that compares the student answer with a model answer on embedding similarity, key word matching and LLM aided rubric scoring. After scoring, all written responses are sent to AI Answer Authenticity Detection, which invokes the GPTZero API in order to calculate a perplexity and burstiness score of the text. When the probability of the returned AI is above 70 percent, it is considered a flag in the database, thus alerting a teacher with a special toggle on his or her dashboard. Lastly, the system creates an individualised report of feedback to the students, which disaggregates their performance by subject, identifies some strengths and weaknesses, and gives them particular areas to review before the next test.

All of this is backed by a technology stack comprising React.js and TailwindCSS on the frontend, Node.js or FastAPI on the backend, PostgreSQL through Supabase to store it, and OpenAI GPT models with Hugging Face Transformers to drive the AI engine.

- Results & Outcome:

The efficacy of this project will be looked at on the basis of clear, practical-based criteria other than the vague ambitions.

By the conclusion of the project, the system must be capable of accepting a piece of course material and generating an exam paper with accurately typed and applicable questions of at least two questions types with the difficulty levels spread across the levels of Bloom. It must be so that a student can take that test using the web interface and provide responses and have a feedback report in a decent time without the manual intervention of the teacher. The AI detection module must identify a set of ChatGPT generated sample answers with a rate more than 80 percent and false positives on the human-written answer with less than 20 percent - these values will be tested at the QA stage on a set of test submissions. The teacher dashboard is supposed to show the results of the detection in a manner that can be understood by a non-technical teacher in order to take the necessary action without training.

The indications to attain these objectives will be in the form of working software, test case outcomes, sample exam outputs and a small user acceptance test with sample teachers and students. By having all these requirements met and the prototype tested as valid by the supervisor, then the project will have achieved what it claimed to accomplish. In addition to the academic necessity, a system like this does have real potential of adoption in institutions where commercial assessment tools currently lack budget allocation - and that is one of the reasons why the work is worth doing.

1.2 Problem Statement

The current method of examination in our educational institutions is paper and pencil, and structurally incapable of scaling to fulfill the needs. Teachers spend inappropriate time assignments of papers that are not always well aligned to learning outcomes, repeated during the sessions, and sometimes have unwanted bias due to personal judgment and exhaustion. Instead, students get minimal or no practical feedback once they have been graded and their gaps in knowledge remain unaddressed semester after semester. In other regions of the world, automated assessment tools are available, but these are either prohibitively costly to the local institution, or too generic and thus cannot work with custom course content or, too limited in scope to span the entire examination lifecycle between generation and feedback. All of these solutions ignore the fact that the threat of AI-generated exam answers is increasingly growing - a phenomenon that has also emerged quite fast due to the prevalence of tools such as ChatGPT and no integrated classroom-ready solution is available yet. The outcome is an evaluation system that fails teachers by overloading them, fails students by informing them less, and has a complete new integrity problem it has no means to address.

1.3 Proposed Solution

The given issue has a multi-layered core, which includes low quality of assessment, absence of student feedback, and the inability to identify AI-generated responses, and focusing on one of them would not allow to eliminate others. AutoExam.ai is meant to be a single, unified system that addresses the three collectively, but builds around the real workflow of a teacher and a student as opposed to what is convenient to construct as a technical matter.

The initial level is content ingestion. A teacher stores his or her course materials in any format that he or she may possess - a PDF, plain text notes, or a web address of online information. The system identifies and purifies the text, filters out noise, divides it into topic-based units by sentence embeddings and topic modelling. This division is significant, as it provides making sure that questions are not simply created based on the most outstanding aspects of the document, but it is spread in a meaningful way across the entire range of the topics that the material addresses.

After the content is divided, it is fed into a pipeline of question generation, which is driven by GPT models of OpenAI. The pipeline generates three kinds of questions (MCQs with wrong-option

distractors, short-answer conceptual questions and essay prompts) and uses the Bloom Taxonomy to allocate them in various cognitive difficulty levels (basic recall, up to analysis and evaluation). Quality filter then focuses on each question generated to make sure it is relevant, grammatically correct, not repeated and meets cognitive level requirements before it is included in the final exam. The teacher has the ability to look through the output, edit it, and publish the exam when the teacher is satisfied and leave the human judgment in the loop in which it counts.

The students take the exam in a timed, non-proctored desktop and mobile browser-based interface. MCQs are automatically scored by matching using the rules after submission. Written responses are processed by a semantic evaluation engine that compares responses to the model answer (through embedding based similarity, keyword matching and LLM based rubric grading) to generate a score that represents understanding and not only the presence of keywords.

Once scored, every written work is sent to the AI Answer Authenticity Detection module. This module makes a call to the GPTZero API, which evaluates the text on perplexity and burstiness two statistical characteristics that vary substantially in human-written versus AI-generated text. In a case where the probability of AI involvement, as the returned probability, is greater than 70 percent, the submission is flagged in the database and the teacher notified by a well-noticed toggle on his or her dashboard. The threshold can be adjusted, allowing the teachers to control the strictness of the system in applying the check.

Lastly, the system will produce customised feedback report to each student. This report decomposed their performance according to their topics, what they had mastered, where they had failed, and what areas they need to work on before the next test. It is the section in the platform that directly contributes to filling the gap of feedback that makes students have no actual guidance post-exam.

The technology stack selected on this platform portrays an equilibrium between the capability and practicality. The frontend is done in React.js and TailwindCSS, the backend is done in FastAPI or Node.js and the database layer is done in PostgreSQL via Supabase, making it easily manageable and scalable. The AI engine is developed based on OpenAI GPT generation and evaluation models, and Hugging Face Transformers can be used in the cases of fine-tuning tasks. GPTZero supports AI identification by a free tier API, which can be easily integrated. The complete system can be deployed to a cloud platform, such as Vercel and Render, so the institution does not need any server infrastructure to use the system.

The difference between this solution and one that is currently in place is not a single feature but the combination. Generation of questions, semantic analysis, AI-answer recognition and customised feedbacks have never been merged in a single platform that is tailored to the particular resource-limited, large student numbers environment that is characteristic of most Pakistani schools. AutoExam.ai is designed in such a manner that it can be used by a teacher without any technical background, will be capable of supporting a large university cohort and will actually be affordable to actually implement.

1.4 Main Objectives

- To come up with web based platform that enables teacher to post course content and simple generation of structured exam papers that are curriculum based and not with manual effort.
- To create a question generation pipeline with AI to generate MCQs, short-answer, and essay questions and which is distributed on the cognitive difficulty levels of the Bloom Taxonomy using Largenet Language Models.
- To create a semantic evaluation engine producing written exam scoring based on embedding-based similarity scoring, keyword matching and LLM-assisted rubric scoring.
- To implement an Authenticity Detection AI Answer Detection method to check written submissions through GPTZero API and mark the text as having a high likelihood of being generated by an AI to examine it by the teacher.

- To create a guest-based feedback mechanism, which generates topical performance breakdowns, determines student strengths and weaknesses, and gives specific study recommendations following each examination.
- To create a role based web application where the teacher and student dashboards are separated to support the entire examination lifecycle where contents are uploaded all the way to delivery of results.

1.5 Assumptions & Constraints

Assumptions:

- It is assumed that those teachers whose work involves the use of the platform will be basic computer literate and will be allowed to have a stable internet connection that will suffice to engage in web-based interactions.
- The assumption is that the course material uploaded by instructors will be in English since the NLP pipeline and the LLM models to be used in this project are trained and optimised to process English language data.
- It is thought that uploaded course materials will be fairly well-organized and will include sufficient meaningful text on which the system will extract topics and use them to create appropriate questions.
- The free or low-cost tiers of these two APIs, GPTZero API and OpenAI API, are assumed to be free and operational at the stages of the project development and testing.
- The prototype testing is based on the assumption that during the prototype testing, students are expected to behave honestly during examinations, and the AI detection module should be adjusted to a real-life combination of human-written and AI-generated responses.
- The Supabase free tier is assumed to be good enough in terms of the database capacity and the request limits to accommodate the prototype and its testing cycle without having to upgrade to a paid service.

Constraints:

- This project should be done within the FYP time frame between February 2026 and February 2027, meaning that not every feature can be fully implemented and tested with that time frame.
- GPTZero free tier has a rate limit of around 10 requests per minute, which implies that the AI detection unit will have to wait until all requests have been served and cannot analyze large quantities of submissions at the same time.
- The OpenAI API has a paid usage model, and the cost will be controlled by restricting the use of tokens via the close design of prompts and the default model is GPT-3.5 unless GPT-4 is needed explicitly.
- This version of the system will support only English-language content and not Urdu or any other regional language because of the constraints of the underlying NLP models.
- The platform will be running on free-tier cloud computing providers like Vercel and Render that will have limits on compute and bandwidth that can influence response times with large concurrent load.
- The application is not mentioned as a specific mobile app, and is limited to a responsive web front so the experience of the students on smaller screens will require a browser-based, as opposed to an actual native application.

1.6 Project scope

- **InScope:**

AutoExam.ai is designed to cover the complete examination lifecycle for educational institutions, targeting two primary users — teachers and students. The following features and functionalities fall within the scope of this project:

- The system will be able to accept course materials in form of text, PDF and URL and analyze them to generate meaningful information to use in question generation.
- The system will automatically create three categories of exam questions– MCQ with distractor choice, short-answer conceptual, and essay prompts will be distributed throughout the Bloom Taxonomy levels of cognitive difficulty.
- A good filtering system will be used to check the relevancy, grammar, duplication and cognitive validity of the questions generated before the questions can be updated on the examination paper.
- The ability to revise, rearrange, and remove questions will allow teachers to preview an exam and then to administer it without losing human control over the final paper-setting process.
- Students can have the option of doing timed exams via a browser-based application with auto-save feature, which can work on both desktop and mobile browsers.
- The system will automatically assess all provided answers i.e. MCQs will be graded by the rule-based scoring and the rest of the answers i.e. written form will be assessed by the semantic similarity, match of the keys, and the LLM-assisted rubric grading.
- An AI Answer Authenticity Detection module will scan all written submissions with the GPTZero API and indicate response scores that are above a 70 percent mark of generating by AI notifying teachers via their dashboard.
- Individual feedback report- each student will receive an individual feedback report after every exam which will include performance by topic, strengths, weaknesses and study recommendations.
- The platform will have role-based access control whereby the teacher and student will have different dashboards and secure authentication and session management.
- All exam data, question banks, student attempts, evaluation scores, AI detection results, and feedback reports will be stored and retrieved out of the database.

➢ **Out of Scope:**

The following are explicitly excluded from the current version of AutoExam.ai to maintain focus and ensure delivery within the project timeline:

- Language support — the system is not going to process or generate anything in other language besides English
- The proctoring based on identity - facial recognition, eye tracking, the lockdown of the browser or monitoring of the screen are not present in this version.
- Mobile application development - no native iOS nor Android application will be developed; the platform will be web-based only.
- Advanced institutional analytics There are more advanced features of institutional analytics that are out of scope at present: cross-cohort performance comparisons, department-level reporting, and longitudinal learning trend analysis.
- Payment or subscription management - the platform does not have any billing, licensing, or institutional account management.
- The version does not include integration with other Learning Management Systems including Moodle, Blackboard, or Google Classroom.
- Other non-functional details, which might be required at the enterprise level, like load balancing, fault tolerance, and high-availability infrastructure are not part of this prototype.

1.7 Software Development Lifecycle Model

Selected Model: Agile

The development of AutoExam.ai will be done using Agile Software Development Model. Agile is a methodology of developing software iteratively and incrementally in small, manageable modules known as sprints where each cycle delivers a working slice of functionality which can be reviewed, tested and increased before starting the subsequent cycle. Agile does not presuppose the strict plan; instead, it empowers the team to modify the plan in response to feedback, test results, and emerging needs; thus, it is especially suited to such a project.

The main factor that contributed to choosing Agile as compared to other models lies with the workplace nature. AutoExam.ai In AutoExam.ai, there is a lot of trial and error with AI ingredients at the prompt engineering level (question generation), threshold tuning of AI detection), and semantic scoring tuning of written answer assessment. They are not those jobs that have a definite, foreseeable result. One kind of course material can have a effective prompt which turns out to be a bad question when applied to another course material. Sensitivity to a similarity score that gives correctly scored answers in one subject might turn out to be too strict in another. Such an iterative refinement just can not be planned in advance as a Waterfall model would insist on.

Conversely, even with a Waterfall approach, all requirements would have to be completed before any development was done, and any change in a requirement during a project would prove expensive and disruptive. The V-Model especially, though it is robust in terms of verification, validation, it is equally inflexible and is only applicable in cases where the requirements are firm and clearly-known at inception. The Spiral model is flexible and provides more overhead during the risk analysis in each cycle than a large-scale or safety-critical system would, which would be specific to an academic prototype. Agile balances well enough - it is not too structured but it is not too flexible to allow the project to stay on track with the learning and adjustment needed by an AI-driven development process.

The project shall be broken down into two or three-week sprints, each sprint shall focus on a particular module or a group of related features. Its total development is planned according to the following stages:

Requirements and Planning The project will start with a requirement gathering phase where all the requirements, including functional and non-functional needs, stakeholders, and the overall system architecture will be agreed upon. The literature review and technology choice is also a part of this stage. Despite this initial occurrence, the requirements are subject to the continuous strengthening during the project as new information appears due to testing and comments of the supervisors.

System Design A database layout, API endpoint design, and UI/UX wireframes, and AI engine architecture are developed during this stage. Decisions made during design are made in writing and checked multiple times before implementation commences but are not considered hard work, but instead are considered as living documents and may be revised as the system changes.

Implementation Development follows the concurrent streams of the AI engine, the backend APIs, and the frontend that the two team members simultaneously develop, and the integration points that are well defined in the beginning of every sprint. Every sprint can produce a workable, testable, increment of the system beginning with the simplest segments of the system, including authentication and the ability to ingest content and proceed to more complicated elements, including semantic evaluation and AI detection.

Testing Testing does not occur at a point of ending. There is a written unit test in association with each module, a run test is done at the end of each sprint, and acceptance testing of the user is done when the entire prototype is assembled. A subset of ChatGPT-generated and human-written sample responses in particular are used to test a threshold calibration of the AI detection module.

Deployment Once the prototype is tested and stable, it will be deployed to cloud hosting providers Vercel to the frontend and Render to the backend. Pilot run would be performed with a small sample number of users to confirm real-world usability and a final submission would take place.

Maintenance and Review Within the FYP model, problems that arise during pilot testing are resolved in a final maintenance sprint. Mid-report and final evaluation checkpoint feedback by the supervisor is embedded into the product at this stage.

Agile ensures that a project is on track by using a mix of brief delivery cycles, intermittent review checkpoints, and the open availability of project progress. Each sprint has a working increment that is presented to the supervisor at the end and this ensures a natural accountability mechanism that ensures that the problems are realized most of the time early in the year instead of suffering finding at the close of the year. The concept of using a common GitHub repository and version control is used to make sure that the entire team is always on the same code base as well as managing the tasks work is done using a basic Kanban like board to ensure that personal responsibilities are within sight and are handled. This architecture will help to make sure the project proceeds in a disciplined and slightly more ordered, yet at the same time, it will not sacrifice the flexibility that AI-driven development truly demands.

Chapter 2. Requirement Analysis

2.1 Literature Review

AutoExam.ai is being developed at the border of a number of currently active fields of research - automatic question generation, automated answer evaluation, autonomously generated text identification, and personalised feedback in learning. This section takes a survey of the most applicable work in these areas, the development of the major techniques, as well as the unavoidable gaps that this project will respond to.

Automatic Question Generation

The concept of automatic generation of exam questions based on text has a history of over ten years. Initial methods were extensively based on rule-based methods of natural language processing and syntactically parsing and transformation rules were used to synthesize declarative sentences into interrogative sentences. [1] was one of the first to study this problem systematically, suggesting an overgenerate and rank strategy in which several candidate questions are produced out of a passage, and ranked by a statistical model which had been trained to do so based on human judgments. Their work has shown that machines could generate grammatically valid questions, but there were inconsistencies in relevance and pedagogic quality. This theoretical groundwork had made the problem viable and the research that followed was based much on this ground work.

With the maturation of deep learning researchers abandoned rule-based pipelines and showed interest in neural sequence-to-sequence models. One of the initial neural methods of question generation was suggested by Du et al [2], where an attention-based encoder-decoder model was trained on the SQuAD dataset to generate a question based on a passage as well as an answer span. They found that their methods were much more fluent and relevant than those based on rules and the work became a common reference in the science. These models were however restricted to produce factoid questions that were confined to certain spans of answers and this limits their use to the entire gamut of question type that are required in formal tests.

Another development of capability was the introduction of models with transformers. Question generation systems became much more contextually sensitive driven by the architectures of BERT, GPT, and T5. A study by Keskar et al. [3] also showed that the use of large pre-trained language models as fine-tuned to controlled text generation tasks had been relatable on relatively limited

amounts of task-specific data, which has since been used extensively in question generation. In more recent studies, authors have investigated applying GPT-3 and GPT-4 to the problem of query generation on zero-shots and few-shots systems, discovering that at least with well-designed prompts such models can generate a variety of high-quality questions across a variety of types with no fine-tuning at all [4]. This is the practice in AutoExam.ai where timely engineering is employed to direct GPT models in the production of MCQs, short-answer questions, and essay prompts based on broken-down course materials.

Bloom's Taxonomy in Automated Assessment

One common difficulty that is faced in automated question generation is that one would like the questions to be not easy but also not all be focused on high-level recall. A hierarchical model of cognitive difficulty by Bloom is used in the form of Taxonomy, which was first introduced by Bloom et al., and later revised by Anderson and Krathwohl [5], and generally a model of classification of cognitive difficulty is based on six levels remembering, understanding, applying, analysing, evaluating and creating. The integration of this framework in automated systems is a concern that has attracted various studies.

Ghuge et al. [6] developed an automatic question generation framework that was clearly based on the levels of Bloom and called the taxonomy, which worked based on NLP preprocessing and rule-based templates in order to generate questions on the lower three levels of the taxonomy. As determined by their system, they had good scores in recall and comprehension questions but were poor in generating analysis and evaluation questions, and these are the questions that need more in-depth analysis and interpretation of the relationship and implications in the text. AutoExam.ai directly overcomes this limitation with LLM by making use of Bloom level prompt template specific templates that encourage the model to form higher order questions.

Kurdi et al. [7] carried out a systematic review of the existing research on automatic question generation conducted in more than a decade and established that lower-order question generation is a well-resolved problem, whereas higher-order question generation is an unsolved challenge. Their review also pointed out that the majority of the existing systems that are tested on datasets on reading comprehension, but not at real-world educational course content, reducing their applicability in real life. AutoExam.ai specifically addresses this gap, since it is engineered and trained on course content in a educational environment.

Automated Answer Evaluation

Student response evaluation One problem which has been heavily studied is the automatic evaluation of student responses especially on open-ended queries. The initial automated essay graders like Project Essay Grade or e-rater [8] relied on statistical indicators like word frequency, sentence length, syntactic complexity and discourse structure to estimate human rated scores. These systems were not very good at generalised writing tests but when subjected to other fields than the one they were being trained on, were fragile and provided no indication of the content validity of a response.

More current methods have been more semantic similarity based evaluation. As illustrated by Riordan et al. [9], semantic distance in the form of sentence embeddings between the answer given by a student and a reference answer yields more credible scores on short-answer questions than those obtained by using the matching of key-words alone. Their study demonstrated that, the embedding-based methods are better suited to withstand paraphrasing and other forms of stating the correct answers and this is a major problem of manual grading. AutoExam.ai is built based on this strategy as the basis of the written answer evaluation engine, combining embedding-based similarity with keyword matching and LLM-based grade the rubric with a more comprehensive and yielding more accurate score.

Answer evaluation using LLMs is a new trend that has generated significant potential. Mizumoto and Eguchi [10]investigated the application of GPT-based models to grade student essays by passing the model a rubric and asking it to compare the response with a set of criteria. Their results revealed that scoring based on LLM is strongly correlated with human ratings, especially when there is a clear definition of the rubric and the model provided with a clear context. The result confirms the grading feature of the AutoExam evaluation engine that is based on the evaluation of theLLM-assisted rubric.

AI-Generated Text Detection

The fast proliferation of generative AI applications has left a gaping hole that must be filled by developing a stable method to detect text generated by AI. Other studies have been conducted in the field to investigate various strategies, with statistical methods, founded on the principle of perplexity and entropy, and classifier-based models that have been trained on human and AI-generated text datasets.

Mitchell et al. [11]suggested the zero-shot detector, DetectGPT, which is inferred to identify machine-generated text using the fact that AI-generated text is more likely to occupy regions of high probability in the language model distribution - that is, it is notably fluent and consistent in comparison with human writing. Their approach applies perturbation-based testing to detect this pattern without a trained classifier. Although it is effective when applied in controlled conditions, DetectGPT needs the original language model with which the language was generated, thus reduces its value in real-life application.

A more feasible solution to this is GPTZero, created by Tian and Cui [12], which integrates perplexity, which quantifies the predictability of the text to a language model, with burstiness, which quantifies the variance in sentence-level complexity. The high burstiness of human writing is due to the fact that human writers are more inclined to change their sentence structure whereas the AI-generated text is more uniformly complex. GPTZero encodes these signals into a convenient API which provides a probability score of any input text and is therefore easily integrated into a production system. AutoExam.ai is based on GPTZero API, which is the core of the AI Answer Authenticity Detection component, marking written submissions that have a 70 percent probability threshold that it was generated by AI.

Another tool of text detection, created by Gehrmann et al. [13], is a method called GLTR, which can be based on visualising the statistical characteristics of text under a language model. Their research supported the conclusion that AI-written texts are statistically different to human writing, but the level of the difference declines with the complexity of AI-based models. This is a major limitation that can be realized because no detection tool can be really accurate, and AutoExam.ai does not consider the outcome of detection as a final verdict and treats it as an informational signal to the teacher.

Feedback Systems in Education: Personalised.

The importance of customised, on-time feedback in the educative system is already thoroughly determined in educational studies. Hattie and Timperley [14]carried out an extensive meta-analysis of the literature on feedback and came up with a conclusion that feedback is one of the most influential factors in the student achievement, although it must be specific, timely, and task-oriented. General feedback like a numerical score does not impact much on learning. This result forms the theoretical basis of the AutoExam.ai feedback module, which is set to provide the analysis of performance by topic, specified weaknesses and recommendations to study instead of a mere grade.

Personalised feedback has always been scaled down in intelligent tutoring systems. VanLehn [15] surveyed a wide range of intelligent tutoring systems and discovered that systems that used fine-grained feedback instantly had learning benefits equal to those of human tutoring one-on-one in certain areas. Although the AutoExam.ai is not an intelligent tutoring system in a broad sense, its feedback system is also based on the same principle - that special feedback, given in response to assessment and given in a timely manner yields the best learning results compared to delayed feedback or generic feedback.

2.2 Stakeholders List

One of the first steps in determining the type of stakeholders that the system is developed is the process of identification of stakeholders, understanding who the system is developed, who is the impacted party, and whose needs must be taken into account during the development process. The stakeholders in AutoExam.ai directly or indirectly include the following individuals and groups and external parties.

2.2.1 Primary Stakeholders

These are those who have the close touch with the system on a daily basis and whose experience will be the ultimate judge on whether the platform will achieve its aim.

- Teachers / Educators These are the most important users of AutoExam.ai. They will be tasked with the uploading of course content, exam papers they generate are to be reviewed and published, they will monitor student performance via the dashboard, and respond to AI detection alerts. This platform is made on the main premise of making their workload less and providing them with improved assessment tools. One of the determinants of the project success will be their satisfaction with the quality of the generated questions and the clear results of the detection.
- Students The second key user group is students. They engage with the system when attempting to exams and in reviewing their individual feedback reports. The platform is there in the end to serve them - to offer them a more equal test experience and to offer them feedback that actually works to their benefit. The fact that they are not confused when using the exam interface and the fact that they interpret the feedback they get is also significant in the assessment of the usability of the system.

2.2.2 Secondary Stakeholders

These are parties that stand to gain a lot in the project and control the direction of the project but they do not have direct contact with the system on a day-to-day basis.

- Academic Supervisor The role of project supervisor is to control the academic and technical quality of work. They assess progress on key milestones, give feedback on methodology and implementation and finally justify the accomplishment of said objectives by the project. Their advice is a direct influence on the direction of the project during the FYP year.
- Institution Management / Administrators The AutoExam.ai deployment in any educational institution would include administrators who would onboard the teachers, manage user accounts, and make sure that the platform is active in all departments. The system design has taken into consideration their needs in the areas of scalability, data management and easy administration. AutoExam.ai is designed to be institution-neutral and, therefore, can be implemented in schools, colleges, universities, or training centres without any modification to the essential platform.
- Adopting Institute IT Departments In institutes that have stable IT support, the technical department would handle the deployment, control over access and the first-line troubleshooting. Their requirement of a simple deployment procedure and documentation

of the system has contributed to making the choice to place the platform on uncomplicated and broadly comprehended cloud infrastructure.

2.2.3 External Stakeholders

These are external parties that have their services, tools or interests that are tied to the project in a significant manner.

- The GPT models that run the question generation and LLM-assisted assessment features of AutoExam.ai are unavailable on OpenAI. Their API policies of availability, pricing, and usage have direct impacts on what the system can accomplish as well as at what rates. Any modification of their terms of service or API structure would directly affect the platform.
- GPTZero GPTZero offers the API Key GPTZero/Answer Authenticity Detection API on which the system is based to identify written submissions that are created using AI. The points that influence the efficiency of the detection module are their free-tier rate limits, their detection accuracy, and their API reliability. They are an outsourced dependency where the quality of service is important to the project.
- Hugging Face Hugging Face contains open-source implementations of transformer models, such as T5 and the BART variants, which can be used in place of the API provided by OpenAI, and which can be used in question generation and semantic evaluation problems. Their platform is utilized to access models and experiment with them in the stage of development of the AI engine.
- Supabase Supabase contains the managed PostgreSQL database being used to keep all user information, exams results, submissions, and detection outcome, and feedback reports. Their uptime, data policy, and free-tier restrictions need to be considered directly as the reliability and scalability because they are a backend-as-a-service provider.

2.3 Requirements Elicitation

Eliciting requirements of AutoExam.ai was done in a mix of approaches to make sure that the system is based on the real needs of the people who will be using the system. This started by document analysis, which included research on the available literature on automated assessment systems, the limitations of other tools, and the way other similar platforms generate questions, evaluate and provide feedback. This provided the team with a solid background under which they could see what is already in existence and their areas of weaknesses.

The identification gaps were converted into tangible system features through brainstorming sessions between the two team members and the project supervisor. The system is divided into parts, with every brainstorming session dedicated to each of them to cover the gap of content ingestion, exam generation, evaluation, AI detection, and feedback. Analysis in the form of scenario was also used whereby the team went through realistic usage scenarios both as a teacher and a student of what the system should do at each step to provide a useful experience.

Unofficial interviews with both students and teachers who are conversant with the Pakistan educational environment served to confirm assumptions regarding the needs of users in this context, specifically regarding the feedback gap and the issue of AI-generated responses. These discussions proved that the features that will be included in AutoExam.ai are based on real, felt problems and not on theory. The requirements obtained using all these techniques were subsequently divided into non-functional and functional requirements and were checked against the project goals so as to be complete.

2.3.1 Functional Requirements

Functional requirements define what the system must do — the specific behaviours, features, and functions it must deliver to meet stakeholder needs.

Table 1 Functional Requirments

FR#	Requirement Description	Actor	Priority
FR-01	The system shall allow teachers to register and log in with secure credentials	Teacher	High
FR-02	The system shall allow students to register and log in with secure credentials	Student	High
FR-03	The system shall support role-based access control, separating teacher and student views and permissions	Both	High
FR-04	The system shall allow teachers to upload course material in text, PDF, or URL format	Teacher	High
FR-05	The system shall extract and clean raw text from uploaded material using NLP preprocessing techniques	System	High
FR-06	The system shall segment processed content into topic-based chunks using sentence embeddings and topic modelling	System	High
FR-07	The system shall generate MCQ questions with distractor options from the processed course material	System	High
FR-08	The system shall generate short-answer conceptual questions from the processed course material	System	High
FR-09	The system shall generate essay-type prompts from the processed course material	System	High
FR-10	The system shall distribute generated questions across Bloom's Taxonomy cognitive difficulty levels	System	High
FR-11	The system shall apply a quality filter to remove duplicate, irrelevant, or low-quality questions before exam assembly	System	High
FR-12	The system shall allow teachers to review, edit, reorder, and delete questions before publishing an exam	Teacher	High
FR-13	The system shall allow teachers to configure exam settings including timer duration, question count, and marks distribution	Teacher	High
FR-14	The system shall publish the exam and make it accessible to students within the configured time window	Teacher	High
FR-15	The system shall deliver timed exams to students through a browser-based interface with auto-save functionality	Student	High
FR-16	The system shall automatically submit the student's saved answers when the timer expires	System	High
FR-17	The system shall score MCQ answers instantly using rule-based matching after submission	System	High
FR-18	The system shall evaluate short-answer and essay responses using semantic similarity scoring and keyword matching	System	High

FR#	Requirement Description	Actor	Priority
FR-19	The system shall apply LLM-assisted rubric grading to written answers for a more accurate and contextual score	System	High
FR-20	The system shall analyse all written submissions using the GPTZero API to compute AI-generation probability scores	System	High
FR-21	The system shall flag any written submission where the AI-generation probability exceeds 70 percent	System	High
FR-22	The system shall alert the teacher through a dashboard toggle displaying per-submission AI probability scores and flags	Teacher	High
FR-23	The system shall allow teachers to configure the AI detection probability threshold for their exams	Teacher	Medium
FR-24	The system shall generate a personalised feedback report for each student after every completed exam	System	High
FR-25	The system shall display topic-wise performance scores, strengths, weaknesses, and study recommendations in the feedback report	Student	High
FR-26	The system shall store all users, exams, questions, attempts, submissions, detection results, and feedback reports in the database	System	High
FR-27	The system shall maintain a question bank from previously generated exams for teacher reuse	Teacher	Medium
FR-28	The system shall provide teachers with a results overview showing class-wide performance statistics for each exam	Teacher	Medium

---

2.3.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints the system must satisfy — how well it performs rather than what it does.

Table 2 Non-Functional Requirments

NFR#	Category	Requirement Description	Priority
NFR-01	Usability	The system interface shall be navigable by a first-time user without requiring any technical training or documentation	High
NFR-02	Usability	All primary teacher and student workflows shall be completable within a maximum of five user interactions from the dashboard	Medium
NFR-03	Performance	MCQ scoring shall return results to the student within 2 seconds of exam submission	High
NFR-04	Performance	Written answer evaluation shall be completed and results displayed within 15 seconds of exam submission under normal API conditions	High

NFR#	Category	Requirement Description	Priority
NFR-05	Performance	The exam interface shall load within 3 seconds on a standard broadband connection	Medium
NFR-06	Reliability	The auto-save feature shall preserve student answers at intervals of no more than 30 seconds throughout the exam attempt	High
NFR-07	Reliability	If the AI detection API is unavailable, the system shall continue functioning normally and queue detection for retry without disrupting evaluation or feedback	High
NFR-08	Security	All user passwords shall be stored using a secure hashing algorithm and never stored or transmitted in plain text	High
NFR-09	Security	All communication between the client and server shall be encrypted using HTTPS	High
NFR-10	Security	Role-based access control shall prevent students from accessing teacher dashboards, exam management features, or detection results	High
NFR-11	Scalability	The system architecture shall support an increasing number of concurrent users without requiring significant changes to the codebase	Medium
NFR-12	Compatibility	The web application shall function correctly on the latest versions of Chrome, Firefox, Safari, and Edge browsers	Medium
NFR-13	Compatibility	The student exam interface shall be fully usable on mobile browsers without loss of functionality	Medium
NFR-14	Maintainability	The codebase shall be modular, with the AI engine, backend APIs, and frontend developed as separate, independently maintainable components	Medium
NFR-15	Availability	The deployed platform shall target an uptime of at least 95 percent during the pilot testing phase	Medium

2.3.3 Requirements Traceability Matrix

The Requirements Traceability Matrix links each functional requirement to its corresponding project objective, system module, use case, and planned test case. This ensures that every requirement has a clear purpose, is covered by the implementation, and can be verified through testing.

Table 3 Requirements Tracebility Matrix

FR#	Project Objective	System Module	Use Case	Test Case
FR-01, FR-02, FR-03	Obj-6: Role-based web application	Authentication & Access Control	UC-1, UC-4	TC-01

FR#	Project Objective	System Module	Use Case	Test Case
FR-04	Obj-1: Question generation pipeline	Content Upload API	UC-1	TC-02
FR-05, FR-06	Obj-1: Question generation pipeline	NLP Preprocessing Module	UC-1	TC-03
FR-07, FR-08, FR-09	Obj-1: Question generation pipeline	LLM Question Generation Engine	UC-2	TC-04
FR-10	Obj-1: Question generation pipeline	Bloom's Taxonomy Classifier	UC-2	TC-05
FR-11	Obj-1: Question generation pipeline	Quality Filter Module	UC-2	TC-06
FR-12, FR-13, FR-14	Obj-6: Role-based web application	Teacher Dashboard & Exam Publisher	UC-3	TC-07
FR-15, FR-16	Obj-6: Role-based web application	Student Exam Interface	UC-4	TC-08
FR-17	Obj-2: Semantic evaluation engine	MCQ Scoring Module	UC-5	TC-09
FR-18, FR-19	Obj-2: Semantic evaluation engine	Written Answer Evaluation Engine	UC-5	TC-10
FR-20, FR-21, FR-22, FR-23	Obj-3: AI detection module	AI Answer Authenticity Detection	UC-6	TC-11, TC-12
FR-24, FR-25	Obj-4: Personalised feedback system	Feedback Generation Module	UC-7	TC-13
FR-26	Obj-6: Role-based web application	Database Layer	UC-1 to UC-8	TC-14
FR-27, FR-28	Obj-6: Role-based web application	Teacher Dashboard Analytics	UC-8	TC-15

2.4 Use Case Description

The descriptions of the use cases presented below detail the interactions of the major participants in the system Teacher and Student with AutoExam.ai to accomplish personal objectives. Each use case has the actor that is involved, preconditions, main flow, alternate flow, and postconditions.

- **Use Case 1: Upload Course Material**

Field	Detail
Use Case Name	Upload Course Material.

Field	Detail
Actor	Teacher
Description	Teacher uploads course content in order to have the system process the content and generate questions.
Preconditions	Teacher is logged in and registered in the system. Materials of the course can be in the form of the text, PDF, or URL.

Main Flow

Teacher clicks on the Upload Material page of the dashboard.

Teacher chooses the input format; text, PDF file or URL.

Teacher copies or pastes the material and clicks On submit.

The uploaded material is extracted and cleansed using systems.

NLP is used to break down the content into topic-like segments of the system.

System signs successful processing and sends the information to a teacher that the material is prepared to be subjected to question generation.

Alternate Flow

- In case the PDF uploaded is scanned or in form of an image and the text may not be extracted, the system will raise an error message and the teacher will be asked to upload a document that may be selected.
- In case the URL is unavailable or the content it provides is unreadable, the system will inform the teacher and request him to supply another input.
- Postcondition The course material has been successfully processed and is stored in the database and is ready to be accessed as the foundation of generating the exams.

- **Use Case 2: Generate Exam Paper**

Field	Detail
Use Case Name	Generate Exam Paper
Actor	Teacher

Field	Detail
Description	Teacher configures exam settings and the system automatically generates a structured exam paper from the uploaded material
Preconditions	Teacher is logged in. Course material has been successfully uploaded and processed

Main Flow

- Teacher goes to the Generate Exam section and chooses the processed material to be used.
- Teacher sets up settings - questions per type, distribution of difficulty and total marks.
- Teacher clicks Generate.
- System uses the question generation pipeline with LLMs in order to generate MCQs, short-answer questions, and essay questions.
- Quality filtering is used in system to eliminate duplicates, irrelevant and low quality questions.
- System displays the created exam paper to the teacher.

Alternate Flow

- In case the uploaded content has less text than the number of questions demanded, the system will create as many as possible and alert the teacher about the constraint.
- In case of the unavailability of the LLM API during the generation, the system provides an error and suggests the teacher to rerun the attempt after a brief delay.

Postconditions

A draft examination paper is created, filtered, and shown to the teacher who reviews the paper and edits it before publication.

- **Use Case 3: Review and Publish Exam**

Field	Detail
Use Case Name	Review and Publish Exam
Actor	Teacher
Description	Teacher reviews the generated exam, makes any necessary edits, and publishes it for students to attempt
Preconditions	An exam paper has been successfully generated and is available in draft state

Main Flow

- Review Interface- Teacher opens draft exam paper.
- Teacher reads out every question and edits, deletes or rearranges accordingly.
- Teacher determines the time of the exam, available date and any other configuration.
- Teacher clicks Publish.
- System stores the final exam and allocates it into the corresponding group of students and provides the same to the students within the specified time.
- Checks and verifies that a publication is valid and the students can now see the examination link or the access code.

Alternate Flow

- In case the exam contains fewer than the minimum number of required questions because of the deletion of questions by the teacher, the system alerts the teacher against publication.
- In case the teacher tries to publish without setting clock, the system will alert him/her to check whether he/she wants an untimed exam and then proceed.

Postconditions The test is issued and available to students in the allotted time. The test is saved in the database and configured to its entirety and questions.

- **Use Case 4: Attempt Exam**

Field	**Detail**
Use Case Name	Attempt Exam
Actor	Student
Description	Student logs in, accesses an available exam, attempts all questions within the time limit, and submits their answers
Preconditions	Student is registered and logged in. An exam has been published and is currently within its available time window

Main Flow

1. The student clicks on the platform and on the dashboard, the available exam appears on his/her screen.
2. Student clicks on Start Exam and timer starts.
3. Student reads each question and writes in his answers - options in MCQs and typing answers in short-answer and essay questions.
4. System auto-saves the answers of the student in intervals during the attempt.
5. Student accesses his replies and clicks Submit when complete.
6. The submission is recorded with a timestamp in system and prevents the attempt by further modification.

Alternate Flow

- In case the timer expires before the student submits, the system automatically submits all the answers saved so far and informs the student.
- In the event that the student loses internet connectivity during exams, the auto-save option will save the last responses given by the student and will resume with his work once they are connected again, as long as the timer does not run out.

Postconditions

The attempt to write the exam is written into the database containing all the answers given by the student and a completion date. The attempt is put on the queue of automatic assessment..

- **Use Case 5: Evaluate Submitted Answers**

Field	**Detail**
Use Case Name	Evaluate Submitted Answers
Actor	System (Automated)
Description	The system automatically evaluates all submitted answers after the student completes the exam
Preconditions	A student has successfully submitted their exam attempt

Main Flow

1. System retrieves all answers placed on the attempt that has been done.
2. System uses rule-based matching with the correct answers of MCQ answers and marks them instantly.
3. Each written answer is inputted into the semantic evaluation engine, and similarity scores between the model answer and the written answer are computed by using embeddings, keyword matching, and rubric grading with the help of the LLM.
4. System generates an overall score and performance per topic in the attempt.
5. All scores and evaluation metadata are stored in the database as system.
6. All written submissions are sent through the System triggering an AI Answer Authenticity Detection module.

Alternate Flow

- In case there is a temporary unavailability of the LLM evaluation API, the system will resort to scoring similarity using only embedding-based scoring and place the submission in a queue to be re-evaluated as soon as the API becomes available again.

Postconditions

Marks have been allocated, the results are recorded and the effort is sent to the AI detection module where written responses will be analyzed.

- **Use Case 6: Detect AI-Generated Answers**

Field	Detail
Use Case Name	Detect AI-Generated Answers
Actor	System (Automated), Teacher
Description	The system analyses written submissions for signs of AI generation and alerts the teacher if the probability exceeds the configured threshold
Preconditions	Written answers have been evaluated and are available for analysis. GPTZero API is accessible

Main Flow

1. System gets all written submissions of the attempt cooked.
2. The written answers are sent to the GPTZero API to analyze them in terms of perplexity and burstiness.
3. GPTZero is used to generate a score of an AI-generated probability per submission.
4. System compares all scores with the default setting of 70 percent.
5. Those submissions that are above the threshold are marked in the database with the probability rating kept.
6. The teacher dashboard is modified to have flagged submissions under AI Detection toggle.

Alternate Flow

- When the GPTZero API rate limit has been hit, the system will queue the remaining submissions and will attempt a retry after the rate limit window has elapsed, without breaking the student feedback flow.
- When the API is totally not available, the system will ignore that attempt and log the failure, and instruct the teacher that detection results are unavailable.

Postconditions

All written submissions analysed. The submissions that are flagged are recordered in the database and the teacher can see them. There is a storage of detection results as well as evaluation scores of the attempt.

- **Use Case 7: View Personalised Feedback**

Field	Detail
Use Case Name	View Personalised Feedback

Field	Detail
Actor	Student
Description	Student views their detailed feedback report after the exam has been evaluated, showing topic-wise performance, strengths, weaknesses, and study recommendations
Preconditions	The student's exam attempt has been fully evaluated and the feedback report has been generated

Main Flow

1. Student enters in the platform and goes to his or her results section.
2. Student picks the exam done to see his/her feedback report.
3. System shows the total score as well as a breakdown of performance in terms of topic.
4. System identifies areas that the student excelled as strengths.
5. System helps point to the areas that are weak and gives specific study recommendations on the matter.
6. Student is able to scroll the entire report and go over individual question feedbacks.

Alternate Flow

- In case the feedback report is yet to be created as a result of the processing delay, the system will show the student a message that the results are being made and to be back in a little bit.

Postconditions

The student has checked his/her complete feedback report. The report is kept on their dashboard to be used in future.

- **Use Case 8: View AI Detection Results**

Field	Detail
Use Case Name	View AI Detection Results
Actor	Teacher
Description	Teacher reviews the AI detection results for a completed exam and decides how to act on flagged submissions
Preconditions	An exam has been completed, evaluated, and processed through the AI detection module

Main Flow

1. On his or her dashboard, Teacher goes to the passed exam.
2. Artificial Intelligence Teacher Helm on the AI Detection Results panel.
3. System presents a list of every written submission along with the probability scores of AI-generation.
4. Submissions that are flagged, i.e. submissions with a rating higher than 70 percent, stand out conspicuously before the teacher.
5. The submissions with flags are reviewed by the teacher and their probability scores.
6. The teacher determines unilaterally how to deal with the flagged cases, which could be to make the student resubmit, or to make a verbal follow-up.

Alternate Flow

- In the event that no submissions were identified, the system will show a confirmation message as to the fact there were no AI-driven answers detected in this exam.
- In the case that the detection results are not available because of API delays, the system alerts the teacher and automatically the panel is updated after results become available.

Postconditions

The teacher has scanned all the AI detection results of the exam. The system does not impose any automatic decision with regard to flagged submissions and instead any such decision made is in the discretion of the teacher.

Chapter 3. System Design

3.1 Use Case Design

The Use Case Design of AutoExam.AI illustrates the interaction between system actors and the platform in order to achieve specific goals within the automated examination lifecycle. The primary actors identified in the system include **Teachers**, **Students**, and the **System (AI Engine)**, with additional support from external services such as AI APIs and detection modules. Teachers interact with the system to upload course material, generate and manage examinations, and analyze student performance, while students engage with the platform to attempt exams and review personalized feedback. The system itself performs automated processes such as question generation, evaluation, AI-based answer detection, and feedback generation.

These interactions are modeled through use cases to clearly define the functional behavior of the system from a user-centric perspective. Each use case represents a specific functionality, such as exam creation, attempt, evaluation, and feedback delivery, and outlines how actors communicate with the system to accomplish these tasks. The purpose of presenting use cases in this section is to provide a structured understanding of system operations, ensure alignment with functional requirements, and serve as a foundation for subsequent system design components including sequence diagrams, activity flows, and implementation logic.

3.1.1 Module 1: Authentication & Access Control

The Authentication and Access Control module governs secure access to the AutoExam.AI platform by ensuring that only authorized users can interact with system functionalities based on their roles. This module supports user registration, login, and session management while enforcing role-based access control for Teachers and Students. It ensures that users are authenticated before accessing sensitive features such as exam creation, attempt, evaluation results, and feedback reports. Additionally, the system maintains session integrity through token-based authentication and prevents unauthorized access by validating user credentials and roles at each interaction. This module is critical for maintaining system security, protecting user data, and ensuring that all actions within the platform are performed by verified and authorized entities.

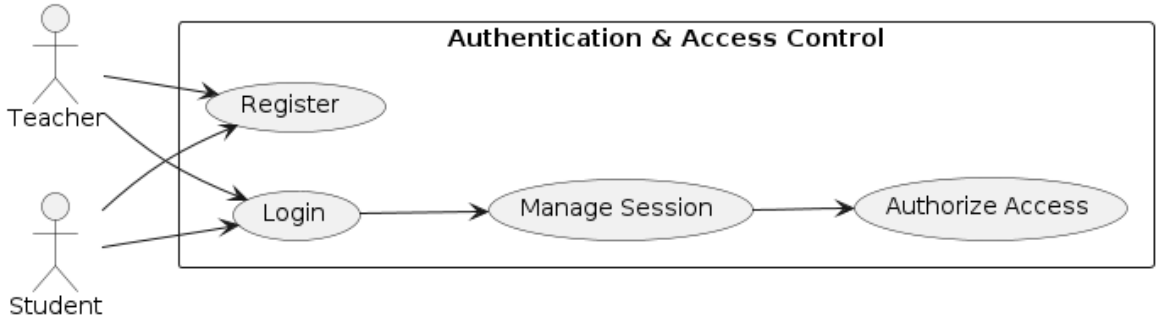


Figure 1 Use Case Diagram for Authentication & Access Control Module

3.1.2 Module 2: Content Upload & Processing

This module enables teachers to upload course material and initiates the AI-driven processing pipeline. The system extracts, cleans, and segments the content into meaningful topics using NLP techniques. These processed segments serve as the foundation for question generation. The module ensures that input data is structured, relevant, and ready for downstream exam creation tasks.

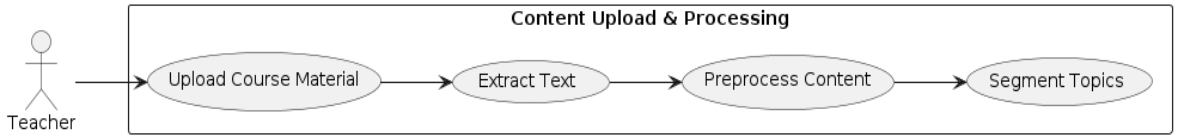


Figure 2 Use Case Diagram for Content Upload & Processing Module

3.1.3 Module 3: Exam Generation

This module enables the automatic creation of structured exams from processed course content. The system generates multiple question types using LLMs and applies quality filtering to ensure relevance and difficulty balance. Teachers can initiate the generation process based on selected content. The module ensures that exams are aligned with curriculum and cognitive levels.

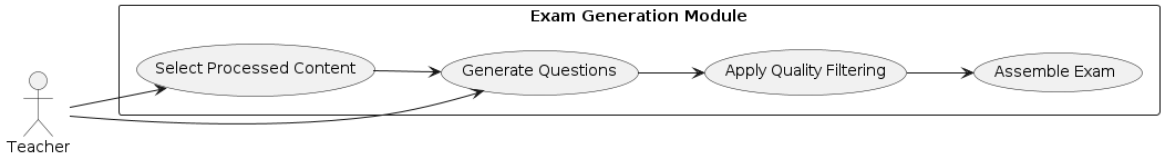


Figure 3 Use Case Diagram for Exam Generation Module

3.1.4 Module 4: Exam Management

This module allows teachers to review, modify, and finalize generated exam papers before publishing. Teachers can edit questions, adjust order, and configure exam settings such as duration and marks distribution. The system ensures that exams meet required criteria before publication. It maintains teacher control over the final assessment process.

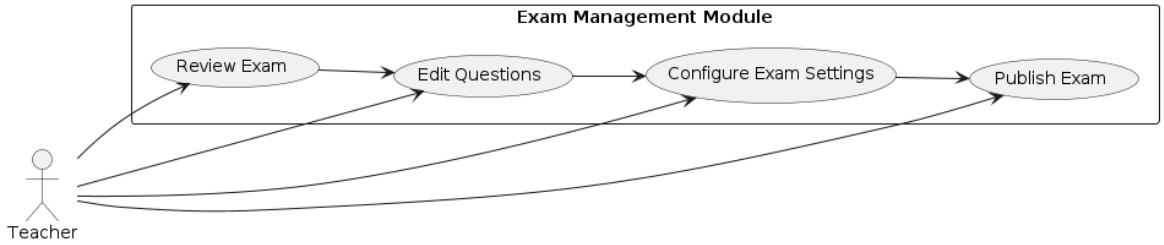


Figure 4 Use Case Diagram for Exam Management Module

3.1.5 Module 5: Exam Attempt

This module enables students to access and attempt published exams through a web-based interface. Students can answer questions within a timed environment with automatic saving of responses. The system ensures smooth interaction during the attempt and enforces submission rules. It supports reliable exam completion under varying conditions.

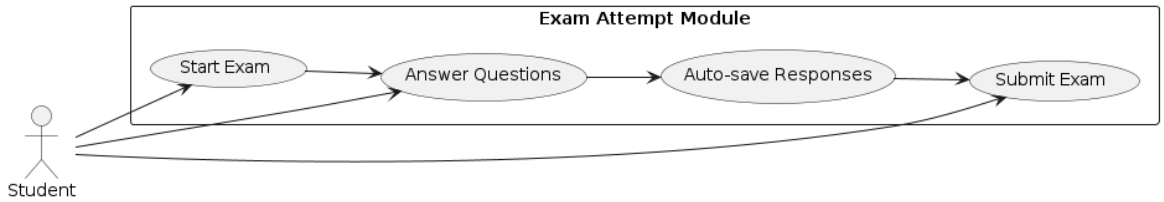


Figure 5 Use Case Diagram for Exam Attempt Module

3.1.6 Module 6: Evaluation System

This module performs automated evaluation of student responses after exam submission. MCQs are graded using rule-based scoring, while subjective answers are assessed using semantic similarity and LLM-assisted methods. The system computes overall and topic-wise scores. It ensures accurate and timely evaluation without manual intervention.

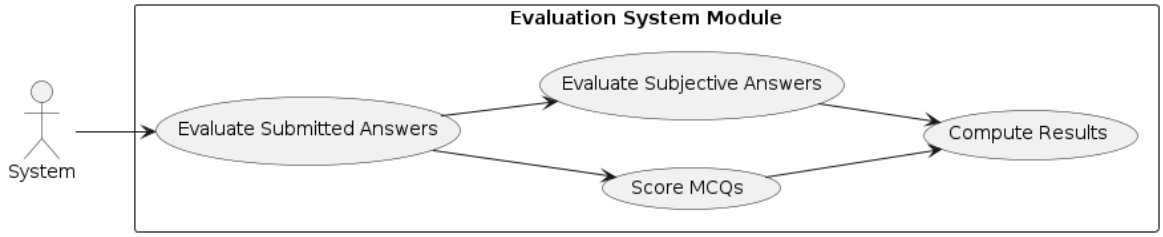


Figure 6 Use Case Diagram for Evaluation System Module

3.1.7 Module 7: AI Detection

This module analyzes submitted written answers to detect potential AI-generated content. The system sends responses to an external detection service and evaluates probability scores. Submissions exceeding the defined threshold are flagged for teacher review. It supports academic integrity by identifying suspicious responses.

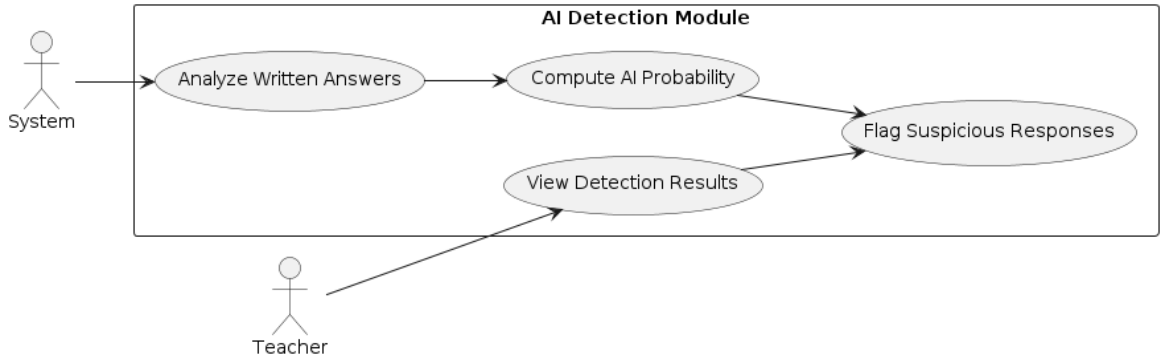


Figure 7 Use Case Diagram for AI Detection Module

3.1.8 Module 8: Feedback System

This module generates and delivers personalized feedback to students after evaluation. The system analyzes performance to identify strengths, weaknesses, and topic-wise scores. Feedback is presented in a structured and understandable format. It helps students improve learning based on actionable insights.

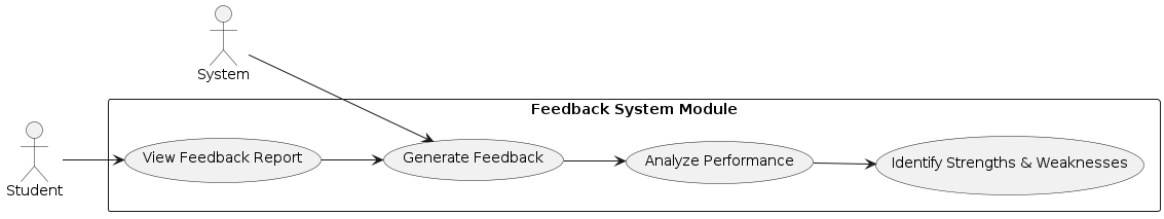


Figure 8 Use Case Diagram for Feedback System Module

3.1.9 Module 9: Analytics Dashboard

This module provides teachers with insights into student and exam performance through visual analytics. The system aggregates results to present class-level statistics and trends.

Teachers can monitor performance and identify learning gaps. It supports data-driven decision-making in assessment.

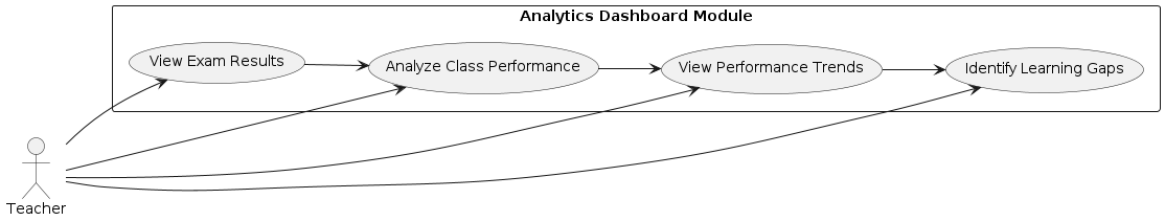


Figure 9 Use Case Diagram for Analytics Dashboard Module

3.2 Activity Diagram

3.2.1 AD-01: Authentication Workflow

This activity diagram represents the process of user authentication in AutoExam.AI, including credential validation, account status checks, and session initialization. It illustrates how the system verifies login inputs and restricts access for invalid or unauthorized users. The workflow ensures secure access control aligned with system requirements. It directly supports authentication-related functionalities in the platform.

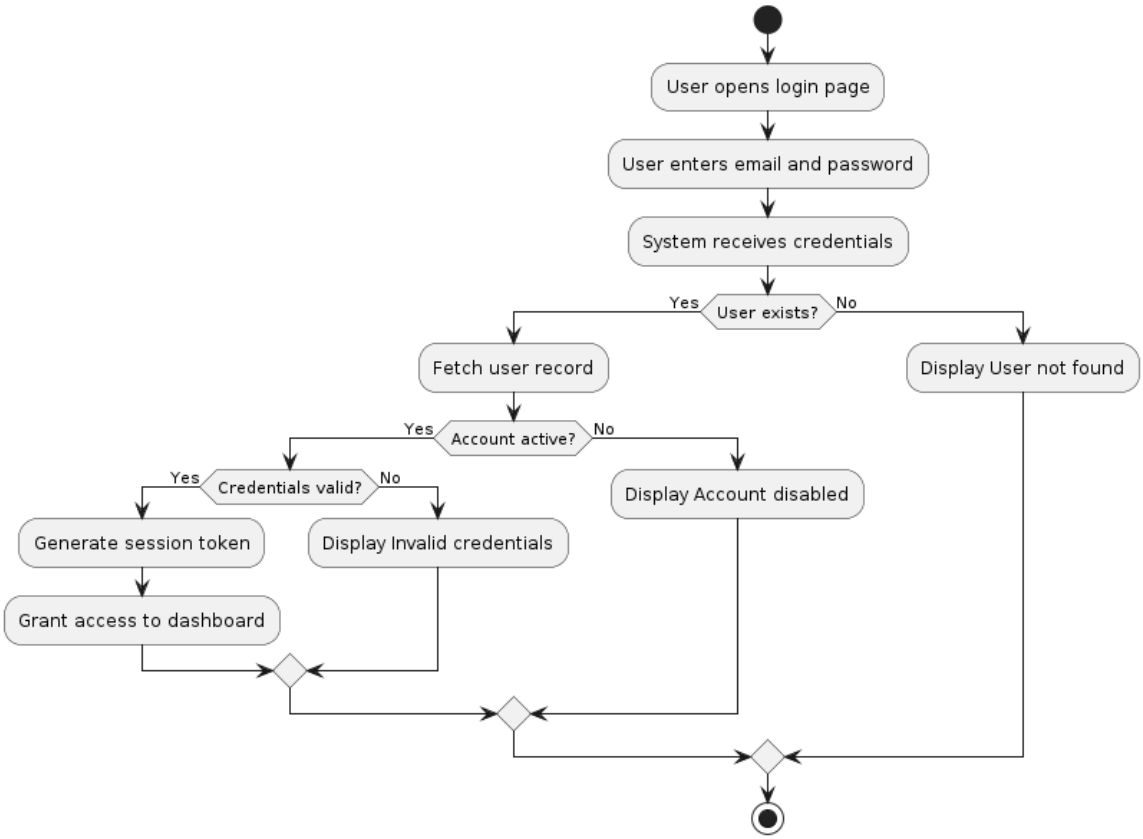


Figure 10 Activity Diagram for Authentication Workflow

3.2.2 AD-02: Content Upload & Processing Workflow

This activity diagram illustrates the process of uploading and processing course material in AutoExam.AI. It shows how the system validates input, extracts text, and applies NLP preprocessing techniques. The workflow ensures that only valid and structured content is prepared for question generation. It supports the content ingestion pipeline of the system.

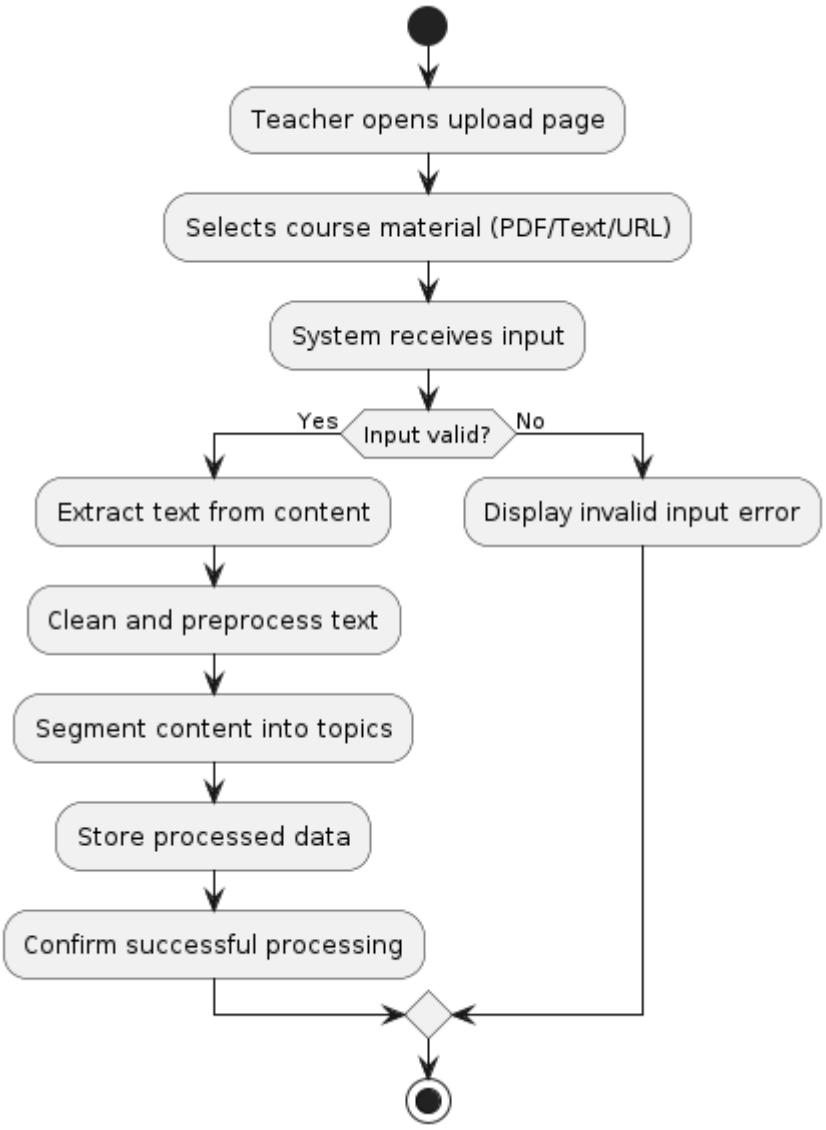


Figure 11 Activity Diagram for Content Upload & Processing Workflow

3.2.3 AD-03: Exam Generation Workflow

This activity diagram illustrates the process of generating exam papers from processed course content in AutoExam.AI. It shows how the system uses AI models to create questions, applies quality filtering, and assembles the final exam. The workflow ensures

that generated questions are relevant, non-duplicate, and balanced in difficulty. It supports automated exam creation aligned with curriculum requirements.

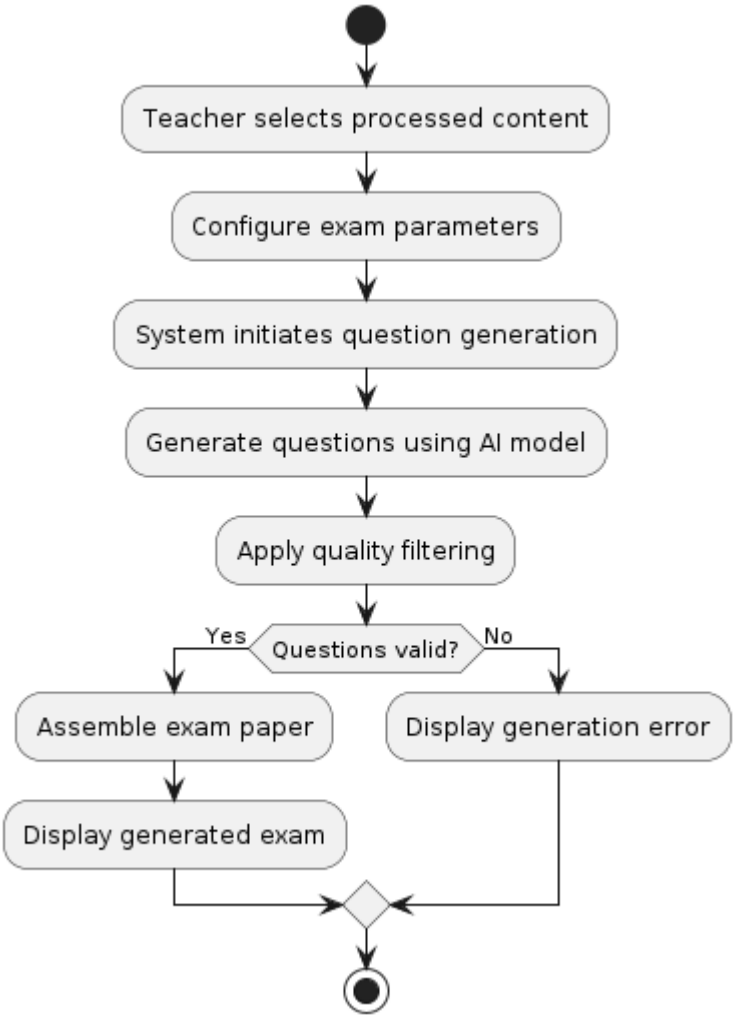


Figure 12 Activity Diagram for Exam Generation Workflow

3.2.4 AD-04: Exam Management Workflow

This activity diagram illustrates how teachers review and manage generated exam papers before publication. It shows the process of editing questions, configuring exam settings, and validating the exam structure. The workflow ensures that the final exam meets required criteria before being published. It maintains teacher control over the assessment process.

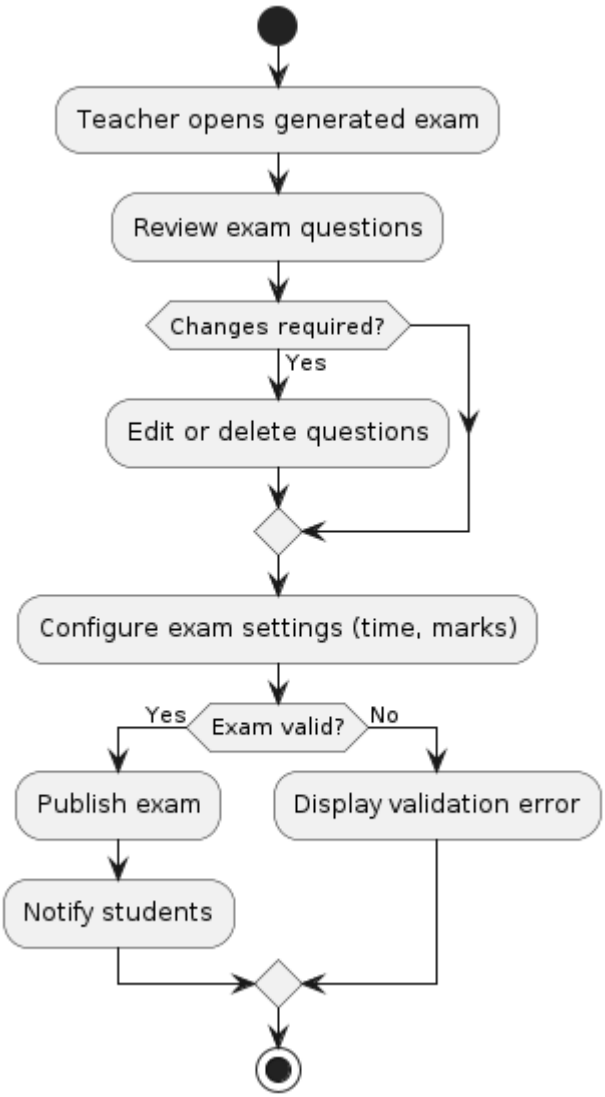


Figure 13 Activity Diagram for Exam Management Workflow

3.2.5 AD-05: Exam Attempt Workflow

This activity diagram illustrates how students access and attempt exams in AutoExam.AI. It shows the process of starting an exam, answering questions, auto-saving responses, and submitting within the time limit. The workflow ensures reliable exam completion and data persistence. It supports smooth interaction during the assessment process.

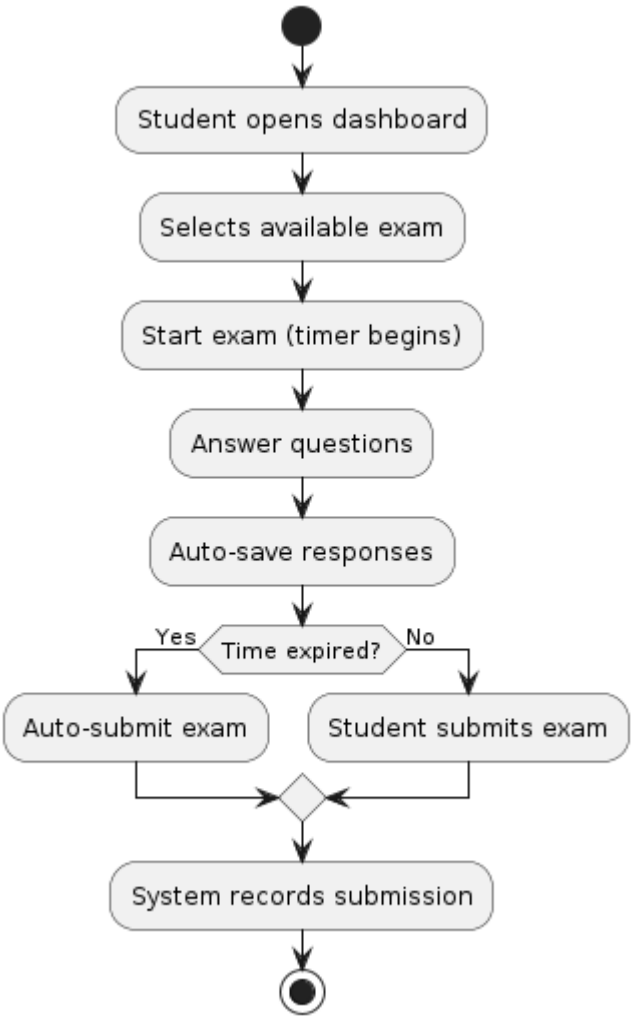


Figure 14 Activity Diagram for Exam Attempt Workflow

3.2.6 AD-06: Evaluation System Workflow

This activity diagram illustrates the automated evaluation process of submitted exams in AutoExam.AI. It shows how the system grades MCQs using rule-based logic and evaluates subjective answers using semantic and AI-assisted methods. The workflow ensures accurate computation of scores and performance metrics. It supports timely and consistent assessment of student responses.

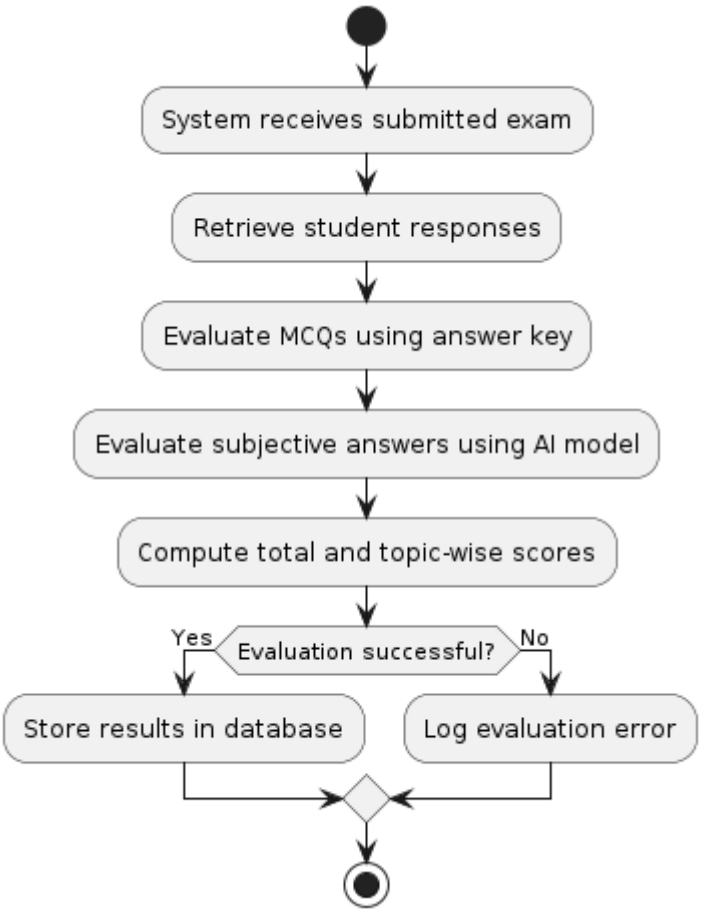


Figure 15 Activity Diagram for Evaluation System Workflow

3.2.7 AD-07: AI Detection Workflow

This activity diagram illustrates how AutoExam.AI detects AI-generated content in student submissions. The system analyzes written answers using an external detection service and evaluates probability scores. Submissions exceeding the defined threshold are flagged for review. The workflow supports maintaining academic integrity in assessments.

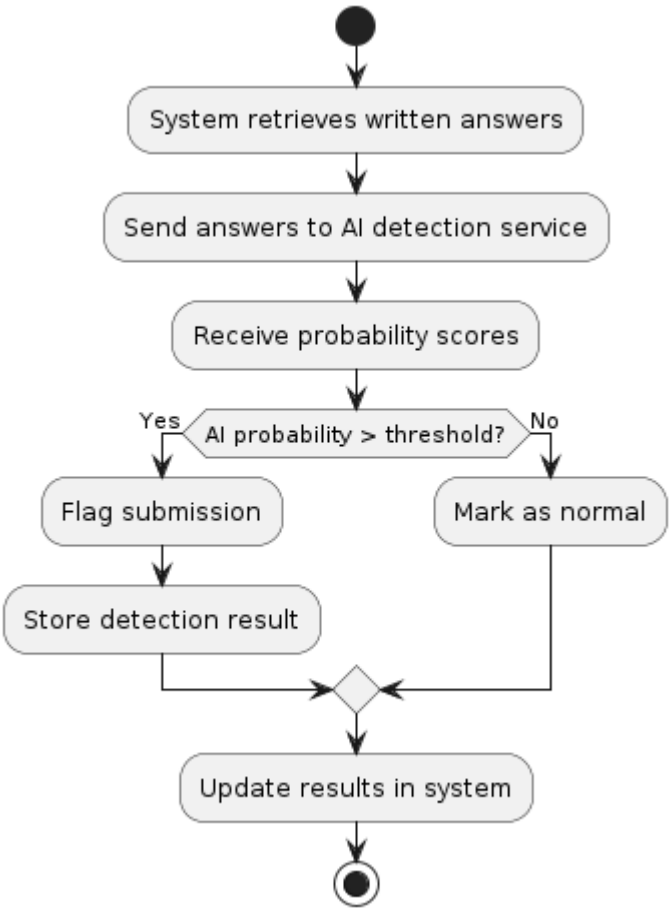


Figure 16 Activity Diagram for AI Detection Workflow

3.2.8 AD-08: Feedback Generation Workflow

This activity diagram illustrates how AutoExam.AI generates personalized feedback after evaluation. The system analyzes performance data to identify strengths, weaknesses, and topic-wise scores. It then creates a structured feedback report for the student. The workflow ensures meaningful and actionable insights are delivered.

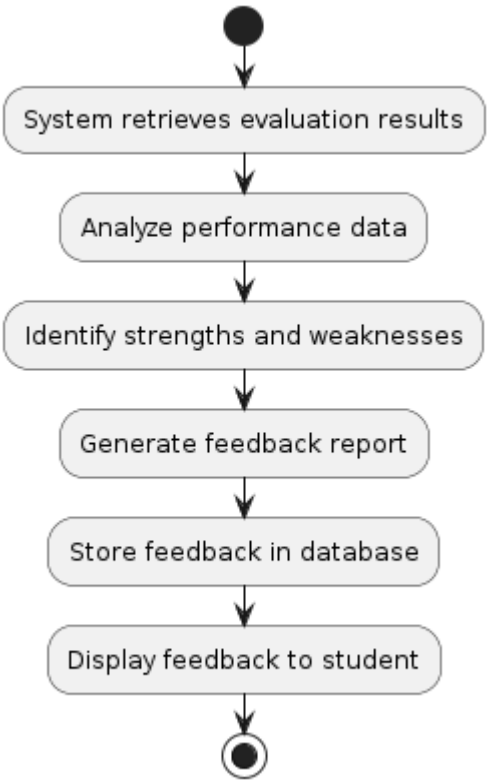


Figure 17 Activity Diagram for Feedback Generation Workflow

3.3 Sequence Diagram

3.3.1 SD-01: User Login & Authentication Flow

This sequence diagram illustrates the process of user authentication in AutoExam.AI, where the user submits login credentials through the frontend. The system validates the credentials, checks account status, and generates a secure session token upon successful authentication. It also handles invalid or unauthorized access scenarios. This flow ensures

secure and controlled access to the platform.

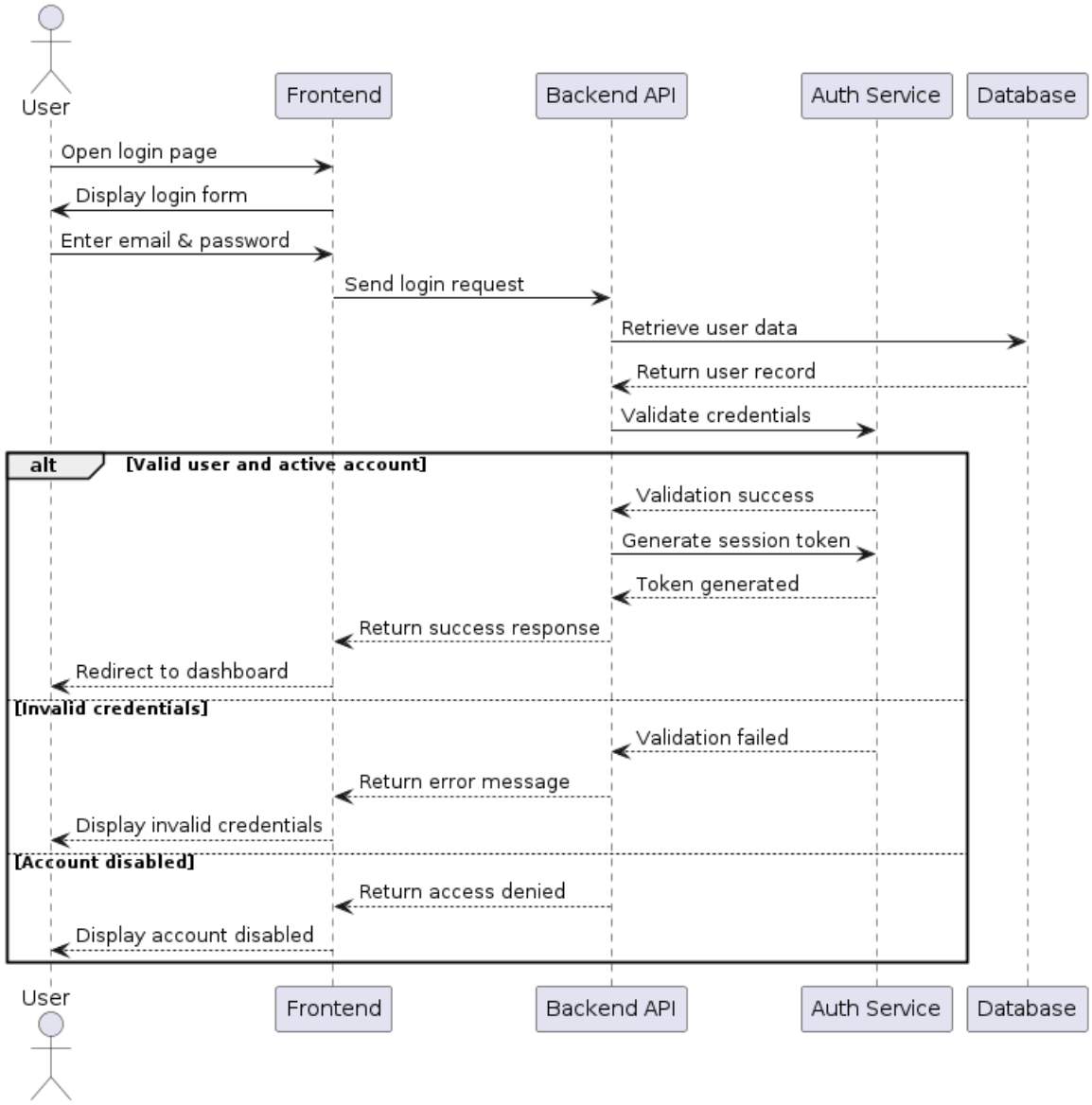


Figure 18 Sequence Diagram for User Login & Authentication Flow

3.3.2 SD-02: User Registration & Verification Flow

This sequence diagram illustrates the process of user registration and email verification in AutoExam.AI. It shows how a new user submits registration details, the system validates and stores the account, and sends a verification email. The account is activated only after successful token validation. This ensures secure onboarding and prevents unauthorized access.

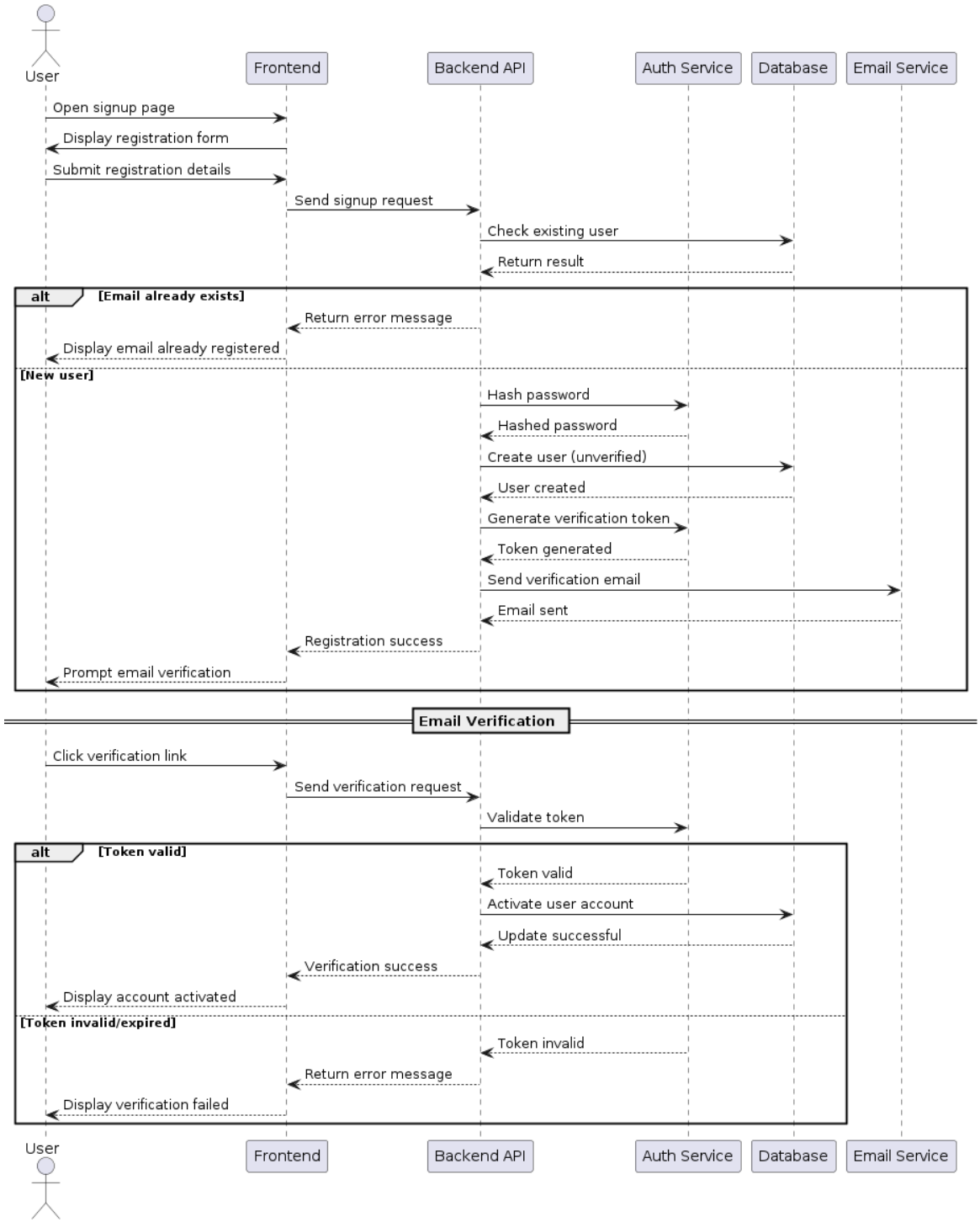


Figure 19 Sequence Diagram for User Registration & Verification Flow

3.3.3 SD-03: Content Upload & Processing Flow

This sequence diagram illustrates how teachers upload course material and how the system processes it using NLP techniques. The workflow includes validation, text extraction, preprocessing, and storage of structured content. It ensures that only valid and usable input

is prepared for exam generation. This flow supports the content ingestion pipeline of AutoExam.AI.

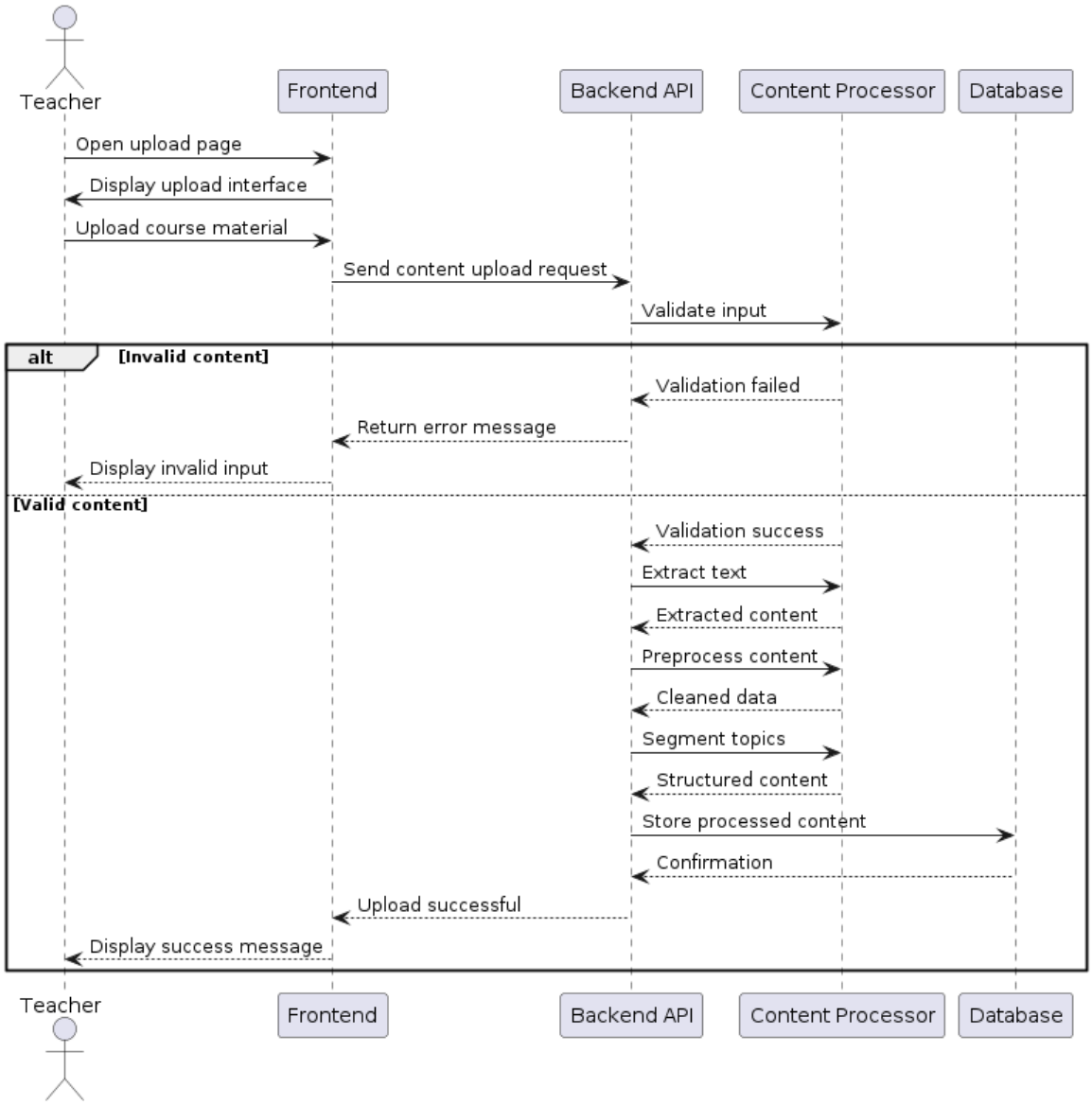


Figure 20 Sequence Diagram for Content Upload & Processing Flow

3.3.4 SD-04: Exam Generation Flow

This sequence diagram illustrates how AutoExam.AI generates exam papers from processed course content. The teacher initiates the process, and the system uses the AI engine to generate questions, apply quality filtering, and assemble the exam. The workflow ensures that generated exams are structured, relevant, and ready for review. It supports automated and efficient exam creation.

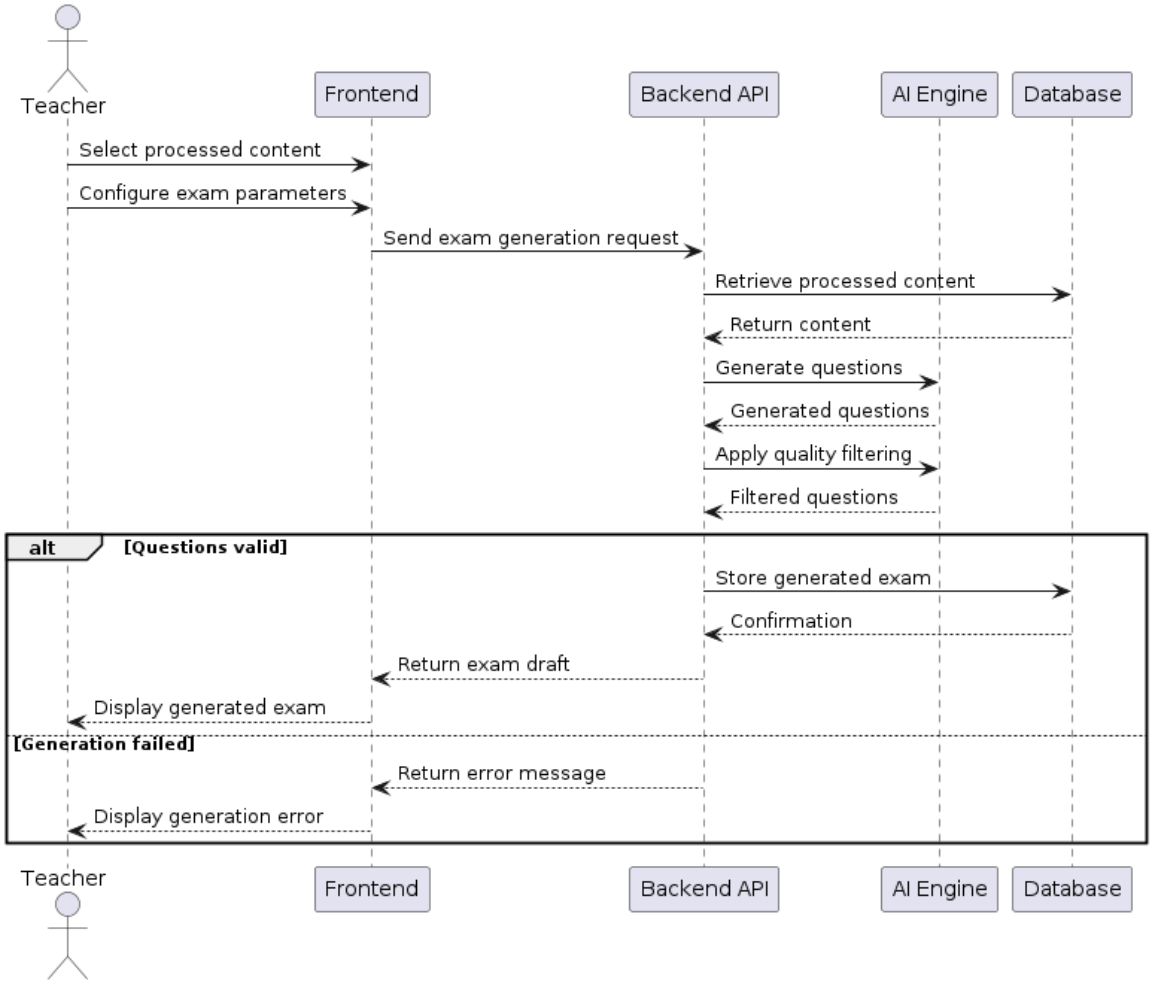


Figure 21 Sequence Diagram for Exam Generation Flow

3.3.5 SD-05: Exam Management & Publishing Flow

This sequence diagram illustrates how teachers review, modify, and publish generated exams in AutoExam.AI. The teacher interacts with the frontend to edit questions and configure exam settings, while the backend validates and stores the final exam. The system ensures that only valid exams are published and made available to students. This workflow maintains control and integrity of the assessment process.

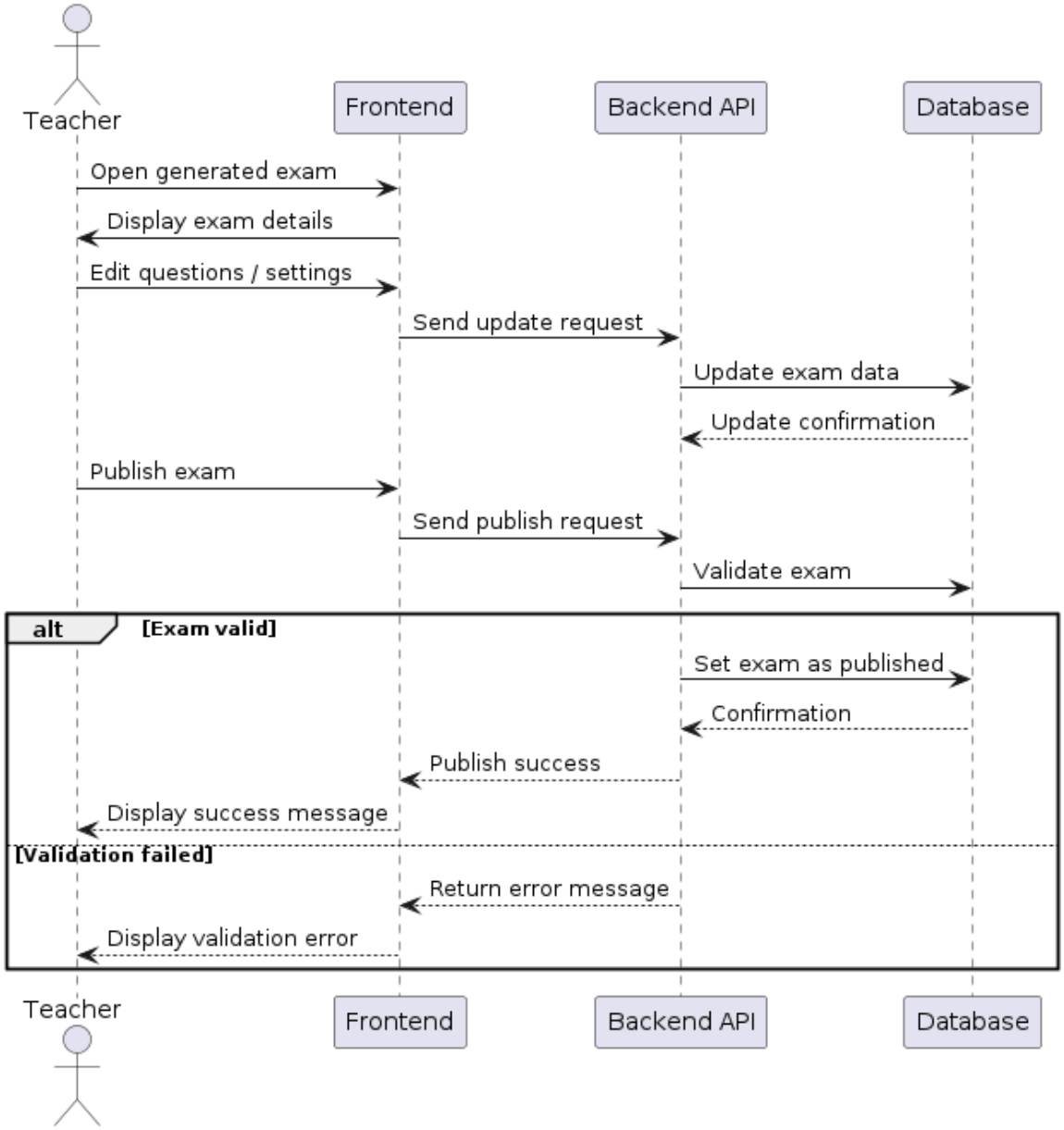


Figure 22 Sequence Diagram for Exam Management & Publishing Flow

3.3.6 SD-06: Exam Attempt Flow

This sequence diagram illustrates how students access and attempt exams in AutoExam.AI. It shows the interaction between the student, frontend, backend, and database during exam execution and submission. The system ensures responses are saved and submitted correctly within the time constraints. This workflow supports reliable and secure exam attempts.

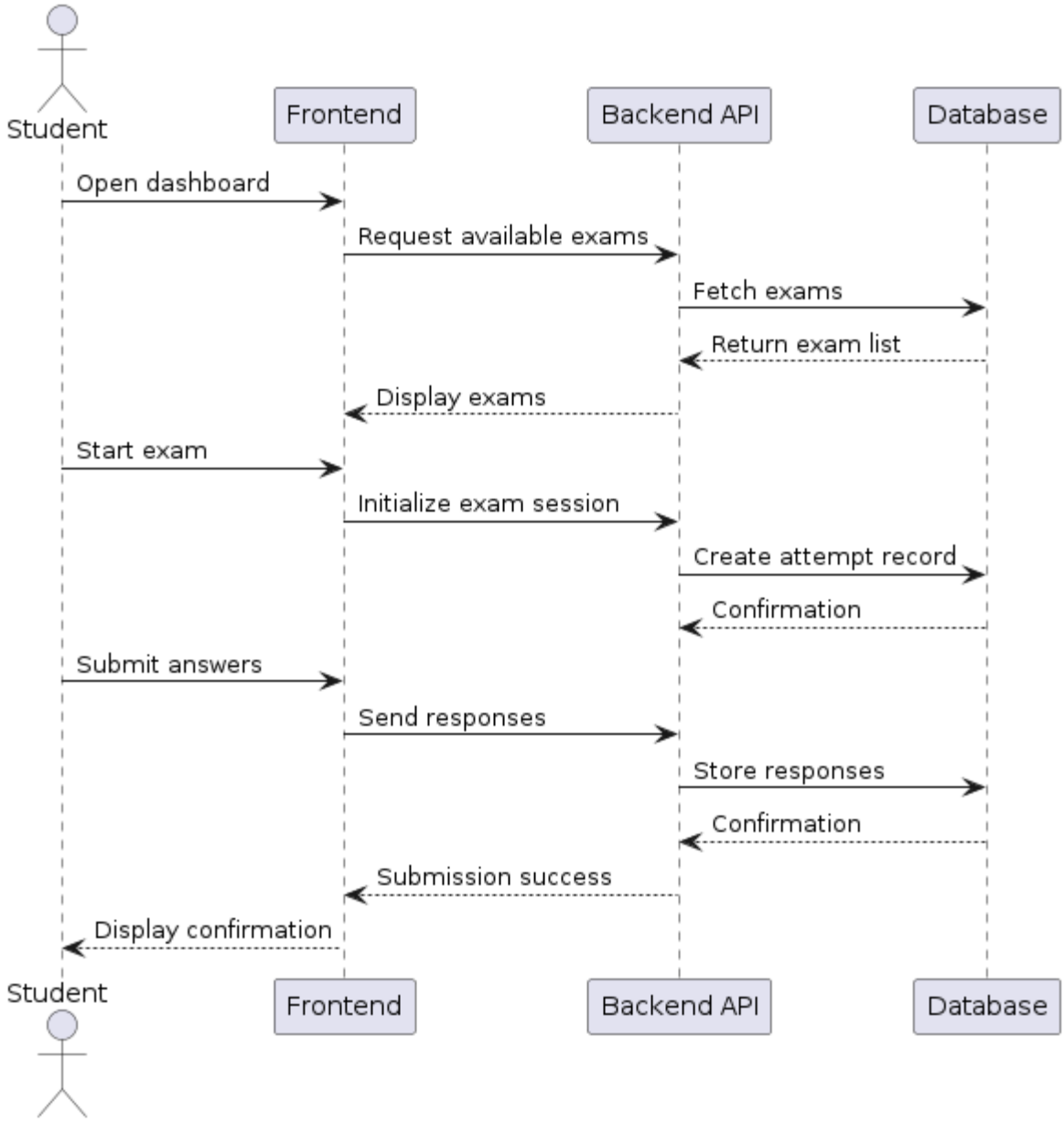


Figure 23 Sequence Diagram for Exam Attempt Flow

3.3.7 SD-07: Evaluation & AI Detection Flow

This sequence diagram illustrates how AutoExam.AI evaluates submitted answers and performs AI-based detection on written responses. The system processes MCQs and subjective answers using evaluation logic and AI models. It then sends written responses to the detection service and flags suspicious submissions. This workflow ensures accurate grading and maintains academic integrity.

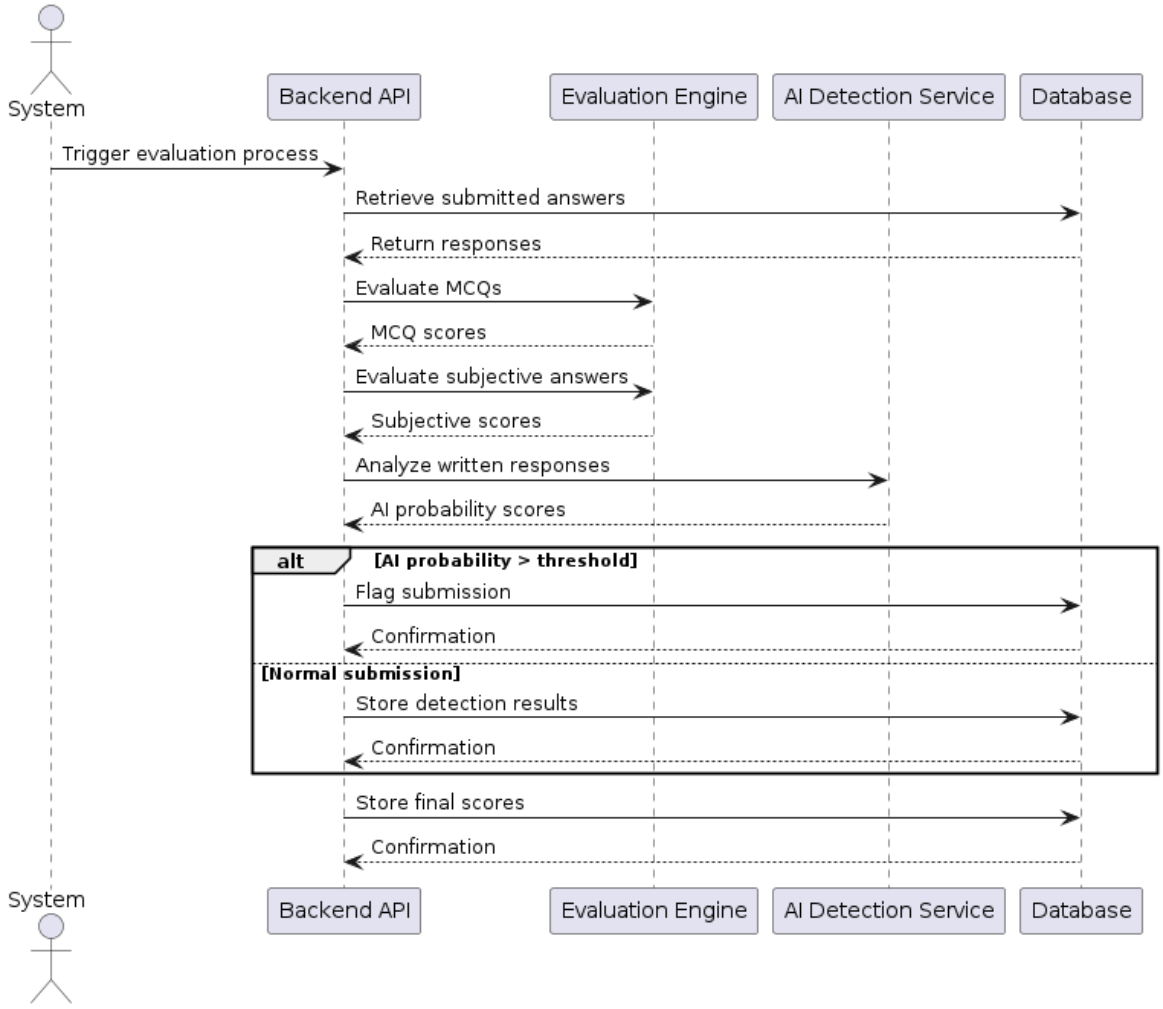


Figure 24 Sequence Diagram for Evaluation & AI Detection Flow

3.3.8 SD-08: Feedback & Analytics Flow

This sequence diagram illustrates how AutoExam.AI generates personalized feedback and presents analytics to users. The system processes evaluation results to produce feedback reports and performance insights. Students can view detailed feedback, while teachers access aggregated analytics. This workflow supports data-driven learning and performance monitoring.

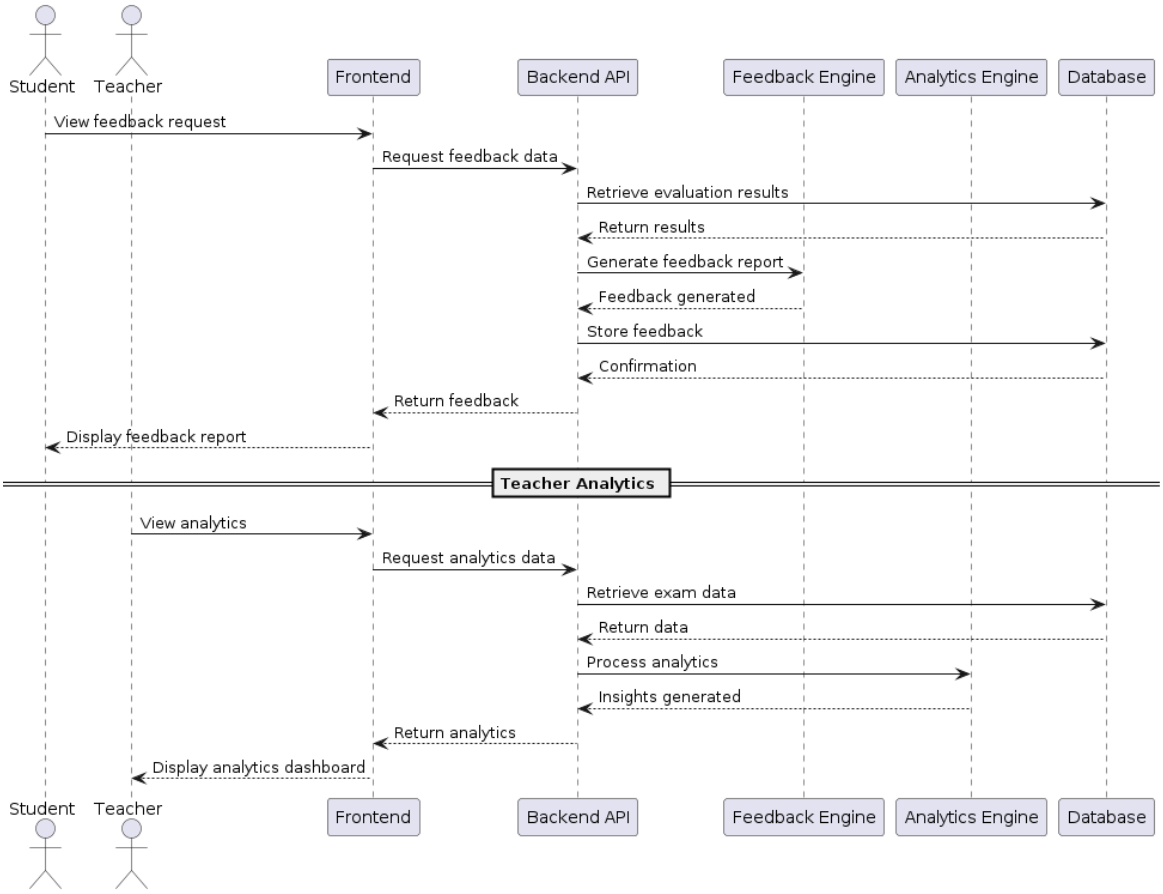


Figure 25 Sequence Diagram for Feedback & Analytics Flow

3.4 Software Architecture Design

The AutoExam.AI system adopts a layered architecture integrated with a dedicated AI processing layer, ensuring clear separation between user interaction, system logic, intelligent processing, and data management. The frontend serves as the interaction point for teachers and students, communicating with backend APIs that manage core modules such as exam generation, evaluation, and feedback. The AI engine performs NLP and LLM-based operations, while the data layer stores structured and unstructured data. External

services are utilized for advanced AI processing and communication, enabling scalability and modularity.

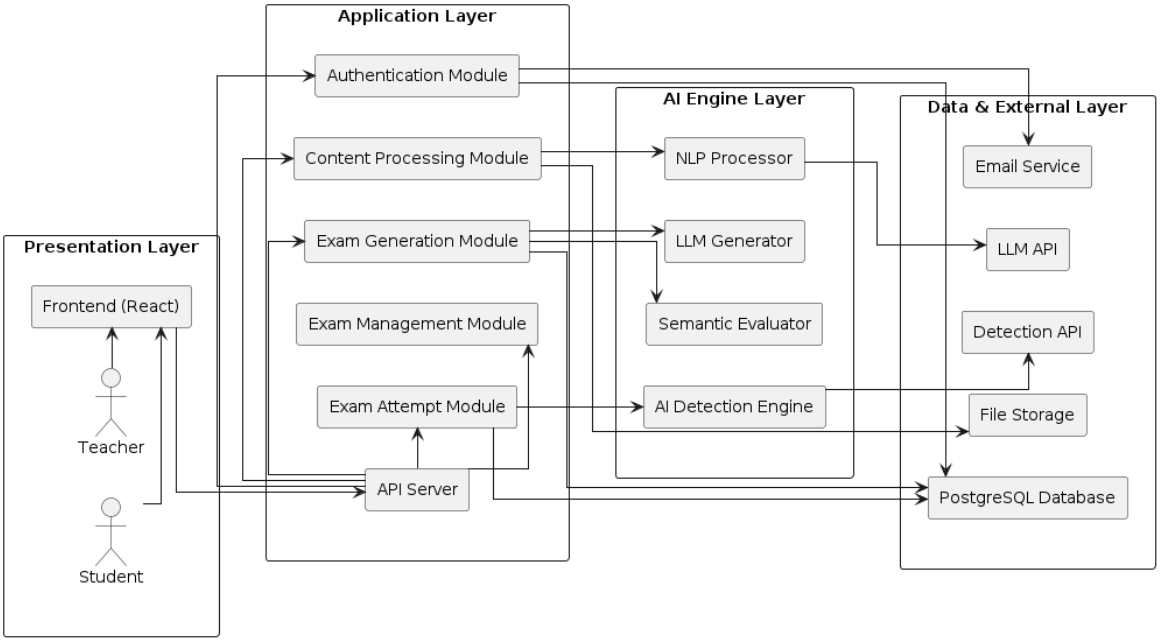


Figure 26 Software Architecture Design Diagram

3.5 Class Diagram

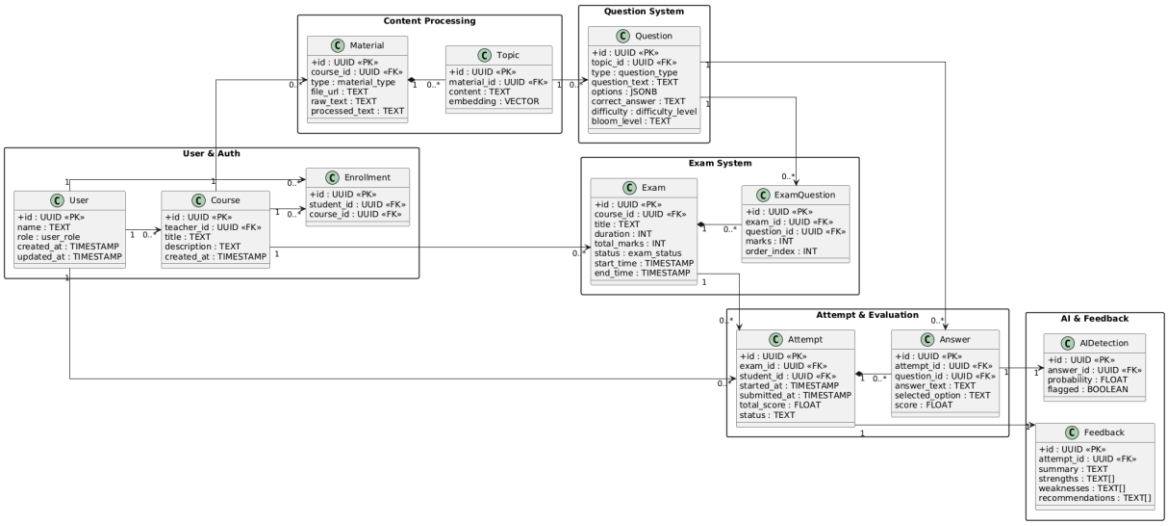


Figure 27 Class Diagram

3.6 Database Diagram

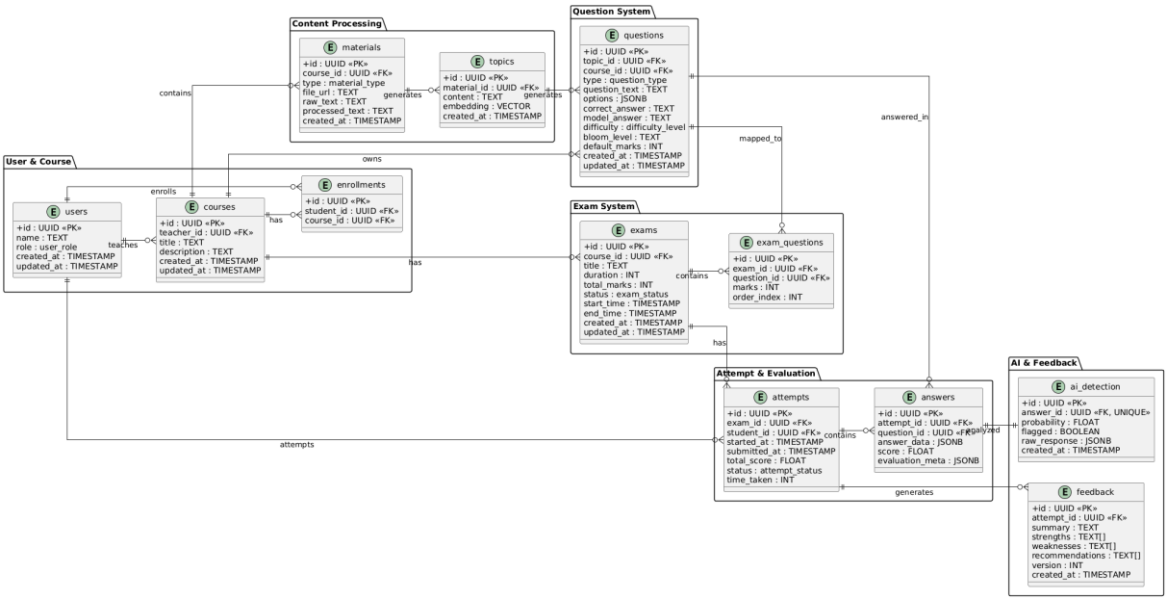


Figure 28 DataBase Diagram

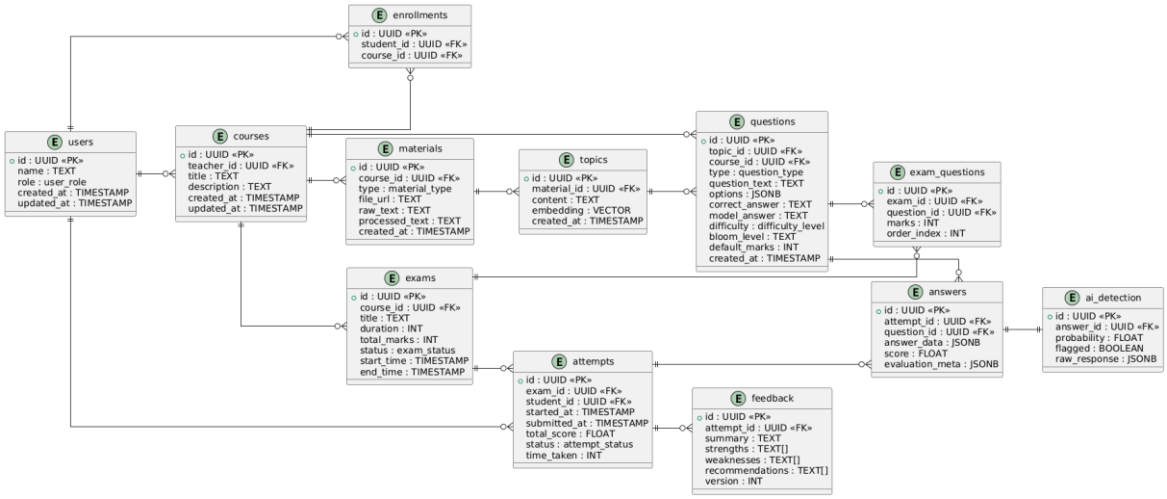


Figure 29 DataBase ERD Diagram

3.7 Collaboration Diagram

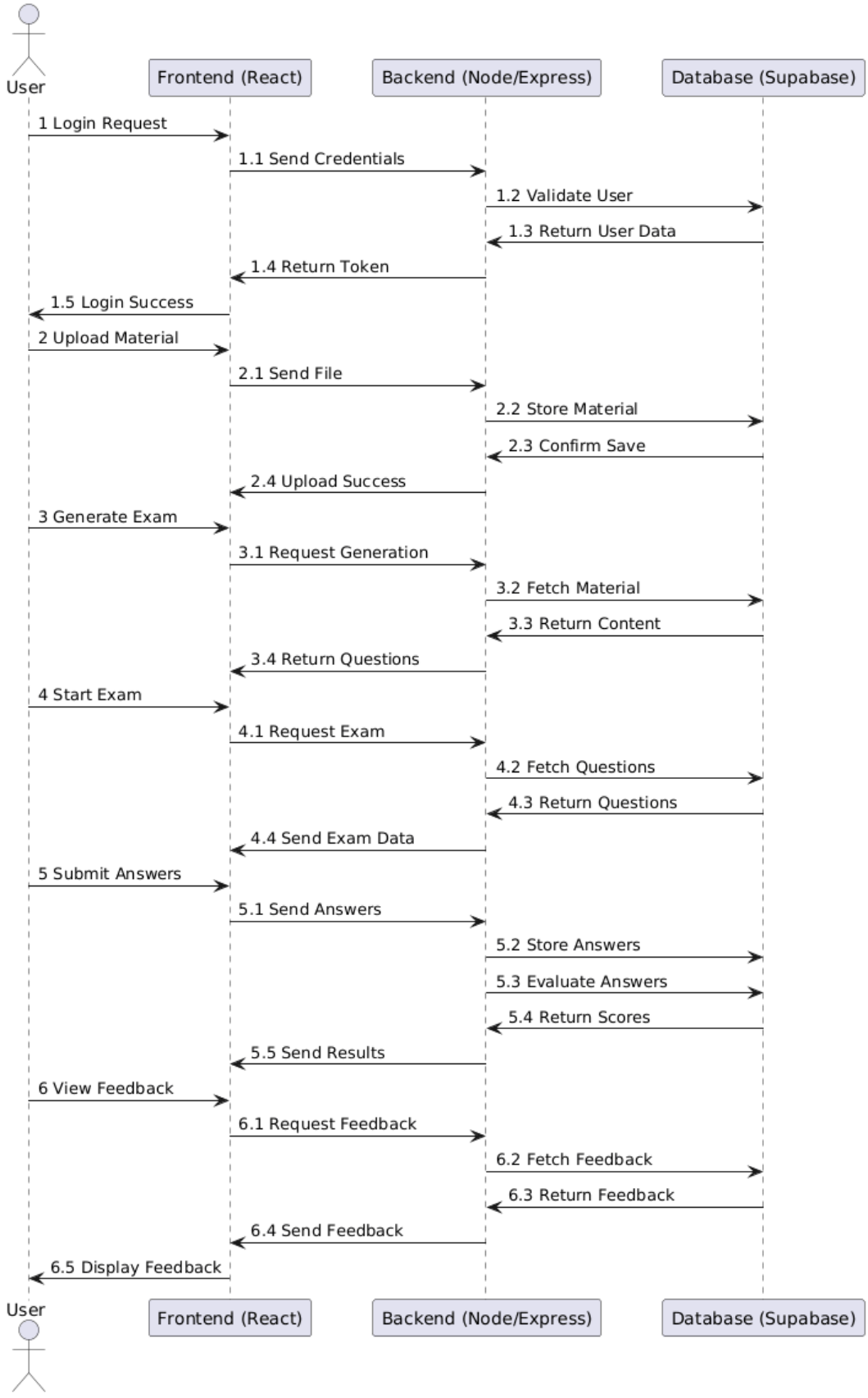


Figure 30 Collaboration Diagram

Chapter 4. System Testing

4.1 Test Cases Design

4.1.1 Test Cases for FR-01: Authentication & Access Control

Table 4 TC-01-01

Field	**Details**
Test Case ID	TC-01-01
Requirement ID	FR-01
Test Case Description	Verify that the system allows a new user to register with valid credentials.
Preconditions	User is not registered in the system.
Test Steps	1. Open registration page. 2. Enter valid name, email, and password. 3. Submit the registration form.
Expected Result	User account is created successfully and confirmation message is displayed.
Actual Result	N/A
Status	N/A

4.1.2 TC-01-02 — Test User Login with Valid Credentials

Table 5 TC-01-02

Field	**Details**
Test Case ID	TC-01-02
Requirement ID	FR-01
Test Case Description	Verify that the system allows user login with valid email and password.

Preconditions	User account exists and is registered.
Test Steps	1. Open login page. 2. Enter valid email. 3. Enter correct password. 4. Click login.
Expected Result	User is authenticated successfully and redirected to dashboard with active session.
Actual Result	N/A
Status	N/A

4.1.3 TC-01-03 — Test User Login with Invalid Credentials

Table 6 TC-01-03

Field	**Details**
Test Case ID	TC-01-03
Requirement ID	FR-01
Test Case Description	Verify that the system restricts login when invalid credentials are provided.
Preconditions	User account exists in the system.
Test Steps	1. Open login page. 2. Enter valid email. 3. Enter incorrect password. 4. Click login.
Expected Result	Login is rejected and an appropriate error message is displayed.
Actual Result	N/A
Status	N/A

4.1.4 TC-01-04 — Test Account Verification Process

Table 7 TC-01-04

Field	**Details**
Test Case ID	TC-01-04
Requirement ID	FR-01
Test Case Description	Verify that the system successfully verifies a user account using a valid verification link.
Preconditions	User has registered and received a verification email.
Test Steps	1. Open verification email. 2. Click verification link. 3. Observe system response.
Expected Result	User account is marked as verified and confirmation message is displayed.
Actual Result	N/A
Status	N/A

4.1.5 4.1.1.5 TC-01-05 — Test Access Restriction for Unverified Users

Table 8 TC-01-05

Field	**Details**
Test Case ID	TC-01-05
Requirement ID	FR-01
Test Case Description	Verify that the system restricts access for users whose accounts are not verified.
Preconditions	User account exists but is not verified.

Test Steps	1. Attempt login with unverified account. 2. Try accessing protected dashboard.
Expected Result	Access is denied and user is prompted to verify their account.
Actual Result	N/A
Status	N/A

4.1.6 TC-01-06: Test Session Management (JWT Handling)

Table 9 TC-01-06

Field	**Details**
Test Case ID	TC-01-06
Requirement ID	FR-01
Test Case Description	Verify that the system manages user sessions securely using JWT tokens.
Preconditions	User is successfully logged in.
Test Steps	1. Login with valid credentials. 2. Inspect token storage. 3. Refresh page and access secured routes.
Expected Result	Valid JWT token is generated, session is maintained, and protected routes remain accessible.
Actual Result	N/A
Status	N/A

4.1.7 TC-01-07 — Test Logout Functionality

Table 10 TC-01-07

Field	**Details**
Test Case ID	TC-01-07

Requirement ID	FR-01
Test Case Description	Verify that the system logs out the user and terminates the active session.
Preconditions	User is logged into the system.
Test Steps	1. Click logout option. 2. Attempt to access dashboard after logout.
Expected Result	User is logged out, session is terminated, and access to protected pages is restricted.
Actual Result	N/A
Status	N/A

4.1.2 Test Cases for FR-02: Content Upload & Processing

4.1.2.1 TC-02-01 — Test Upload Valid Study Material

Table 11 — TC-02-01

Field	**Details**
Test Case ID	TC-02-01
Requirement ID	FR-02
Test Case Description	Verify that the system allows instructors to upload valid study material files.
Preconditions	Instructor is logged into the system.
Test Steps	1. Navigate to upload section. 2. Select a valid file (PDF/DOC/DOCX). 3. Click upload.
Expected Result	File is successfully uploaded and stored for further processing.
Actual Result	N/A
Status	N/A

4.1.2.2 TC-02-02 — Test Upload Invalid File Format

Table 12 — TC-02-02

Field	Details
Test Case ID	TC-02-02
Requirement ID	FR-02
Test Case Description	Verify that the system rejects unsupported file formats during upload.
Preconditions	Instructor is logged into the system.
Test Steps	1. Navigate to upload section. 2. Select an invalid file format (e.g., .exe, .jpg). 3. Click upload.
Expected Result	Upload is rejected and an appropriate error message is displayed.
Actual Result	N/A
Status	N/A

4.1.2.3 TC-02-03 — Test File Size Validation

Table 13 — TC-02-03

Field	Details
Test Case ID	TC-02-03
Requirement ID	FR-02
Test Case Description	Verify that the system enforces file size limits during upload.
Preconditions	Instructor is logged into the system.
Test Steps	1. Navigate to upload section. 2. Select a file exceeding allowed size limit. 3. Click upload.

Expected Result	Upload is blocked and a message indicating file size limit is displayed.
Actual Result	N/A
Status	N/A

4.1.8 TC-02-05 — Test Content Segmentation for Question Generation

Table 14 — TC-02-05

Field	**Details**
Test Case ID	TC-02-05
Requirement ID	FR-02
Test Case Description	Verify that the system correctly segments extracted content into meaningful units for question generation.
Preconditions	Valid study material is uploaded and processed.
Test Steps	1. Upload valid content. 2. Trigger content processing. 3. Inspect segmented output used for question generation.
Expected Result	Content is divided into logical segments (topics/paragraphs) suitable for generating questions.
Actual Result	N/A
Status	N/A

4.1.9 TC-02-06 — Test Storage of Uploaded Content

Table 15 — TC-02-06

Field	**Details**
Test Case ID	TC-02-06
Requirement ID	FR-02

Test Case Description	Verify that uploaded study material and processed content are stored securely in the system.
Preconditions	File upload and processing are completed successfully.
Test Steps	1. Upload valid study material. 2. Complete processing. 3. Verify storage in database and file storage system.
Expected Result	Uploaded file and processed data are stored correctly and are retrievable when required.
Actual Result	N/A
Status	N/A

4.1.3 Test Cases for FR-03: Exam Generation

4.1.10 TC-03-01: Test Automatic Question Generation

Table 16 --TC-03-01

Field	**Details**
Test Case ID	TC-03-01
Requirement ID	FR-03
Test Case Description	Verify that the system automatically generates questions from uploaded study material using AI models.
Preconditions	Study material is uploaded and processed successfully.
Test Steps	1. Navigate to exam generation module. 2. Select processed content. 3. Trigger question generation.
Expected Result	System generates relevant questions based on the provided content.
Actual Result	N/A
Status	N/A

4.1.11 TC-03-02 — Test Question Difficulty Classification

Table 17— TC-03-02

Field	**Details**
Test Case ID	TC-03-02
Requirement ID	FR-03
Test Case Description	Verify that the system classifies generated questions into appropriate difficulty levels.
Preconditions	Questions are generated from content.
Test Steps	1. Generate questions. 2. Inspect assigned difficulty levels (easy, medium, hard).
Expected Result	Each question is correctly categorized into a difficulty level based on content complexity.
Actual Result	N/A
Status	N/A

4.1.12 TC-03-03 — Test Question Quality Validation

Table 18— TC-03-03

Field	**Details**
Test Case ID	TC-03-03
Requirement ID	FR-03
Test Case Description	Verify that the system filters and validates generated questions for quality and relevance.
Preconditions	Questions are generated using AI.
Test Steps	1. Generate questions. 2. Apply quality validation process. 3. Review filtered output.

Expected Result	Only relevant, non-duplicate, and meaningful questions are retained for exam creation.
Actual Result	N/A
Status	N/A

4.1.13 TC-03-04 — Test Exam Creation from Generated Questions

Table 19— TC-03-04

Field	**Details**
Test Case ID	TC-03-04
Requirement ID	FR-03
Test Case Description	Verify that the system allows creation of an exam using the validated generated questions.
Preconditions	Questions are generated and validated successfully.
Test Steps	1. Navigate to exam creation module. 2. Select generated questions. 3. Create exam.
Expected Result	Exam is successfully created with selected questions and stored in the system.
Actual Result	N/A
Status	N/A

4.1.14 TC-03-05 — Test Exam Generation Failure Handling

Table 20 — TC-03-05

Field	**Details**
Test Case ID	TC-03-05
Requirement ID	FR-03

Test Case Description	Verify that the system handles failures during exam generation gracefully.
Preconditions	System encounters an error during question generation (e.g., invalid content or AI failure).
Test Steps	1. Trigger exam generation with invalid or insufficient content. 2. Observe system response.
Expected Result	System displays an appropriate error message and prevents creation of an incomplete exam.
Actual Result	N/A
Status	N/A

4.1.4 Test Cases for FR-04: Exam Management

4.1.4.1 TC-04-01: Test View Generated Exam

Table 21 TC-04-01

Field	**Details**
Test Case ID	TC-04-01
Requirement ID	FR-04
Test Case Description	Verify that the instructor can view the generated exam before publishing.
Preconditions	Exam is successfully generated and saved in the system.
Test Steps	1. Navigate to exam management section. 2. Select generated exam. 3. Open exam preview.
Expected Result	Exam is displayed with all questions, options, and structure correctly.
Actual Result	N/A
Status	N/A

4.1.4.2 TC-04-02 — Test Edit Exam Questions

Table 22: TC-04-02

Field	**Details**
Test Case ID	TC-04-02
Requirement ID	FR-04
Test Case Description	Verify that the instructor can edit or modify generated exam questions.
Preconditions	Exam is available in editable state.
Test Steps	1. Open generated exam. 2. Select a question. 3. Modify question or options. 4. Save changes.
Expected Result	Changes are successfully saved and reflected in the exam.
Actual Result	N/A
Status	N/A

4.1.4.3 TC-04-03 — Test Configure Exam Settings (Time, Marks)

Table 23 : TC-04-03

Field	**Details**
Test Case ID	TC-04-03
Requirement ID	FR-04
Test Case Description	Verify that the instructor can configure exam settings such as duration and total marks.
Preconditions	Exam is created and available for configuration.
Test Steps	1. Open exam settings. 2. Set time duration and marks. 3. Save settings.
Expected Result	Exam settings are saved correctly and applied during exam attempt.

Actual Result	N/A
Status	N/A

4.1.4.4 TC-04-04 — Test Publish Exam

Table 24 : TC-04-04

Field	**Details**
Test Case ID	TC-04-04
Requirement ID	FR-04
Test Case Description	Verify that the instructor can publish the exam for student access.
Preconditions	Exam is fully configured and ready for publishing.
Test Steps	1. Open exam management panel. 2. Select exam. 3. Click publish.
Expected Result	Exam is marked as published and becomes available for students.
Actual Result	N/A
Status	N/A

4.1.4.5 TC-04-05 — Test Assign Exam to Students

Table 25 : TC-04-05

Field	**Details**
Test Case ID	TC-04-05
Requirement ID	FR-04
Test Case Description	Verify that the instructor can assign a published exam to specific students.
Preconditions	Exam is published and student accounts exist.
Test Steps	1. Select published exam. 2. Choose students. 3. Assign exam.

Expected Result	Exam is successfully assigned and becomes accessible to selected students.
Actual Result	N/A
Status	N/A

4.1.5 Test Cases for FR-05: Exam Attempt

4.1.15 4.1.5.1 TC-05-01 — Test Start Exam

Table 26 : TC-05-01

Field	Details
Test Case ID	TC-05-01
Requirement ID	FR-05
Test Case Description	Verify that a student can start an assigned exam successfully.
Preconditions	Exam is assigned and accessible to the student.
Test Steps	1. Login as student. 2. Navigate to exam dashboard. 3. Select assigned exam. 4. Click start exam.
Expected Result	Exam interface loads successfully and timer starts.
Actual Result	N/A
Status	N/A

4.1.5.2 TC-05-02 — Test Answer Submission

Table 27 : TC-05-02

Field	Details
Test Case ID	TC-05-02
Requirement ID	FR-05
Test Case Description	Verify that the system allows students to submit answers during the exam.
Preconditions	Student has started the exam.
Test Steps	1. Attempt questions. 2. Enter/select answers. 3. Save or proceed to next question.
Expected Result	Answers are recorded and associated with the student's exam attempt.
Actual Result	N/A
Status	N/A

4.1.16 4.1.5.3 TC-05-03 — Test Auto-Save Functionality

Table 28 : TC-05-03

Field	Details
Test Case ID	TC-05-03
Requirement ID	FR-05
Test Case Description	Verify that the system automatically saves student responses during the exam.
Preconditions	Exam is in progress.
Test Steps	1. Answer questions. 2. Wait for auto-save trigger. 3. Refresh or revisit exam session.

Expected Result	Previously entered answers are retained without data loss.
Actual Result	N/A
Status	N/A

4.1.5.4 TC-05-04 — Test Timer Functionality

Table 29 : TC-05-04

Field	Details
Test Case ID	TC-05-04
Requirement ID	FR-05
Test Case Description	Verify that the exam timer functions correctly and tracks remaining time.
Preconditions	Exam has been started by the student.
Test Steps	1. Start exam. 2. Observe countdown timer. 3. Monitor time progression.
Expected Result	Timer counts down accurately and reflects remaining exam duration.
Actual Result	N/A
Status	N/A

4.1.5.5 TC-05-05 — Test Manual Exam Submission

Table 30 : TC-05-05

Field	Details
Test Case ID	TC-05-05
Requirement ID	FR-05

Test Case Description	Verify that the student can manually submit the exam before time expires.
Preconditions	Exam is in progress.
Test Steps	1. Attempt questions. 2. Click submit exam. 3. Confirm submission.
Expected Result	Exam is submitted successfully and further editing is disabled.
Actual Result	N/A
Status	N/A

4.1.5.6 TC-05-06 — Test Auto Submission on Timeout

Table 31 : TC-05-06

Field	Details
Test Case ID	TC-05-06
Requirement ID	FR-05
Test Case Description	Verify that the system automatically submits the exam when the time limit is reached.
Preconditions	Exam is active and timer is running.
Test Steps	1. Start exam. 2. Allow timer to reach zero.
Expected Result	Exam is auto-submitted and student is redirected to submission confirmation page.
Actual Result	N/A
Status	N/A

4.1.6 Test Cases for FR-06: Evaluation System (Continued)

4.1.6.1 TC-06-01 — Test MCQ Auto Evaluation

Table 32 : TC-06-01

Field	Details
Test Case ID	TC-06-01
Requirement ID	FR-06
Test Case Description	Verify that the system automatically evaluates MCQ responses and assigns correct scores.
Preconditions	Student has submitted an exam containing MCQ questions.
Test Steps	1. Submit exam with MCQ answers. 2. Trigger evaluation process. 3. Check evaluated results.
Expected Result	System correctly compares answers with answer key and assigns accurate scores.
Actual Result	N/A
Status	N/A

4.1.6.2 TC-06-02 — Test Subjective Answer Evaluation (AI-Based)

Table 33 : TC-06-02

Field	Details
Test Case ID	TC-06-02
Requirement ID	FR-06
Test Case Description	Verify that the system evaluates subjective answers using AI-based semantic analysis.
Preconditions	Student has submitted subjective answers.

Test Steps	1. Submit exam with descriptive answers. 2. Trigger AI evaluation. 3. Review assigned scores.
Expected Result	AI evaluates answers based on relevance and assigns appropriate scores.
Actual Result	N/A
Status	N/A

4.1.6.3 TC-06-03 — Test Score Calculation Accuracy

Table 34 : TC-06-03

Field	Details
Test Case ID	TC-06-03
Requirement ID	FR-06
Test Case Description	Verify that the system calculates total exam scores accurately based on evaluated responses.
Preconditions	Evaluation of all questions is completed.
Test Steps	1. Complete evaluation process. 2. Calculate total score. 3. Compare with expected score.
Expected Result	Total score is calculated correctly based on marks assigned to each question.
Actual Result	N/A
Status	N/A

4.1.6.4 TC-06-04 — Test Storage of Evaluation Results

Table 35 : TC-06-04

Field	Details
Test Case ID	TC-06-04
Requirement ID	FR-06
Test Case Description	Verify that evaluation results are stored correctly in the system for future access.
Preconditions	Evaluation process is completed successfully.
Test Steps	1. Complete evaluation. 2. Check database records. 3. Retrieve results from system.
Expected Result	Evaluation results are stored securely and can be retrieved accurately.
Actual Result	N/A
Status	N/A

4.1.7 Test Cases for FR-07: AI Detection (Academic Integrity)

4.1.7.1 TC-07-01 — Test AI Detection on Student Responses

Table 36 : TC-07-01

Field	Details
Test Case ID	TC-07-01
Requirement ID	FR-07
Test Case Description	Verify that the system analyzes student responses using AI detection mechanisms to identify potential misconduct.
Preconditions	Student has submitted exam responses including subjective answers.

Test Steps	1. Submit exam with written responses. 2. Trigger AI detection process. 3. Analyze detection output.
Expected Result	System processes responses and generates AI detection scores or indicators.
Actual Result	N/A
Status	N/A

4.1.7.2 TC-07-02 — Test Detection Threshold Handling

Table 37 : TC-07-02

Field	Details
Test Case ID	TC-07-02
Requirement ID	FR-07
Test Case Description	Verify that the system correctly applies detection thresholds to classify responses as normal or suspicious.
Preconditions	AI detection results are generated with probability scores.
Test Steps	1. Process responses through detection system. 2. Compare scores against threshold.
Expected Result	Responses exceeding threshold are marked as suspicious, others remain normal.
Actual Result	N/A
Status	N/A

4.1.7.3 TC-07-03 — Test Flagging of Suspicious Responses

Table 38 : TC-07-03

Field	Details
Test Case ID	TC-07-03
Requirement ID	FR-07
Test Case Description	Verify that the system flags responses identified as suspicious during AI detection.
Preconditions	Detection system identifies responses exceeding threshold.
Test Steps	1. Submit responses with high AI detection probability. 2. Trigger detection. 3. Check flagged results.
Expected Result	Suspicious responses are flagged and marked for review in the system.
Actual Result	N/A
Status	N/A

4.1.7.4 TC-07-04 — Test Storage of Detection Results

Table 39 : TC-07-04

Field	Details
Test Case ID	TC-07-04
Requirement ID	FR-07
Test Case Description	Verify that AI detection results are stored correctly for analysis and reporting.
Preconditions	Detection process is completed.
Test Steps	1. Complete detection process. 2. Inspect stored results in database. 3. Retrieve detection data.

Expected Result	Detection results are stored accurately and accessible for review and analytics.
Actual Result	N/A
Status	N/A

4.1.8 Test Cases for FR-08: Feedback System

4.1.8.1 TC-08-01 — Test Feedback Generation

Table 40 : TC-08-01

Field	Details
Test Case ID	TC-08-01
Requirement ID	FR-08
Test Case Description	Verify that the system generates feedback for student responses after evaluation.
Preconditions	Evaluation process is completed successfully.
Test Steps	1. Complete exam evaluation. 2. Trigger feedback generation. 3. View generated feedback.
Expected Result	System generates feedback highlighting performance, strengths, and weaknesses.
Actual Result	N/A
Status	N/A

4.1.8.2 TC-08-02 — Test Feedback Accuracy

Table 41 : TC-08-02

Field	Details
Test Case ID	TC-08-02
Requirement ID	FR-08
Test Case Description	Verify that the generated feedback accurately reflects the student's performance and responses.
Preconditions	Feedback has been generated based on evaluated answers.
Test Steps	1. Review student answers and scores. 2. Compare with generated feedback.
Expected Result	Feedback aligns with actual performance, correctly indicating strengths and areas for improvement.
Actual Result	N/A
Status	N/A

4.1.8.3 TC-08-03 — Test Feedback Availability to Student

Table 42 : TC-08-03

Field	Details
Test Case ID	TC-08-03
Requirement ID	FR-08
Test Case Description	Verify that students can access their feedback after exam evaluation.
Preconditions	Feedback is generated and stored in the system.
Test Steps	1. Login as student. 2. Navigate to results/feedback section. 3. View feedback.

Expected Result	Student can successfully view detailed feedback for the completed exam.
Actual Result	N/A
Status	N/A

4.1.9 Test Cases for FR-09: Analytics Dashboard

4.1.9.1 TC-09-01 — Test View Exam Results

Table 43 : TC-09-01

Field	Details
Test Case ID	TC-09-01
Requirement ID	FR-09
Test Case Description	Verify that the system displays exam results for students and instructors correctly.
Preconditions	Exam is completed and evaluation results are available.
Test Steps	1. Login as user. 2. Navigate to results section. 3. View exam results.
Expected Result	Exam results including scores and status are displayed accurately.
Actual Result	N/A
Status	N/A

4.1.9.2 TC-09-02 — Test Student Performance Analytics

Table 44 : TC-09-02

Field	Details
Test Case ID	TC-09-02

Requirement ID	FR-09
Test Case Description	Verify that the system provides detailed analytics for individual student performance.
Preconditions	Student has completed one or more exams.
Test Steps	1. Navigate to analytics dashboard. 2. Select a student. 3. View performance metrics.
Expected Result	System displays detailed metrics such as scores, trends, and feedback insights for the student.
Actual Result	N/A
Status	N/A

4.1.9.3 TC-09-03 — Test Class-Level Analytics

Table 45 : TC-09-03

Field	Details
Test Case ID	TC-09-03
Requirement ID	FR-09
Test Case Description	Verify that the system generates analytics for overall class performance.
Preconditions	Multiple students have completed exams.
Test Steps	1. Open analytics dashboard. 2. Select class-level view. 3. Review aggregated data.
Expected Result	System displays aggregated statistics such as average scores, distribution, and performance trends.
Actual Result	N/A
Status	N/A

4.1.9.4 TC-09-04 — Test Visualization of Performance Trends

Table 46 : TC-09-04

Field	Details
Test Case ID	TC-09-04
Requirement ID	FR-09
Test Case Description	Verify that the system visualizes performance trends using charts and graphs.
Preconditions	Analytics data is available in the system.
Test Steps	1. Navigate to analytics dashboard. 2. Select trend visualization. 3. Observe charts and graphs.
Expected Result	Performance trends are displayed using clear and accurate visual representations.
Actual Result	N/A
Status	N/A

4.2 Unit / Integration / Acceptance Testing

This section presents the testing strategies used to validate the correctness, reliability, and performance of the AutoExam.AI system. It includes unit testing for individual components, integration testing for inter-module interactions, and acceptance testing to ensure the system meets user and project requirements. These testing levels collectively ensure that the system functions as intended across all modules and workflows.

4.2.1 Unit Testing

Unit testing focuses on validating individual components of the system in isolation. It ensures that each module performs its intended functionality correctly before integration with other components.

AutoExam.AI utilizes:

- **Frontend Testing**: React Testing Library / Jest
- **Backend Testing**: Node.js (Jest, Supertest)

- **AI Module Testing**: Python-based validation (NLP, LLM outputs)
- **Utility Testing**: Validation, parsing, scoring logic

4.2.1.1 Frontend Unit Testing

Authentication Components:

Test Scenario	**Description**
Input Validation	Verify email and password fields enforce required constraints
API Trigger	Ensure login/signup API is called with correct payload
Error Handling	Validate error messages for invalid credentials
Session Handling	Ensure user session persists after login

Content Upload Interface:

Test Scenario	**Description**
File Format Validation	Accept only supported formats (PDF, DOCX)
File Size Validation	Reject files exceeding allowed size
Upload Progress	Verify progress indicator updates correctly
Upload Feedback	Confirm success/error messages are displayed

Exam Interface (Student Side):

Test Scenario	**Description**
Question Rendering	Verify questions display correctly
Answer Selection	Ensure selected answers are stored
Timer Display	Validate countdown timer functionality
Auto-Save UI	Confirm answers persist during navigation

Analytics Dashboard UI:

Test Scenario	Description
Data Rendering	Verify charts and metrics display correctly
Filtering	Ensure filtering options update results dynamically
Visualization Accuracy	Validate graphs reflect correct data

4.2.1.2 Backend Unit Testing

Backend testing validates APIs, business logic, and system services.

Authentication Module:

- Validate login API returns JWT token
- Verify access restriction for unauthorized users
- Test logout and session invalidation

Content Processing Module:

- Validate file validation logic
- Test content extraction and segmentation
- Ensure storage and retrieval of processed content

Exam Generation Module:

- Validate AI-based question generation
- Test difficulty classification logic
- Ensure question filtering and validation

Evaluation Module:

- Test MCQ evaluation logic
- Validate subjective answer scoring
- Verify total score calculation

AI Detection Module:

- Validate detection score generation
- Test threshold classification logic
- Ensure suspicious flags are triggered correctly

4.2.1.3 AI Module Unit Testing

This testing focuses on validating AI specific components.

Question Generation (LLM):

- Validate relevance of generated questions
- Ensure output format consistency
- Test handling of insufficient input data

Semantic Evaluation Engine:

- Verify similarity scoring accuracy
- Test grading consistency for similar answers

AI Detection Engine:

- Validate probability scoring
- Test threshold behavior
- Ensure detection consistency across inputs

4.2.2 Integration Testing

Integration testing ensures that multiple modules interact correctly and data flows seamlessly across the system.

4.2.2.1 Content Processing Pipeline Integration:

Flow:

Upload → Processing → Segmentation → Storage → Exam Generation

Ensures:

- Content flows correctly across modules
- No data loss occurs

- Process completes without failure

4.2.2.2 Exam Lifecycle Integration:

Flow:

Create Exam → Publish → Assign → Attempt → Submit → Evaluate → Feedback

Validates:

- Complete exam workflow
- Synchronization between modules
- Accurate result propagation

4.2.2.3 Evaluation & AI Detection Integration

Flow:

Submission → Evaluation → AI Detection → Result Storage

Ensures:

- Evaluation and detection work together
- Results are consistent and stored correctly

4.2.2.4 Feedback & Analytics Integration

Flow:

Evaluation → Feedback → Analytics Dashboard

Validates:

- Feedback reflects evaluation results
- Analytics uses correct data

4.2.3 Acceptance Testing

Acceptance testing ensures that AutoExam.AI satisfies user requirements and project objectives.

4.2.3.1 User Acceptance Testing (UAT)

Conducted with:

- Students

- Instructors

Users Test:

- Content upload and processing
- Exam generation
- Exam attempt workflow
- Evaluation and feedback
- Analytics dashboard

Feedback Collected On:

- Ease of use
- Accuracy of AI-generated questions
- Evaluation reliability
- System responsiveness

4.2.3.2 Supervisor / Academic Evaluation

System is evaluated by:

- Project Supervisor
- FYP Evaluation Panel

Validation Criteria:

- Alignment with proposal objectives
- Completeness of system features
- Accuracy of AI components
- System performance and usability

Chapter 5. Implementation

This section should provide the implementation details of your project. For FYP Mid Report, the details should comprise 30% implementation of your project while for FYP Final Report the details should comprise 100% implementation of your project. This section should have the following subsections providing detail of your project.

5.1 Work Breakdown Structure (WBS)

A Work Breakdown Structure (WBS) is a hierarchical decomposition of a project into smaller, manageable tasks or modules, making it easier to plan, assign responsibilities, and track progress during implementation.

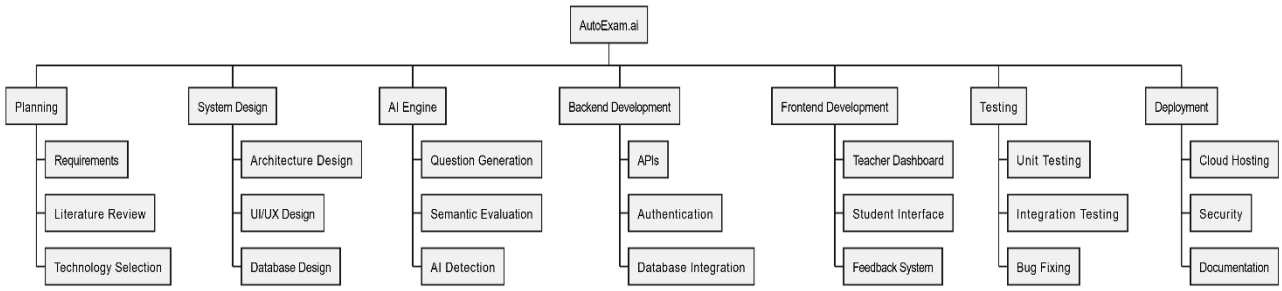


Figure 31 --WorkBreakDown Structure

5.2 Team Roles and Responsibilities

The table below will contain the allocation of roles of both team members and the tasks that each of you will complete during the course of 12 months. The sharing of the tasks helps in justifying the effort taken during the year and to make sure that each of the two members has made considerable contribution.

Team Member	Activity	Tentative Date
Hammad Zubair	Requirement gathering & literature review	**Feb 2026 – Mar 2026**
Ahsan Junaid	UI/UX sketches & frontend initial mockups	**Mar 2026 – Apr 2026**
Hammad Zubair	NLP preprocessing module (text cleaning, chunking)	**Apr 2026 – May 2026**
Ahsan Junaid	Build frontend skeleton (login, dashboards, navigation)	**Apr 2026 – May 2026**
Hammad Zubair	Question generation core (LLM prompts for MCQs, short answer)	**May 2026 – Jul 2026**
Ahsan Junaid	Frontend integration with API (student exam interface)	**May 2026 – Jul 2026**
Hammad Zubair	Evaluation & feedback module (short answers, essays)	**Jul 2026 – Sep 2026**
Ahsan Junaid	UI/UX refinement & usability testing	**Jul 2026 – Sep 2026**
Hammad Zubair	Parallel tasks: bias / difficulty filters, metrics & analytics backend	**Aug 2026 – Oct 2026**
Ahsan Junaid	Parallel tasks: writing tests, QA, user feedback sessions	**Aug 2026 – Oct 2026**
Hammad Zubair	Deployment setup, security modules, database scaling	**Oct 2026 – Dec 2026**

Ahsan Junaid	Final polishing of frontend, responsive design, accessibility	**Oct 2026 – Dec 2026**
Hammad Zubair	Pilot testing + improvements based on pilot feedback	**Dec 2026 – Jan 2027**
Ahsan Junaid	Final UI/UX fixes + documentation + user manual	**Dec 2026 – Jan 2027**
Hammad Zubair & Ahsan	Final integration, testing, project wrap-up, presentation prep	**Jan 2027 – Feb 2027**

Table 5.1. Individual Tasks

It is **MANDATORY** to provide a **GANTT CHART** to clearly illustrate the project sch

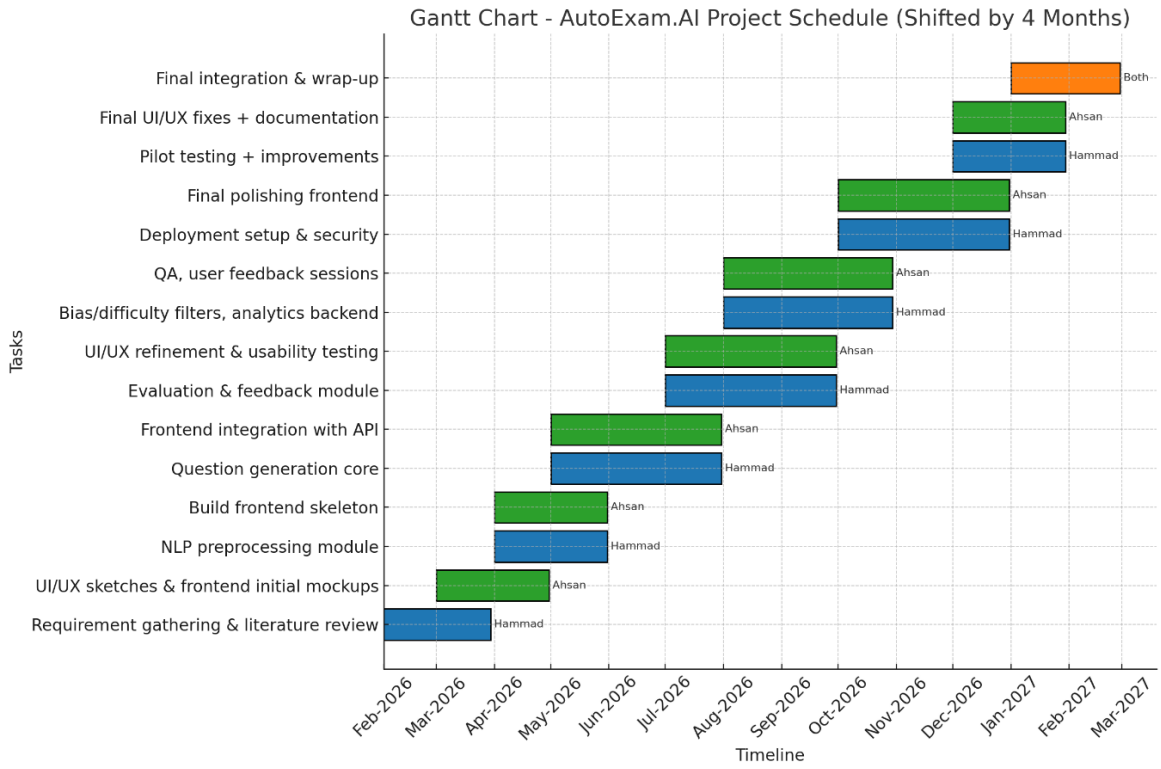


Figure 32--GANTT Chart

5.3 Tools and Technologies

- **Frontend:** React.js, TailwindCSS
- **Backend:** Node.js (Express) or FastAPI
- **Database:** PostgreSQL / Supabase (preferably)
- **AI/NLP Models:** OpenAI GPT models, Hugging Face Transformers (T5/BART)
- **Hosting:** Vercel/Netlify (frontend), Render/Heroku (backend)

- **Version Control:** Git + GitHub
- **Visualization:** Chart.js / Recharts

5.4 Implementation Details

5.5 Implementation Details

5.5.1 Frontend Implementation

The frontend of AutoExam.ai was developed using React.js and TailwindCSS to create a responsive and user-friendly interface for both instructors and students. Multiple interfaces including dashboards, authentication pages, exam attempt screens, analytics, and feedback modules were implemented using component-based architecture. Responsive layouts and reusable UI components were designed to improve maintainability and usability across devices.

5.5.2 Backend Implementation

The backend services were implemented using Node.js with Express/FastAPI to manage APIs, authentication, exam workflows, evaluation processing, and AI module integration. RESTful APIs were developed to support communication between frontend components and backend services. Secure session handling and role-based access control mechanisms were also implemented for system protection.

5.5.3 Database Implementation

PostgreSQL/Supabase was used to manage system data including users, examinations, submissions, evaluation results, analytics, and feedback records. Database schemas and relational structures were designed to ensure efficient storage and retrieval of examination-related data. Cloud-based database services were utilized to simplify scalability and remote access.

5.5.4 Content Processing Implementation

The content processing module was implemented to extract and preprocess educational material uploaded by instructors. Uploaded PDFs, documents, and text files were cleaned, segmented, and converted into structured content using NLP preprocessing techniques. This processed content was then utilized for AI-based question generation and evaluation tasks.

5.5.5 AI Based Question Generation Implementation

AI-powered question generation was implemented using OpenAI GPT models and transformer-based NLP techniques. The system generates MCQs, short questions, and subjective assessments from uploaded study material. Bloom's Taxonomy concepts and prompt engineering techniques were utilized to improve question quality, difficulty balancing, and curriculum relevance.

5.5.6 Automated Evaluation Implementation

The evaluation module was implemented to automatically assess objective and subjective student responses. MCQs were evaluated using rule-based logic, while descriptive answers utilized semantic similarity and AI-assisted scoring mechanisms. The system also calculates total marks and generates detailed performance summaries for students.

5.5.7 AI Detection Module Implementation

The AI Detection module was implemented to identify AI-generated or suspicious examination responses. Detection mechanisms analyze response patterns and generate probability scores to classify potentially AI-assisted content. Threshold-based logic and suspicious activity flags were integrated to support academic integrity monitoring.

5.5.8 Analytics and Feedback Implementation

The analytics and feedback modules were implemented to provide intelligent insights and personalized recommendations for students and instructors. Performance metrics, graphical reports, and topic-wise analysis were visualized using charting libraries. Personalized feedback generation was integrated to help students identify strengths, weaknesses, and improvement areas.

5.5.9 System Integration and Deployment

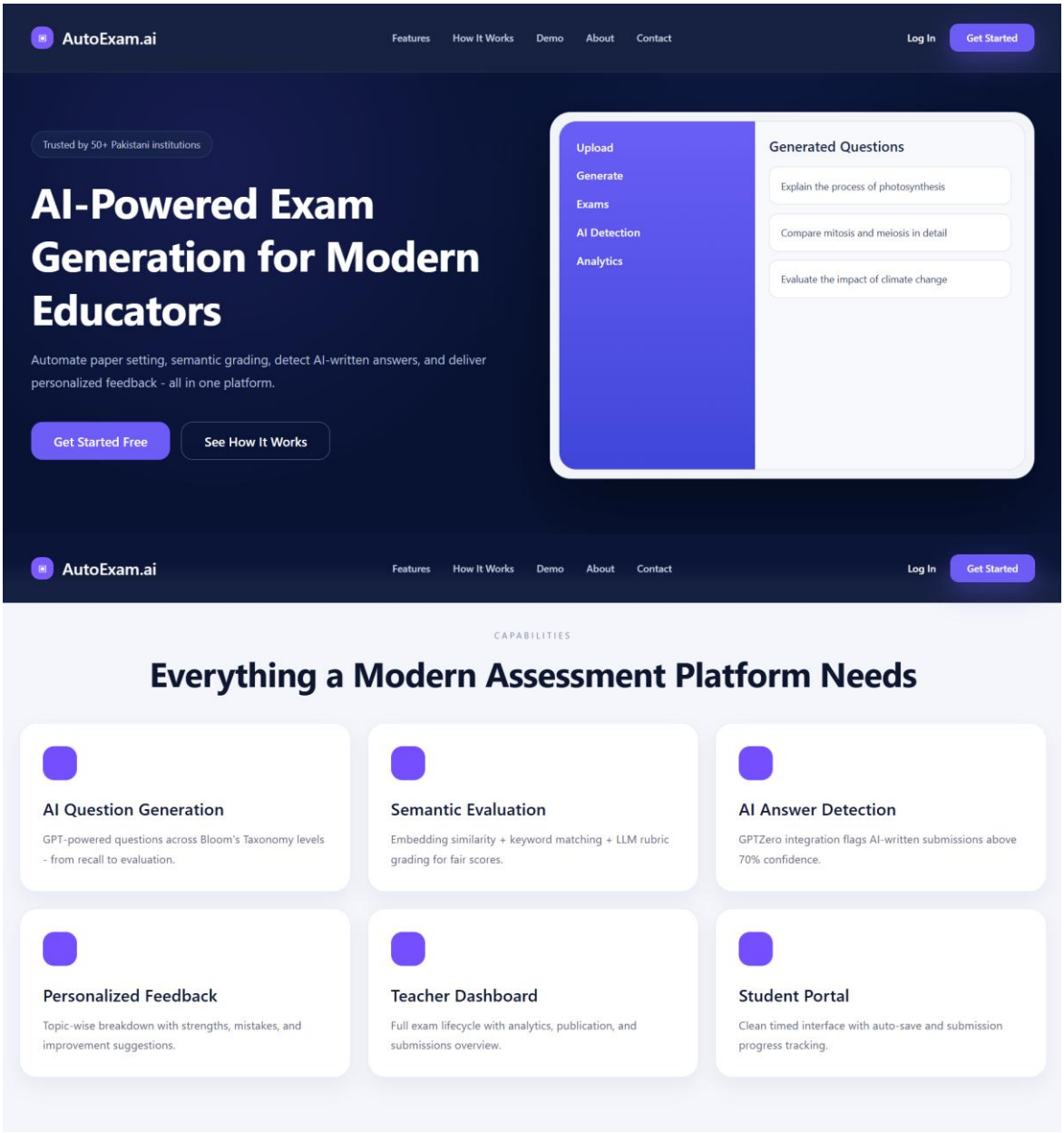
All frontend, backend, AI, and database modules were integrated using API-based communication to ensure smooth workflow execution across the platform. GitHub was used for version control and collaborative development. The system deployment was planned using Vercel/Netlify for frontend hosting and Render/Heroku for backend services to enable cloud-based accessibility and scalability.

5.6 Screenshots of Prototype / System

5.6.1 Landing Page

This prototype represents the Landing Page of the AutoExam.ai platform, which introduces the system's AI-powered examination environment. The page highlights major functionalities such as automated question generation, AI-based evaluation, and personalized feedback mechanisms. It also provides navigation options including Login, Get Started, and Explore Features for users. The interface is designed to create a simple, responsive, and professional user experience while presenting the core objectives of the

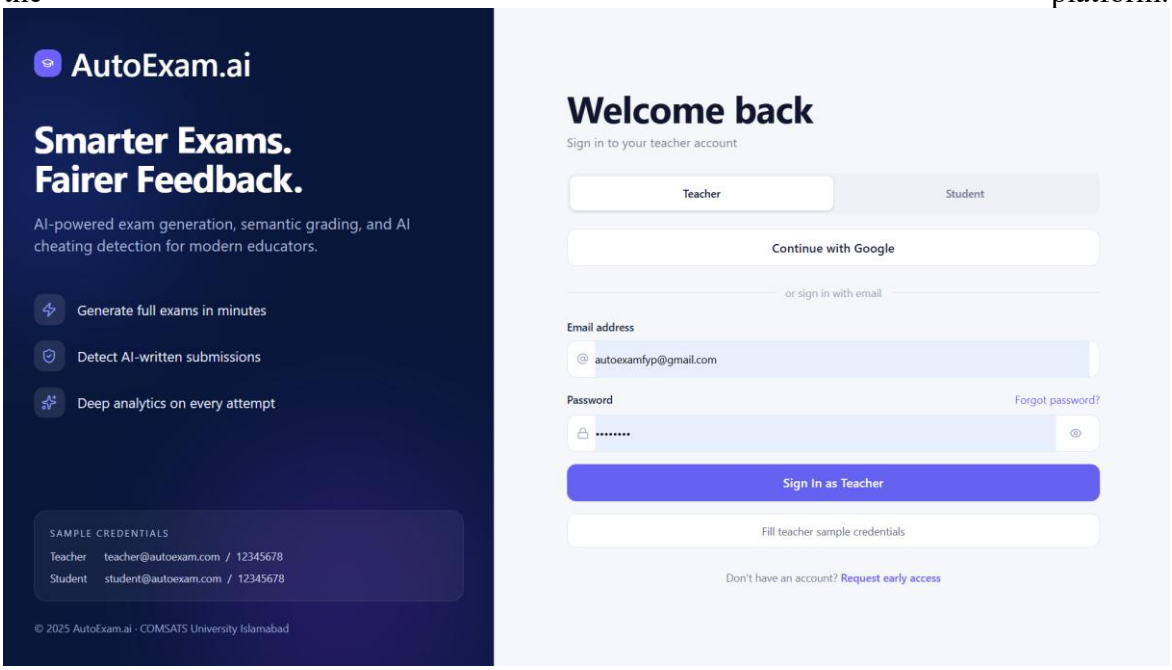
proposed system.



5.6.2 Login & Authentication Page

This prototype represents the Login and Authentication Page of AutoExam.ai, where users can securely access the platform using their credentials. The interface supports role-based authentication for teachers and students to ensure controlled system access. The page is designed with a simple and secure layout to enhance usability and user management within

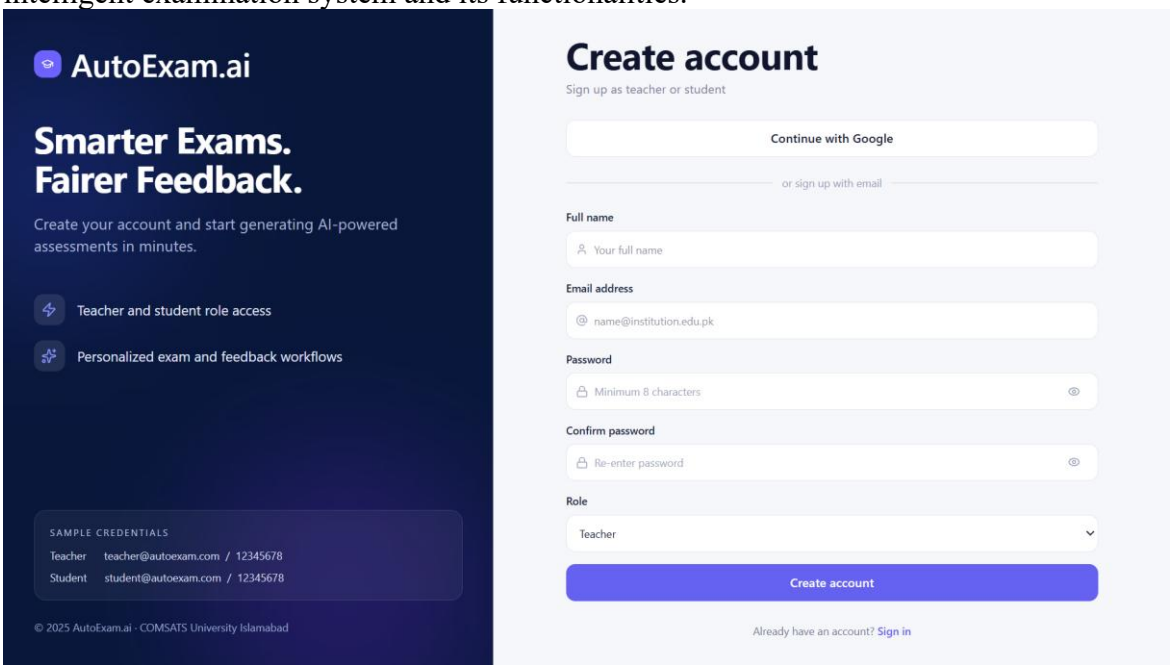
the platform.



5.6.3 Create Account Page

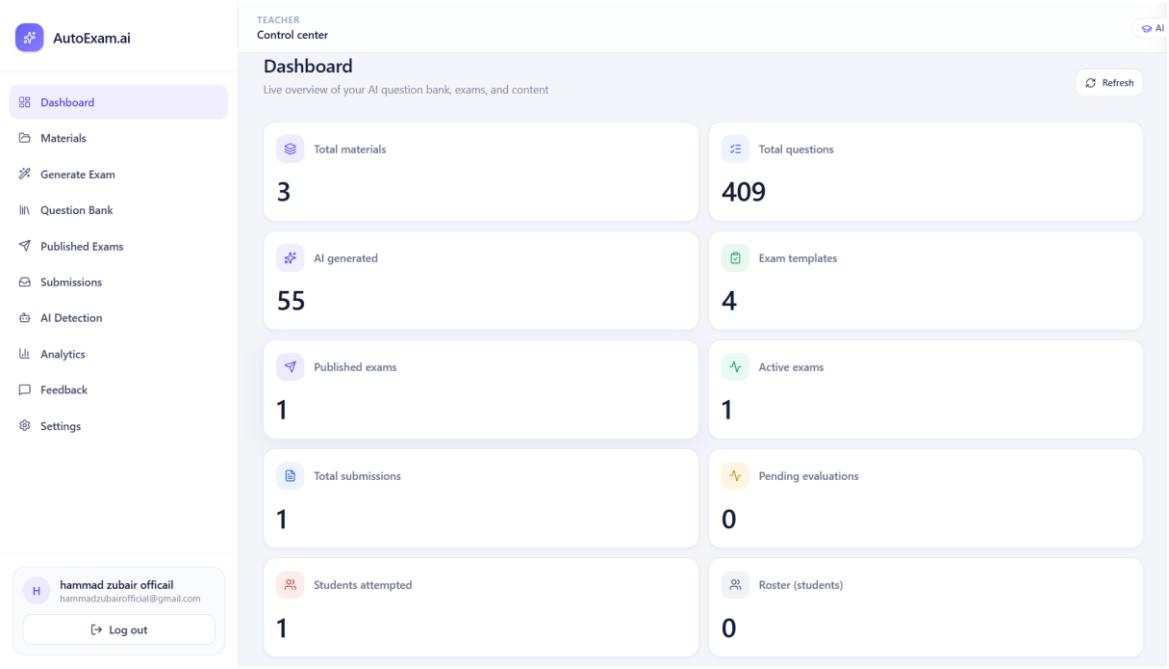
This prototype represents the user registration interface of AutoExam.ai, where new teachers and students can create their accounts to access the platform. The page collects essential user information required for secure account creation and role-based system access.

The interface includes registration fields such as name, email, password, and role selection along with input validation to ensure accurate and secure data entry. This module serves as the onboarding mechanism of the platform and enables authorized users to access the intelligent examination system and its functionalities.



5.6.4 Teacher Dashboard

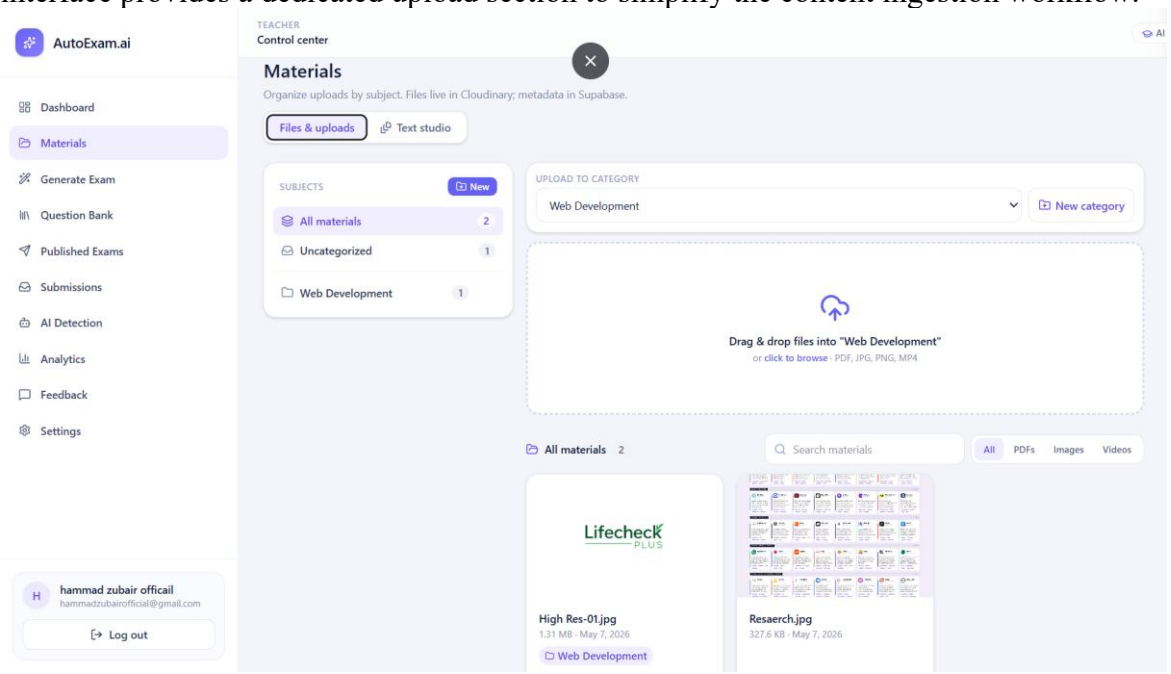
This prototype shows the Teacher Dashboard, which acts as the central management panel for instructors within the AutoExam.ai platform. The dashboard provides quick access to exam creation, uploaded materials, analytics, and student performance reports. It also displays important statistics and AI-generated insights to assist teachers in managing assessments efficiently.

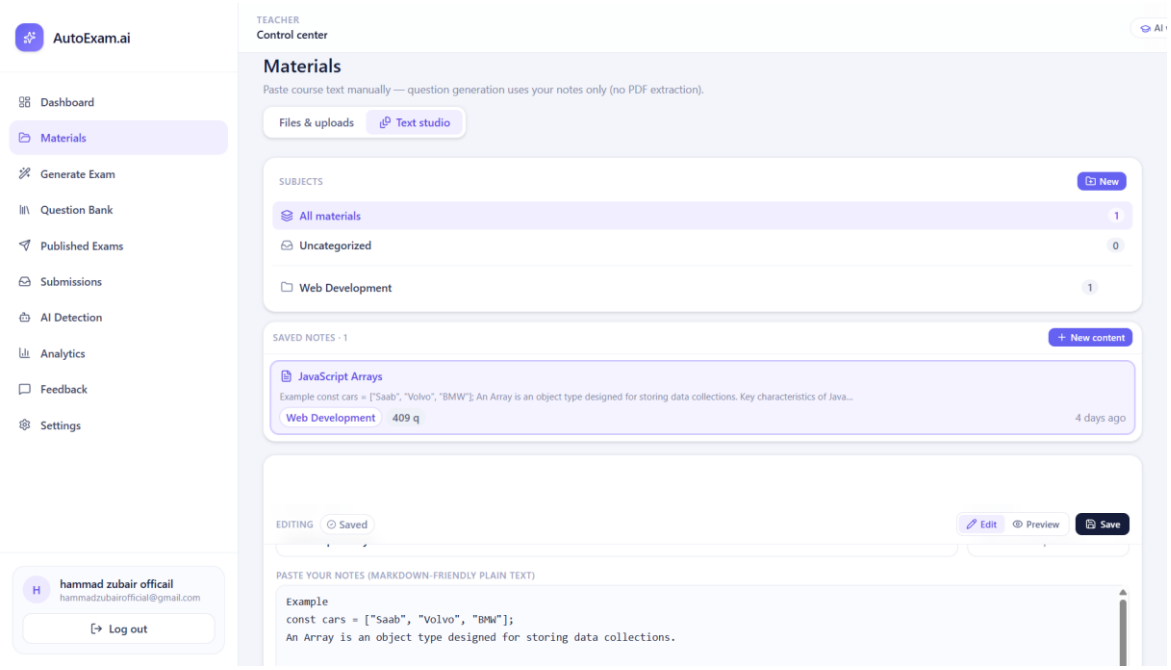


5.6.5 Upload Course Material Page

This prototype illustrates the Upload Course Material Page, where teachers can upload PDFs, documents, or text content for exam generation. The system processes the uploaded material using AI and NLP techniques to extract relevant educational content. The

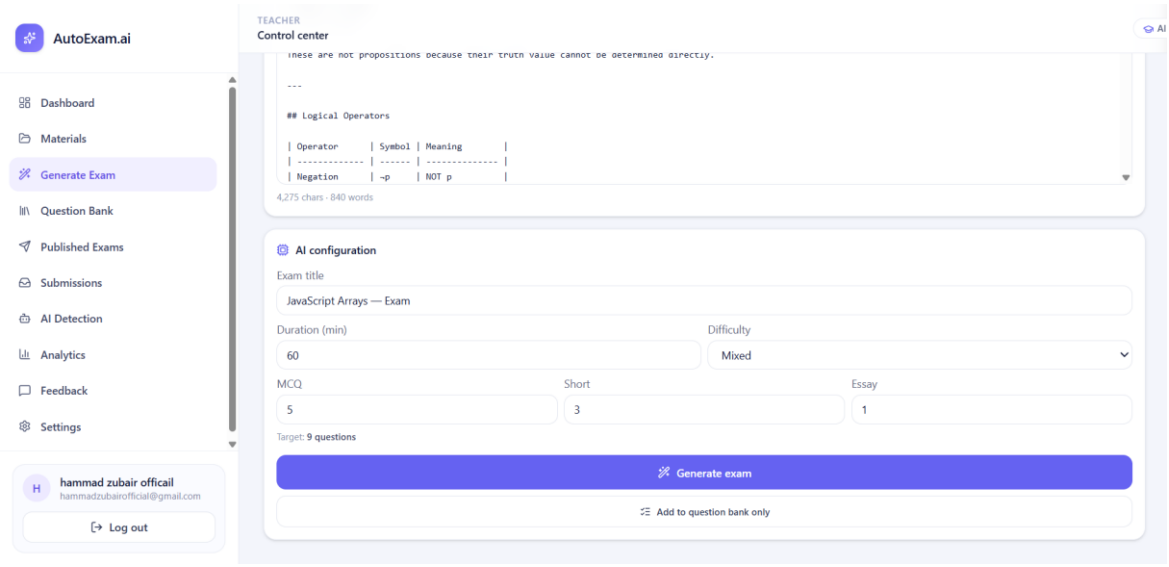
interface provides a dedicated upload section to simplify the content ingestion workflow.





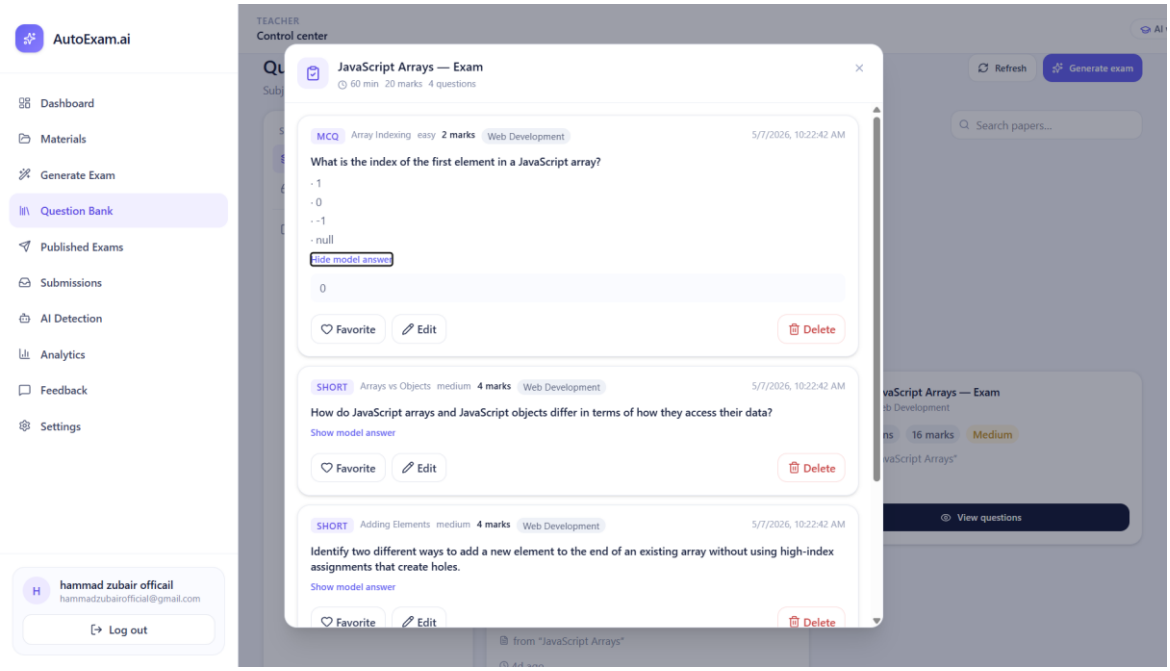
5.6.6 AI Question Generation Page

This prototype represents the AI Question Generation module of AutoExam.ai, where the system automatically generates examination questions from uploaded material. The interface allows teachers to configure question types, difficulty levels, and assessment formats. The module utilizes AI models and Bloom's Taxonomy principles to create intelligent and curriculum-aligned questions.



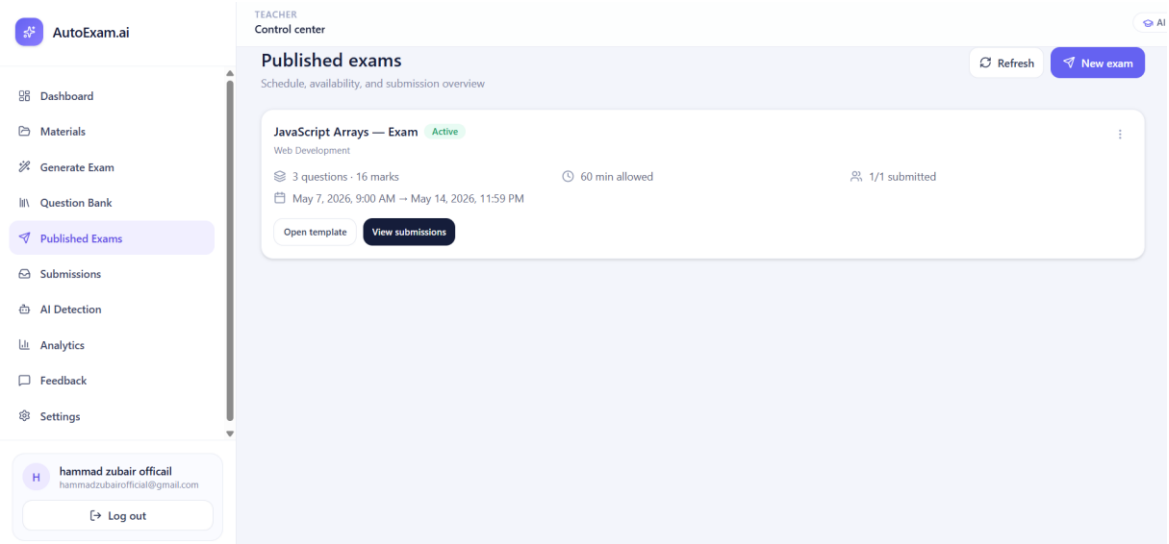
5.6.7 Generated Exam Review Page

This prototype shows the Generated Exam Review Page, where teachers can preview and manage AI-generated examination questions before publishing. The interface allows editing, deleting, and rearranging questions to ensure assessment quality and accuracy. This module provides human supervision within the automated exam generation process.



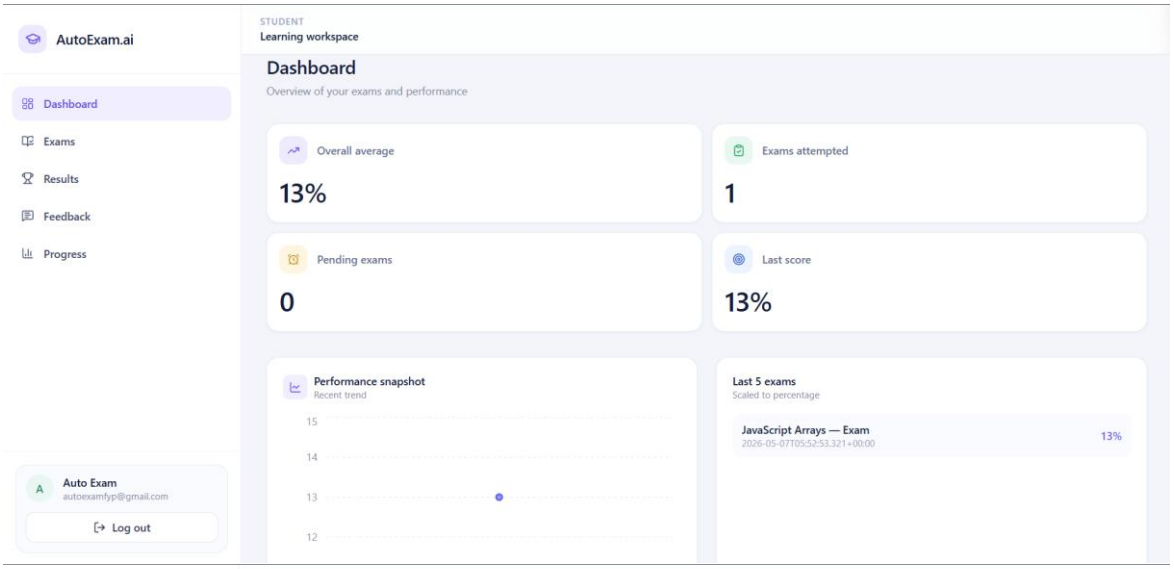
5.6.8 Publish Exam Page

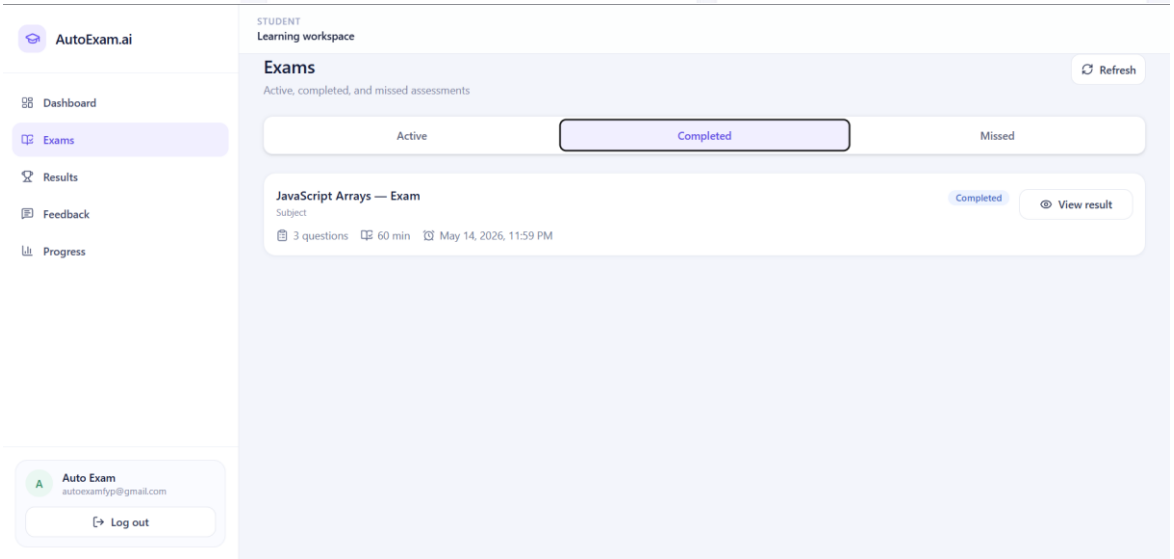
This prototype represents the Publish Exam Page, where teachers can configure and publish examinations for students. The interface allows setting exam duration, total marks, availability time, and submission rules before finalizing the assessment. This module ensures controlled and organized exam distribution within the system.



5.6.9 Student Exam Attempt Interface

This prototype illustrates the Student Exam Attempt Interface, where students can participate in online assessments through a responsive examination environment. The page includes question sections, timers, navigation controls, and automatic answer-saving functionality. The interface is designed to provide a smooth and secure examination experience for students.





5.7 Challenges During Implementation

Lack of Domain-Specific Reference Systems

At the initial stage of the project, there were very limited academic examination platforms that integrated AI-based question generation, automated evaluation, and AI-detection mechanisms together. Due to the absence of suitable reference systems, it was difficult to finalize the system workflow, UI structure, and intelligent assessment features for the proposed platform.

5.7.1 Limited Availability of Educational Training Data

One of the major challenges faced during implementation was the unavailability of high-quality educational datasets for training and testing AI models. Most publicly available datasets were either incomplete or unrelated to academic assessment systems. This limited the accuracy and effectiveness of automated evaluation and feedback generation modules.

5.7.2 Computational Resource Constraints

The AI models and NLP-based processing modules required significant computational resources during development and testing. However, the project relied mainly on free-tier cloud platforms and limited local hardware resources. This created difficulties in running large-scale processing, testing multiple AI models, and optimizing system performance efficiently.

5.7.3 Difficulties in Real-Time Examination Testing

Testing the platform with real examination environments and student responses was another major challenge. Due to limited access to actual academic data and participants, most testing was performed using manually created sample papers and mock student submissions. As a result, validating real-world performance and scalability became comparatively difficult.

5.7.4 Performance Issues with Free Cloud Services

The system utilized free-tier backend and database services during development, which occasionally caused delays in file processing, AI response generation, and data retrieval operations. These performance limitations affected the overall responsiveness of the platform, especially during simultaneous testing of multiple examination workflows.

5.7.5 5.6.6 Financial and Resource Limitations

As undergraduate students, managing the financial requirements of AI APIs, hosting services, and cloud-based tools was challenging throughout the project lifecycle. Most development and deployment expenses were handled personally by the team members, while institutional support and paid infrastructure resources remained limited during implementation.

Chapter 6. References

[1] M. a. S. N. A. Heilman, "Good Question Statistical Ranking for Question Generation," in *NAACL-HLT 2010 — Human Language Technologies Conference*, 2010.

[2] X. S. J. a. C. C. Du, "Learning to Ask: Neural Question Generation for Reading Comprehension," in *55th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2017.

[3] N. S. M. B. V. L. R. X. C. a. S. R. Keskar, "CTRL: A Conditional Transformer Language Model for Controllable Generation," *arXiv preprint arXiv:1909.05858,* 2019.

[4] T. B. e. a. Brown, "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[5] L. W. a. K. D. R. Anderson, A Taxonomy for Learning Teaching and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives, Longman — Pearson Education, 2001.

[6] R. G. S. G. P. a. D. P. Ghuge, "Automatic Question Generation System Using Natural Language Processing," *International Journal for Research in Applied Science and Engineering Technology (IJRASET),* vol. 9, p. 1041, 2021.

[7] G. L. J. P. B. S. U. a. A.-E. S. Kurdi, "A Systematic Review of Automatic Question Generation for Educational Purposes," *International Journal of Artificial Intelligence in Education,* vol. 30, p. 121, 2020.

[8] J. Burstein, "The e-rater Scoring Engine: Automated Essay Scoring with Natural Language Processing," In: Automated Essay Scoring: A Cross-Disciplinary Perspective — Lawrence Erlbaum Associates, 2003, p. 121.

[9] B. H. A. C. A. Z. T. a. L. C. Riordan, "Investigating Neural Architectures for Short Answer Scoring," in *12th Workshop on Innovative Use of NLP for Building Educational Applications (BEA) — ACL*, 2017.

[10] A. a. E. M. Mizumoto, "Exploring the Potential of Using an AI Language Model for Automated Essay Scoring," *Research Methods in Applied Linguistics,* vol. 2, 2023.

[11] E. L. Y. K. A. M. C. D. a. F. C. Mitchell, "DetectGPT: Zero-Shot Machine-Generated Text Detection Using Probability Curvature," in *40th International Conference on Machine Learning (ICML)*, 2023.

[12] E. a. C. S. Tian, "GPTZero: Towards Detection of AI-Generated Text Using Zero-Shot and Supervised Methods," 2023.

[13] S. S. H. a. R. A. M. Gehrmann, “GLTR: Statistical Detection and Visualization of Generated Text,” in *57th Annual Meeting of the Association for Computational Linguistics (ACL) — System Demonstrations*, 2019.

[14] J. a. T. H. Hattie, “The Power of Feedback,” *Review of Educational Research,* vol. 77, no. SAGE Publications, p. 81, 2007.

[15] K. VanLehn, “The Relative Effectiveness of Human Tutoring Intelligent Tutoring Systems and Other Tutoring Systems,” *ArticleEducational Psychologist — ,* vol. 46, no. Taylor & Franci, p. 200, 2011.

Appendix A

Hammad Zubair

AutoExam AI

Quick Submit

Quick Submit

COMSATS University Islamabad, Lahore Campus

Document Details

Submission ID

trn:oid:::1:3565348337

Submission Date

May 10, 2026, 12:43 PM GMT+5

Download Date

May 10, 2026, 1:33 PM GMT+5

File Name

plag_copy.docx

File Size

1.0 MB

81 Pages

17,254 Words

97,676 Characters

2% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

4 AI-generated only 2%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.



False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

Turnitin Originality Report

Processed on: 10-May-2026 12:44 PM PKT
ID: 2957216316
Word Count: 17254
Submitted: 1

AutoExam AI By Hammad Zubair

Similarity Index	Similarity by Source	
10%	Internet Sources:	3%
	Publications:	2%
	Student Papers:	9%

3% match (student papers from 23-Apr-2026)
Submitted to The University of the West of Scotland on 2026-04-23

1% match (student papers from 03-Feb-2026)
Submitted to Higher Education Commission Pakistan on 2026-02-03

1% match (student papers from 14-Dec-2025)
Submitted to COMSATS University Islamabad, Lahore Campus on 2025-12-14

< 1% match (student papers from 19-May-2016)
Submitted to Higher Education Commission Pakistan on 2016-05-19

< 1% match (student papers from 08-Dec-2025)
Submitted to Higher Education Commission Pakistan on 2025-12-08

< 1% match (student papers from 23-Dec-2025)
Submitted to COMSATS University Islamabad, Lahore Campus on 2025-12-23

< 1% match (student papers from 22-Apr-2026)
Submitted to Islington College,Nepal on 2026-04-22

< 1% match (student papers from 22-Apr-2026)
Submitted to Islington College,Nepal on 2026-04-22

< 1% match (student papers from 30-Apr-2026)
Submitted to Islington College,Nepal on 2026-04-30

< 1% match (student papers from 22-Apr-2026)
Submitted to Islington College,Nepal on 2026-04-22

< 1% match (student papers from 22-Apr-2026)
Submitted to Islington College,Nepal on 2026-04-22

< 1% match (student papers from 21-Apr-2026)
Submitted to Islington College,Nepal on 2026-04-21

< 1% match (student papers from 22-Apr-2026)
Submitted to Islington College,Nepal on 2026-04-22

< 1% match (student papers from 22-Apr-2026)
Submitted to Islington College,Nepal on 2026-04-22

< 1% match (student papers from 22-Apr-2026)
Submitted to Islington College,Nepal on 2026-04-22

< 1% match (Internet from 04-May-2026)
http://dspace.daffodilvarsity.edu.bd:8080/bitstream/handle/123456789/16823/P30052.pdf?isAllowed=y&sequence=1

< 1% match (student papers from 30-Nov-2024)
Submitted to University of Greenwich on 2024-11-30

< 1% match (Internet from 22-Jul-2024)
https://www.mdpi.com/2673-2688/5/3/53

< 1% match (student papers from 27-Aug-2025)
Submitted to University of SWAT on 2025-08-27

< 1% match (student papers from 04-May-2026)
Submitted to University of Ulster on 2026-05-04

< 1% match (Internet from 23-Aug-2020)
https://mafiadoc.com/learn-english-vocabulary-and-writing_5c1376ca097c47ab078b4a9f.html

< 1% match (Internet from 13-Jan-2026)
https://portalcris.vdu.lt/server/api/core/bitstreams/f81e686f-c9c1-448b-8ac0-6a8d794e0bd5/content

< 1% match (student papers from 09-Mar-2026)
Submitted to University of Wales Institute, Cardiff on 2026-03-09

< 1% match (student papers from 08-Mar-2026)
Submitted to University of Wales Institute, Cardiff on 2026-03-08

< 1% match (student papers from 09-Aug-2024)
Submitted to De Montfort University on 2024-08-09

< 1% match (Internet from 30-Dec-2020)
https://www.springerprofessional.de/mining-disinformation-and-fake-news-concepts-methods-and-recent-/18093432

< 1% match (student papers from 19-Apr-2024)
Submitted to University of Durham on 2024-04-19

< 1% match (Internet from 06-Oct-2020)
https://www.cambridge.org/core/journals/natural-language-engineering/article/automatic-generation-of-short-answer-questions-for-reading-comprehension-assessment/701D7F67BE5C8DBEE60C7906C5E0E437

< 1% match (student papers from 16-Feb-2019)
Submitted to City University on 2019-02-16

< 1% match (student papers from 14-Jan-2026)
Submitted to universiliteknologimara on 2026-01-14

< 1% match (Internet from 23-Dec-2022)
https://www.arxiv-vanity.com/papers/2104.08801/

< 1% match (Internet from 21-Jun-2022)
https://www.researchgate.net/profile/Nguyen-Giang-18/publication/354075854_Evaluating_Feasibility_and_Effectiveness_of_Digital_Game-Based_Instructional_Technology/links/6125ddc50c2bfa282a6b5e95/Evaluating-Feasibility-and-Effectiveness-of-Digital-Game-Based-Instructional-Technology.pdf#page=251

< 1% match (Hilman Jihadi, Dade Maulana, Harjo Baskoro. "Construction And Design of A Web Information System For Extracurricular Activities In High Schools In Bogor City-Indonesia", Digitus : Journal of Computer Science Applications, 2024)
Hilman Jihadi, Dade Maulana, Harjo Baskoro. "Construction And Design of A Web Information System For Extracurricular Activities In High Schools In Bogor City-Indonesia", Digitus : Journal of Computer Science Applications, 2024

< 1% match (Internet from 01-Dec-2023)
https://export.arxiv.org/pdf/2210.06104

< 1% match (Internet from 23-Dec-2024)
https://www.slideshare.net/slideshow/development-of-system-for-generating-questions-answers-distractors-using-transformers/267891776

< 1% match (student papers from 14-Jun-2025)
Submitted to Harrisburg University of Science and Technology on 2025-06-14

< 1% match (student papers from 06-Jan-2026)
Submitted to University of Teesside on 2026-01-06

< 1% match (Internet from 13-Jul-2023)
https://www.sciencegate.app/document/10.3233/shti210719

< 1% match (student papers from 18-Mar-2026)
Submitted to Purdue University on 2026-03-18

< 1% match (Internet from 12-Dec-2022)
http://repository.cuilahore.edu.pk/xmlui/bitstream/handle/123456789/3275/7503..pdf?isAllowed=y&sequence=1

< 1% match (student papers from 14-Dec-2023)
Submitted to UC, Irvine on 2023-12-14

< 1% match (Internet from 06-May-2026)
https://technobelieve.com/blog/ai-automation-failure-prevention-2

< 1% match (Internet from 29-Jun-2025)
https://www.arcjournals.org/pdfs/ijrscse/v11-i2/1.pdf

< 1% match (student papers from 26-Apr-2026)
Submitted to Vellore Institute of Technology on 2026-04-26

< 1% match (Internet from 18-Apr-2019)
https://hci.stanford.edu/publications/2015/PeerStudio/thesis.pdf

< 1% match (student papers from 13-Apr-2025)
Submitted to Hasan Kalyoncu Üniversitesi on 2025-04-13

< 1% match (Nunes, Diogo. "System for Pratical Evaluations in Network Administration Courses.", Universidade do Porto (Portugal))
Nunes, Diogo. "System for Pratical Evaluations in Network Administration Courses.", Universidade do Porto (Portugal)

< 1% match (Internet from 29-May-2025)
https://link.springer.com/article/10.1007/s40593-019-00186-y?code=ca9acdc6-dddd-428c-a94c-d9a88ecff779&error=cookies_not_supported

< 1% match (Internet from 10-Oct-2022)
http://eprints.utar.edu.my/3956/1/18ACB03446_FYP.pdf

< 1% match (Internet from 12-Jan-2025)
https://WWW.coursehero.com/file/82798164/Sample-Ramesh-Reportdoc/

< 1% match (Internet from 07-Feb-2026)
https://www.leespring.fr/fr/decouvrez-les-mini-ressorts-bantam

< 1% match ("Natural Language Processing and Information Systems", Springer Science and Business Media LLC, 2023)
"Natural Language Processing and Information Systems", Springer Science and Business Media LLC, 2023